



**SAPIENZA**  
UNIVERSITÀ DI ROMA

INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E STATISTICA

**Analyzing Deployment Strategies for  
Autonomous System Provider  
Authorization (ASPA) in BGP Security**  
CYBERSECURITY

**Professors:**

Prof. Marco Polverini

**Students:**

Yigit Hakverdi

---

Academic Year 2024/2025

# Contents

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                         | <b>3</b>  |
| 1.1      | Background on BGP Security Issues . . . . .                                 | 6         |
| 1.2      | Overview of Route Leaks and Prefix Hijacking Problems . . . . .             | 7         |
| 1.3      | Current solutions and limitations (RPKI, BGPsec) . . . . .                  | 11        |
| 1.4      | Research Objectives: Analysis of ASPA Deployment Strategies . . . . .       | 13        |
| <b>2</b> | <b>Background and Related Work</b>                                          | <b>13</b> |
| 2.1      | Link State and Distance Vector . . . . .                                    | 14        |
| 2.2      | Fundamentals of BGP and Autonomous Systems . . . . .                        | 16        |
| 2.3      | IP Classes and CIDR Notation . . . . .                                      | 17        |
| 2.4      | BGP Session Establishment . . . . .                                         | 19        |
| 2.5      | BGP Messages Structure and Types . . . . .                                  | 20        |
| 2.6      | BGP Data Structures . . . . .                                               | 21        |
| 2.7      | BGP Path Attributes . . . . .                                               | 22        |
| 2.8      | BGP Route Selection Process . . . . .                                       | 24        |
| 2.9      | BGP From Perspective of ISPs . . . . .                                      | 25        |
| 2.10     | Gao-Rexford and Valley-Free Routing . . . . .                               | 29        |
| 2.11     | Security Properties . . . . .                                               | 31        |
| 2.12     | Early security mechanisms such as TCP, Hashing, MD5 etc. . . . .            | 33        |
| 2.13     | BGP Security Mechanisms (BGP-Sec, RPKI, ROV etc.) . . . . .                 | 33        |
| 2.14     | BGPsec and RPKI Limitations . . . . .                                       | 36        |
| 2.15     | BGP Attacks, Route Leaks, Route Hijacks . . . . .                           | 37        |
| 2.16     | Path Plausibility and Path Selection Algorithms . . . . .                   | 40        |
| 2.17     | Autonomous System Provider Authorization (ASPA) . . . . .                   | 41        |
| 2.18     | Related Research . . . . .                                                  | 45        |
| 2.18.1   | BGP Security Related Papers . . . . .                                       | 45        |
| 2.18.2   | BGP Simulation Related Papers . . . . .                                     | 46        |
| <b>3</b> | <b>Methodology and Experimental Design</b>                                  | <b>47</b> |
| 3.1      | Simulation Framework Architecture . . . . .                                 | 47        |
| 3.2      | Core Design Principles (Plugin System, Event Driven Architecture) . . . . . | 52        |
| 3.3      | Event System and Module Communication . . . . .                             | 55        |
| 3.4      | Topology Management . . . . .                                               | 58        |
| 3.5      | Protocols (plugins) Implemented Base, Route Leak ASPA . . . . .             | 61        |
| 3.6      | Route Operations . . . . .                                                  | 66        |

|          |                                                                                                                   |           |
|----------|-------------------------------------------------------------------------------------------------------------------|-----------|
| 3.7      | Deployment Strategy Analysis: Random Deployment Approach and Selective Deployment (top-down, bottom-up) . . . . . | 70        |
| 3.8      | Metrics and Evaluation Criteria . . . . .                                                                         | 72        |
| <b>4</b> | <b>Technical Implementation and Results</b>                                                                       | <b>73</b> |
| 4.1      | Deployment Strategy Results . . . . .                                                                             | 73        |
| 4.2      | Critical Mass Threshold . . . . .                                                                                 | 77        |
| <b>5</b> | <b>Technical Implementation and Results</b>                                                                       | <b>78</b> |
| 5.1      | Deployment Strategy Effectiveness . . . . .                                                                       | 78        |
|          | <b>References</b>                                                                                                 | <b>83</b> |

# 1 Introduction

BGP (Border Gateway Protocol) is the routing protocol that facilitates communication between different networks on the internet by interconnecting Autonomous Systems (ASes). To visualize this, imagine the internet as a vast network of nodes, where each node is an AS and the links between them form a complex, graph-like structure. Before delving into a detailed discussion of BGP's mechanisms and its security challenges, it is essential to first understand what an Autonomous System is and how it functions as the backbone of interdomain routing. In the sections that follow, this thesis will examine the role and structure of ASes, explore the various types of AS relationships and routing policies, and then transition into an analysis of BGP's vulnerabilities including issues like route leaks and prefix hijacking. This foundation will set the stage for a subsequent discussion on security enhancements such as ASPA deployment, Resource Public Key Infrastructure (RPKI), and other strategies aimed at bolstering BGP's resilience. An Autonomous System (AS) is formally defined as a collection of routers under the control of a single administrative authority [1]. Within the AS, routers use an interior gateway protocol (IGP) to manage routing internally. For communication with networks outside the AS, an exterior gateway protocol (such as BGP) is employed. This distinction between internal and external routing is fundamental to how the internet operates. Each AS, is identified with a unique AS number (ASN) [2], such as, Google's ASN is AS15169 [3], one of the largest providers, serving to billions of users and hosts, and has a very high customer-to-provider ratio (or customer degree), thus it primarily operates as content provider, it does not need to rely on many upstream providers [4], this of course stems from massive user activity on their major services, Google Search Engine, YouTube, and other platforms [5]. Another major content provider is Meta. Meta's global network infrastructure relies heavily on BGP (Border Gateway Protocol) to manage routing and connectivity across its vast ecosystem of Points of Presence (PoPs), data centers. BGP enables Meta to efficiently route traffic between its Autonomous Systems (AS32934 and others) [6] [7] and external networks, ensuring delivery of content and services to billions of users worldwide. Meta operates numerous PoPs strategically located around the globe, which act as interconnection points with internet service providers (ISPs). These PoPs, combined with Meta's proprietary Content Delivery Network (CDN), allow the company to cache content closer to users, reducing latency and improving performance. BGP plays a critical role in dynamically routing traffic through these PoPs, optimizing paths based on real-time network conditions and ensuring high availability. [8] [9] [10] With such a robust infrastructure in place, it becomes essential to understand how the global internet is structured through the interconnection of various Autonomous Systems (ASes). Depending on traffic flows, an AS may carry transit traffic, acting as a transit AS, or handle only local traffic, in which case it is classified as a stub AS. Some ASes, however,

may choose not to carry transit traffic at all and are referred to as multihomed ASes. One of BGP’s key strengths is its flexibility: rather than enforcing a fixed topology, it leverages the AS path attribute to prevent loops and allow for dynamic, policy-based route selection. In effect, the internet can be modeled as a vast graph, where ASes serve as nodes interconnected by BGP links, with each AS type transit, stub, or multihomed playing its part in the overall routing ecosystem. [1] [11] BGP provides capabilities beyond basic route advertisement by enabling robust policy enforcement. Policy in BGP acts as an extension of the protocol’s configuration, allowing administrators to influence route selection and filtering based on traffic characteristics and inter-AS relationships. Typically, these policies let an Autonomous System (AS) control route propagation and acceptance according to three primary factors: political, security, and economic considerations. For example, an AS might be restricted from accepting traffic from certain sources due to regulatory or security concerns, or it might choose to avoid routing traffic through specific regions. Conversely, an AS can agree to utilize or even prioritize certain neighboring ASes based on performance or cost criteria. Additionally, quality checks, such as preferring routes with shorter AS paths, help determine the most efficient route among several alternatives. [12] [13]

There is an abundance of research aimed at establishing a fundamental framework for understanding AS behavior under various policy configurations and how this affects routing and information exchange in the BGP network. Foundational models such as the Gao-Rexford framework and the valley-free routing assumption have been developed to formalize these relationships. However, recent studies [?] have shown that these models have limitations in capturing the full complexity of contemporary AS interactions. Lixin Gao and Jennifer Rexford [14] introduced a foundational guideline for Autonomous Systems (ASes) to consider when making routing decisions. Their approach allows ASes to establish forwarding rules based solely on their local business view information about their immediate neighborhood without requiring additional data from ASes further along the path. The Gao-Rexford model ensures route convergence and provides flexibility in policy configurations by relying on pure business relationships between ASes. However, this model has sparked extensive discussions and raised concerns about its ability to accurately reflect real-world AS relationships and the broader structure of the internet. [15] [16] [17] [18] While the Gao-Rexford framework provides a rigorous basis for ensuring routing stability through business relationship-based policies, it is important to recognize that these guidelines operate locally. In other words, each AS enforces its own policies based on its immediate economic and technical considerations. As a result, even though the valley-free principle helps prevent anomalies, for example, ensuring that routes learned from providers or peers are only advertised to customers, the decentralized decision-making inherent in BGP can still lead to situations where the global routing

state becomes unstable. This phenomenon, known as Internet routing divergence (this can result in issues like routing loops, where packets circulate endlessly between ASes, or suboptimal routing, where traffic takes longer or less efficient paths), occurs when the routing system fails to settle on a consistent set of paths, leading to oscillations, routing loops, or inefficient traffic flows. Numerous studies have explored this tension between local policy enforcement and the need for global convergence [19] [20] [21] Gao and Rexford’s work laid the foundation for understanding how these local business relationships might guarantee stability under idealized conditions. However, subsequent research such as work on the limitations of the valley-free model and investigations into routing dynamics under conflicting policies has shown that real-world interdomain routing is often more complex. [22] [23] Factors such as misconfigurations, dynamic traffic demands, and heterogeneous policy preferences can disrupt the neat partial ordering envisioned by Gao and Rexford, leading to divergence in the network. Every now and then, routing divergence can pop up in some networks, but it’s not something you see too often. This is largely because network engineers keep a close eye on BGP setups and follow frameworks like the Gao-Rexford model to maintain stable routing. They also rely on steps like route aggregation and filtering to keep the number of routes manageable and avoid clashes in the routing data. Still, a single misconfiguration or an unanticipated shift in network topology can trigger short-lived or localized divergence, emphasizing the need for prudent oversight. On the whole, the internet’s routing infrastructure tends to be sturdy enough to prevent significant disruptions from routing divergence. While the Gao-Rexford framework shows how business relationships between customer-provider and peer-peer can guide policy enforcement to stabilize BGP, another fundamental mechanism that underpins routing stability is the structured use of IP prefixes. By breaking the vast Internet address space into hierarchically organized prefixes (e.g., 192.168.1.0/24), network operators can perform route aggregation, thereby reducing the number of individual routes that must be managed and propagated. [24] This process not only eases the burden on global routing tables but also minimizes the risk of routing divergence by limiting conflicting or redundant route information. In effect, the aggregation of prefixes supports the same hierarchical principles found in the Gao-Rexford model where routes learned from customers, peers, and providers are treated differently to maintain a coherent and convergent routing system. While the aggregation of IP prefixes reduces routing table size and helps mitigate divergence, the path-vector nature of BGP allows for sophisticated routing strategies that go well beyond simply choosing the shortest path. Mechanisms such as local preference settings and community attributes enable an Autonomous System (AS) to influence route selection on both incoming and outgoing traffic. [25] [26] [27] These techniques allow an AS to prioritize one route over another, advertise routes differently to neighbors, or even manipulate the AS path such as by artificially extending it to discourage traffic. Autonomous System

(AS) can give preference to one route over another by adjusting a local preference setting. Additionally, BGP enables an AS to influence how its neighbors select routes by using community attributes to indicate preferred paths. To control incoming traffic, an AS can assign different values to the route advertisements it sends through each connection to a neighboring AS. Without these adjustments, the neighboring AS would select the connection based on its own internal routing methods. Building on the stability provided by hierarchical prefix aggregation and the Gao-Rexford framework, the next layer of BGP's robustness emerges from how individual routers handle BGP advertisements. In other words, while route aggregation reduces global table sizes and limits conflicting route information, the practical enforcement of routing policies depends on three critical steps: applying import policies to decide which routes to accept, selecting the best route using attributes like AS path length and local preference, and employing export policies to control which routes are shared with neighboring ASes.

## 1.1 Background on BGP Security Issues

While BGP's flexibility and policy-based routing have been instrumental in scaling the internet, its design also introduces significant security vulnerabilities. The protocol's reliance on trust between ASes and its lack of built-in mechanisms for verifying the authenticity of routing information make it susceptible to attacks such as route hijacking and route leaks. These vulnerabilities can lead to traffic being misrouted, intercepted, or dropped, posing serious risks to network reliability and data integrity. As the internet continues to grow, addressing these security challenges has become a critical focus, prompting the development of measures like RPKI and BGP-Sec to enhance the security and resilience of BGP. In today's interconnected world, the system that coordinates how data travels between different networks remains dangerously exposed, largely because of the trust-based structure at the heart of its underlying protocol. Instead of simply verifying credentials or confirming the legitimacy of routes, the protocol simply assumes that each participant, or Autonomous System (AS), will provide truthful and accurate information. Attackers can exploit weaknesses by injecting bogus route data, removing legitimate pathways, or subtly tweaking routing details to redirect information for their own benefit. Such actions can cause anything from service slowdowns and poor network performance to large-scale disruptions that knock entire services offline and fracture global connectivity. Beyond just suboptimal routing, these attacks can enable eavesdropping, or even the complete severing of certain internet segments. Worse yet, once a route has been compromised, detecting the manipulation can be challenging, allowing attackers to persist undetected and potentially escalate their activities over time. The current research landscape in BGP primarily focuses on several key areas: route propagation, BGP scalability, and the scalability of security

measures. Researchers are also exploring various methodologies to extend and improve BGP security mechanisms, aiming to facilitate easier adoption and ensure consistent results even in partially deployed environments. These efforts are critical to addressing the challenges of securing BGP while maintaining its efficiency and global scalability. Given that BGP's design traditionally emphasizes openness over secrecy, it is worth to note that while authenticity and authorization (ensuring routes come from legitimate ASes and providers) are well-addressed by mechanisms such as RPKI and ASPA, integrity is typically enforced via cryptographic techniques in proposals like BGP-Sec, and confidentiality remains an emerging concern in some contexts

## 1.2 Overview of Route Leaks and Prefix Hijacking Problems

Examining the security metrics reveals that the underlying issues give rise to two critical concerns in BGP; route leaks and prefix hijacking. [28] [29] These vulnerabilities are central to contemporary research in BGP security. To understand why route leaks occur and what constitutes a route leak, we can examine several similar incidents previously classified as route leaks and develop a robust classification system for better comprehension. One of the most significant events that has spurred extensive research in this field is the Pakistan YouTube incident. Researchers and professionals have highlighted that such incidents can be weaponized to "steal" routing, traffic, and transit paths from legitimate sources. [30] [31] [32] [33] As explained before inter-AS routing depends on three major driving factors, economical (i.e. business relationships and being the dominant factor) and political and security. Route leaks occur due to violation of these business relationships, and usually due to configuration issues. One example of route leak is Dodo Network announcing to its provider. The issue likely stemmed from an accidental misconfiguration in the Dodo network (AS38285), where an edge router began announcing its internal routes to Telstra. Being a customer of Telstra, Dodo's routes were prioritized over those from Telstra's peers and providers, in line with Telstra's routing policy. As a multi-homed network, Dodo connects not only to Telstra but also to Optus (AS7474), PIPE Internet Exchange (AS18398), and Equinix (AS24115). BGP logs from February 23, 2012, show the first irregular route re-advertised by Telstra at 02:39 UTC. [34] [35] [36] Telstra's preference for Dodo's customer routes led to disruptions, as Dodo's announcements included routes from Optus and full transit routes from other providers, eventually encompassing the entire Internet. Telstra propagated these routes to its customers and peers, causing widespread connectivity issues, including for its transit provider, Telstra International (AS4637). The problem persisted for 46 minutes, with partial withdrawals and reannouncements exacerbating the situation. The event highlights the cascading failures triggered by misconfigured routing policies in complex, inter-connected networks. Although potential route leaks occur on a regular basis, most slip by unnoticed and cause no

harm. In contrast, the incident described here turned into a far more serious disruption. [37] You might be wondering why some route leaks go unnoticed while others wreak havoc. The answer lies primarily in the presence (or absence) of properly implemented filtering mechanisms. Under normal circumstances, Internet service providers use input routing filters to strictly confine each customer's advertised routes to those that have been authorized. In other words, if a customer tries to announce any routes beyond what the provider has agreed to carry, the filter steps in and discards those unauthorized prefixes before they propagate further. This practice is considered a core component of Border Gateway Protocol (BGP) best practices, often highlighted in recommendations like those found in RFC 7454. To comment on this, in this particular episode, all signs point to missing filters between Telstra and Dodo. One plausible (and highly likely) explanation is that Telstra did not have an input filter to screen Dodo's announcements, and Dodo similarly lacked an output filter. This gap in basic BGP hygiene can be especially problematic when dealing with extensive route tables. Indeed, Dodo is known to originate slightly more than a /11 in size advertising 843 separate prefixes and also provides transit for five other networks, adding another 252 prefixes when passing routes to Telstra. Altogether, that's 1095 prefixes that should ideally be governed by per-prefix filtering. It's possible that the size of this routing table surpassed the capacity of their operational support tools, leading to filters being omitted or overlooked. This is, of course, speculative, but it aligns with the observable outcome that no filtering was in effect during the incident. When Dodo changed its route configuration, its internal routes escaped into Telstra's routing table. Telstra subsequently believed that the best path to the entire Internet was via Dodo, redirecting all its outbound traffic through this newly learned (but incorrect) path. That misdirection didn't stop there. Telstra also announced these routes to its own customers, peers, and transit providers. As a result, the error rippled outward, leaving multiple networks grappling with service disruptions. Route leak occurrences are relatively simpler compared to their solutions. According to the Gao-Rexford model, valley-free routing policies, and similar guidelines, achieving a cycle-free and stable routing environment requires local decision-making by Autonomous Systems (ASes). Each AS determines its routing preferences based on local considerations, which are primarily influenced by business or economic factors. As a result, ASes lack a global view of the network topology; their perspective is confined to their immediate neighbors. In summary, preventing route leaks presents two main challenges. The first is implementing a global registry, which is difficult due to the internet's vast, dynamic nature. Keeping up with the rapid pace of traffic exchange, bandwidth fluctuations, and topology updates makes this approach impractical. The second is developing a solution based on the local view of routers, but a viable method leveraging this approach has yet to be proposed [38]. Part of the difficulty stems from the fact that inter-AS relationships and internal policies are rarely disclosed, so

when these leaks occur, there is often insufficient visibility to confirm or diagnose the policy violations that precipitated them. Moreover, while route filters, Internet Route Registries (IRRs), and BGP monitoring tools are valuable lines of defense, they suffer from inherent limitations such as high operational overhead and dependence on data sources that can be incomplete, outdated, or simply missing. This lack of precise, current information frequently leads to gaps in coverage, allowing route leaks to slip through uncorrected and sometimes escalate into significant disruptions. Another major security issue with the BGP is prefix hijacking. Prefix hijacking is a severe security issue in the Border Gateway Protocol (BGP) that arises when an Autonomous System (AS) illegitimately announces IP prefixes owned by another AS. This deception causes internet traffic intended for the original AS to be rerouted through or terminated by the malicious AS. Attackers leverage this vulnerability to intercept, eavesdrop, or disrupt communications. In some cases, such as the infamous Pakistan-YouTube incident, prefix hijacking has resulted in global-scale outages, disrupting services far beyond the immediate victim [39] [40]. The root cause again, often lies in inadequate filtering mechanisms, which fail to verify the authenticity of prefix advertisements. Effective countermeasures, such as prefix filtering and cryptographic validation through Resource Public Key Infrastructure (RPKI), are essential to mitigate these risks and preserve the integrity of global internet routing. To provide a clear understanding of prefix hijacking, real-life examples can offer valuable insight. Here are a few notable incidents:

In February 2013, a sequence of highly sophisticated BGP prefix hijacking incidents was detected, with traffic rerouted through Belarusian ISP GlobalOneBel. Each hijacking event lasted between minutes and several hours, occurring almost daily. The affected traffic included sensitive data belonging to financial institutions, government agencies, and international service providers from countries like the US, South Korea, and Germany. The attack mechanism involved GlobalOneBel advertising false AS paths for a range of IP prefixes. For example, a route originating in Guadalajara, Mexico, destined for Washington, DC, was hijacked. Normally, this route would have traversed PCCW (AS3491) and Qwest/CenturyLink (AS209). However, attackers advertised a malicious route that traversed PCCW, Level3 (AS3356), and Russia's TransTelecom (AS20485), ultimately reaching Belarus Telecom (AS6697). The traffic was inspected or modified at Belarus Telecom before being returned to the legitimate path via Russian provider ReTN (AS9002), Frankfurt, and finally to its US destination [41] [42]. From a technical perspective, the attack exploited the lack of authentication in BGP announcements, where the advertised AS paths appeared plausible to upstream routers. The attacker ensured a "clean path" for returning traffic, maintaining end-to-end communication without alerting the target. High-latency round-trip times observed during active measurements confirmed the geographic detour of traffic through Minsk

and Moscow. These hijacks highlighted vulnerabilities in route validation mechanisms and the global dependency on trust-based BGP routing.



Figure 1: BGP Hijacking In Iceland And Belarus

Between July 31 and August 19, 2013, Iceland experienced a series of 17 targeted BGP hijacking events. Icelandic provider Opin Kerfi (AS48685) advertised false routes for 597 IP prefixes belonging to a major US VoIP provider. This was anomalous as Opin Kerfi typically originated only three IP prefixes. The rogue advertisements propagated exclusively through Síminn (AS6677), Iceland's incumbent telecom provider, while avoiding its other transit provider, Fjarskipti (AS12969). One notable case involved traffic originating and terminating in Denver, Colorado. Normally, this traffic would remain within a local routing domain. However, Opin Kerfi's malicious advertisements directed the traffic to Atrato (AS5580) in London, which handed it to Síminn in Iceland. The traffic was then rerouted back to North America via Cogent's Greenland cable, passing through Montreal, Chicago, and New York before finally returning to Denver via Qwest/CenturyLink (AS209). The attackers exploited BGP's lack of origin authentication and routing policy enforcement. Specifically, the propagation of falsified announcements relied on the absence of Route Origin Validation (ROV) at the upstream peers of Síminn. Detailed traceroute data revealed latency spikes and unusual geographic detours consistent with the malicious routes. For example, packet delays of over 200ms were observed as traffic looped through Reykjavik before being restored to the legitimate path [43]. As with route leaks, detecting prefix hijacking is challenging due to similar reasons: the lack of a global view by Autonomous Systems (ASes) and the fact that routing announcements are made based on local perspectives and business relationships. To advance research and develop effective methods for preventing hijacking, researchers have proposed a set of guidelines known as HEAP (Hijacking Event Analysis and Prevention). These guidelines aim to provide a structured approach to identifying, analyzing, and mitigating prefix hijacking incidents, offering a foundation for improving the security and stability of global routing systems [44]. Another example for understanding how and why prefix hijack occurs [45]. In the paper they claim that PHAS has a potential use

case of detecting prefix hijacking with specific prefix announcements. One incident is the hijack attack on AS9121 (TTNet), on 24 Dec 2004. TTNet (AS9121) mistakenly announced an almost complete routing table to its peers, including Telecom Italia (AS6762). Telecom Italia, due to a misconfiguration where no maximum prefix limit was set, accepted over 100,000 prefixes. This error caused Telecom Italia to propagate these excessive prefixes to its peers, overwhelming their systems and triggering their maximum prefix limits. As a result, the "bad" prefixes from TTNet replaced the legitimate routing information from the actual owners, leading to incorrect routing and widespread connectivity disruptions across multiple networks. [46] [47] [48] This chain of events highlights how a single misconfiguration can cascade and impact global routing stability. [49] Until the end of the day AS9121 announced its full routing tables, causing AS6762 to carry out 106606 bad prefixes. AS1299, despite having a relatively low maximum prefix limit configured for AS9121, could not completely avoid the issue. It directly received 1,849 bad prefixes from AS9121 but also collected a total of 10,925 bad prefixes from other peers, including AS6762 and others like AS1239, AS6453, and AS701. Specifically, AS1299 received 4,413 bad prefixes from AS6762, 3,997 from AS1239, 2,522 from AS6453, and 612 from AS701. This illustrates how the misconfigured routes propagated widely through the network, with AS1299 becoming a transit point for the spread of these bad prefixes, amplifying the overall impact of the routing incident.

### 1.3 Current solutions and limitations (RPKI, BGPsec)

Current proposals or methods generally can be categorized into in-band and out-of-band, referring to the methods used for communication, configuration and management of the protocol especially in terms of performance and security. For in-band methods, communication occurs over the same network and infrastructure, used for regular data traffic [?] [?]. In an out-of-band approach, communication and management occur through a dedicated and physically separate network from the one being made. For example, although, anomaly detection systems use the underlying infrastructure as a means of collecting data, based on pure prediction based algorithms they could be included in this category as an out-of-band solution, as if they are a plugin to the underlying BGP infrastructure, or for example some part of the proposed communication might be conducted through HTTPs separate from BGP infrastructure [50] [51]. Contemporary work on BGP concentrates on identifying and tackling both operational and security challenges. In terms of operations, researchers have devoted substantial effort to areas such as scalability, convergence time (the interval needed for every router to maintain a unified perspective of the network), route stability, and performance. Concurrently, much of today's security research focuses on ensuring data integrity, authentication, confidentiality, authorization, and validation within BGP. These two

facets operational considerations and security are deeply interwoven, and progress or setbacks in one sphere offer valuable insights for the other. Most of the proposed work on these issues heavily rely on the presence of a secure trusted party for validating objects cryptographically through certification and attestation processes. This mechanism is provided by the Resource Public Key Infrastructure (RPKI). One of the major proposals that utilized RPKI was BGP-Sec. Earlier solutions proposed such as [52] [53] propose simple modifications and operate within the existing BGP framework, such as using underlying TCP authentication with MD5, MACs, or key management with IPsec etc. primarily addressing the session-level security and localized attacks. However later on overhaul of the underlying BGP security framework designed and architectural changes implemented such as Public Key Infrastructure (PKI), initial solutions struggled with scalability due to challenges like key management, hence architectures such as RPKI adopted for scalability in mind using automated PKI systems. One of the in-band technologies built upon the RPKI is BGP-Sec, to certify IP prefix ownership and AS numbers/ Despite its robust security features, BGP-Sec has faced widespread adoption due to factors such as complexity and performance overhead, operation challenges and incremental deployment difficulties [54]. Similarly to BGP-Sec, these problems could be expanded to include categorization we've made on in-band and out-of-band solutions too, especially the latest approach on architectural changes and adoption of RPKI. ROAs and related security extensions to BGP such as BGP-Sec have proven to be insufficient, one of the main reasons is the adoption, such as, according to a research, FCCs plan on gathering all 9 major providers to create a plan on the future risks and problems, have been shown that major ISPs still up to date don't have a full coverage of RPKI, ROAs such as the biggest ISP, AT&T have approximately 100 ASNs with two of them only having the 98% of the routes AS7018 and AS20057, have 960 of RPKI-valid prefix out of 2995 prefixes, and rest is unknown. In total only 58% of the whole BGP traffic is RPKI-valid [55]. Furthermore RPKI has it's own problems such as downgrade attacks or vulnerabilities in RPKI relaying part (RP), such as path traversal attacks, remote crashes or operational challenges and complexities on RPKI deployments also cause significant security issues [56] [57] [58]. RPKI adoption also could vary significantly according to regional features such as Middle East countries such as Turkey, Israel, Saudi Arabia have over 90% RPKI deployment, this might be due to the market in these countries being monopolised so these type economical and architectural configurations have easier uptakes. And for ARIN zone adoption is relatively lower compared to rest of the world with APNIC, this could be attributed to having larger address space [59]. Network size influences RPKI implementation, with major networks generally taking the lead thanks to their superior resources and technical knowledge. However, regional differences exist - in the Asia-Pacific region (APNIC), smaller networks have achieved similar adoption rates to their larger counterparts. In contrast, North America (ARIN) and Africa (AFRINIC)

show increasing disparities between large and small networks. In Europe (RIPE), while the overall adoption pattern remains steady, smaller networks consistently trail about two years behind larger ones in implementing RPKI.

## 1.4 Research Objectives: Analysis of ASPA Deployment Strategies

The security challenges outlined until here particularly route leaks and forged-origin prefix hijacks stress on the need for improved path validation mechanisms. Current measures such as RPKI and BGP-Sec and several in-band and out-of-band solutions and extension to BGP have highlighted both operational limitations and deployment hurdles [51] [60]. In contrast, the path plausibility algorithm ASPA offers a different perspective by validating inter-AS relationships to decrease these vulnerabilities. ASPA is built on the insight that not all ASes contribute equally to global routing security. Much like the hierarchical principles seen in the Gao-Rexford model, ASPA's impact is maximized when key players with extensive connectivity and high customer-to-provider ratios adopt the algorithm early. This design decision introduces new questions about the minimum deployment threshold necessary to ensure that ASPA can operate effectively on a global scale. The research objective is twofold. First, it seeks to develop an open-source inter-AS routing simulator in C++ that tests various inter-AS protocols including experimental ones to generate quantitative data for rigorous research analysis, as detailed in Chapter 3 on the simulator's extensibility and modularity. Second, it aims to assess how effectively ASPA mitigates route leaks and prefix hijacks under both random and selective (top-down) deployment scenarios. This evaluation will examine performance variations across different deployment thresholds to identify the optimal strategy for early ASPA adoption. In doing so, we aim to provide a different view on broader discourse on enhancing BGP security offering concrete recommendations for network operators and policy makers on how to strategically deploy security enhancements to safeguard the global routing infrastructure. Thesis structure organised as follows; Chapter 2 lays the background for BGP, BGP security measurements (RPKI, BGP-Sec etc.), Chapter 3 provides methodology, providing detailed presentation on the simulator, and provides deployment strategies and experimentations, Chapter 4 provides results, presenting it with different plots, on Chapter 5, discussion on the results is given, and Chapter 6 concludes the thesis.

## 2 Background and Related Work

## 2.1 Link State and Distance Vector

To understand the underlying logic of how BGP works we need to understand distance vector, and link-state routing protocols. First, let's dive into the link-state routing protocol. It's better to explain the LS with its evolution with related papers. Paper that gave start to all of the LS routing protocols is A link-state routing protocol works by giving each node a complete view of the network's topology and link costs, so every node can independently compute shortest paths. Early research by McQuillan et al. pioneered this approach in the ARPANET by having each node measure delays to its neighbors every 10 seconds and flood small "routing updates" containing only those local measurements. Since those updates were short and high-priority, they propagated very quickly, letting all nodes keep databases that were nearly identical. From that database, each node performed a Shortest-Path-First (SPF) calculation (based on Dijkstra's algorithm) to determine best routes. Later refinements appeared in works by Perlman, who introduced more robust link-state packet (LSP) distribution techniques to solve practical issues such as keeping sequence numbers from wrapping around incorrectly when nodes restart or when hardware faults generate out-of-date packets. A key innovation was the "lollipop" sequence number space, which starts from a series of negative values and then transitions to a circular range of positive sequence numbers. By doing this, a node that has just rebooted can begin using a number that is definitively smaller than anything in current circulation, and then rapidly "catch up" if older LSPs still exist in the network. In practice, each node broadcasts link-state packets to all others whenever there are significant changes in delay or topology, typically enforcing an upper bound on how frequently new packets can be issued. The ARPANET design illustrated that if broadcasts happen too often, the overhead becomes problematic, but if updates happen too infrequently, routers risk basing their SPF computations on stale data. To handle old or corrupted link-state advertisements, Perlman's approach specified that each LSP should carry an "age" field decremented at each hop or at fixed time intervals so that stale information naturally expires out of the network.

Flooding rules (new LSPs overwrite old ones, duplicates act as acknowledgments, etc.) helped ensure reliable propagation even if some nodes or links failed intermittently. This approach of using a synchronized yet incrementally updated topology database is also the basis for modern link-state protocols like OSPF. Each node's final view of the network is identical, and each node can run Dijkstra's algorithm to find least-cost paths. The result is the same set of shortest paths across the network, improving routing consistency and reducing the chance of persistent routing loops compared to distance-vector approaches. However, as noted, link-state protocols can still experience transient oscillations, particularly when link costs depend on current traffic loads. Avoiding all nodes recalculating at once or randomizing the timing of updates helps reduce these oscillations. Overall, link-state routing remains a core technology, combining

the rapid dissemination of small local updates from McQuillan’s original ARPANET design and the robustness and fault-tolerance mechanisms described by Perlman [61] [62] [63]. Distance-vector routing protocols operate by having each router (or host acting as a router) maintain a table of distances (or metrics) to every possible destination in the network, updating those distances based on periodic or triggered exchanges with immediate neighbors. These protocols derive from the Bellman-Ford approach, where each node’s distance to a destination is the minimum of its neighbors’ distances plus the cost of reaching those neighbors.

This is typically captured in the Bellman-Ford recurrence,  $D(d) = \min\{c(r, n) + D(d)\}$ , where  $D(d)$  is node  $r$ ’s current cost estimate for destination  $d$ , and  $c(r, n)$  is the cost of the link from  $r$  to neighbor  $n$ . Unlike link-state routing, where each node obtains a full map of the network and runs a shortest-path calculation locally, distance-vector protocols require nodes only to broadcast their distance vectors and to recompute new distances upon receiving neighbors’ updates. This iterative process converges as each node learns better (i.e., smaller) metrics over time, with the Bellman-Ford guarantee that those estimates stabilize to shortest paths, assuming there are no persistent changes or failures. If a route becomes invalid say a neighbor or link fails routers detect this through timer expirations or via explicit “unreachable” announcements (often coded as a distance of 16 or some “infinity”). Upon detecting a changed metric, distance-vector protocols typically trigger immediate updates rather than waiting for the next scheduled broadcast, which helps propagate new information quickly and reduce “counting to infinity” problems. The Routing Information Protocol (RIP) is the classic distance-vector protocol for IP networks. It periodically exchanges route entries on UDP port 520 and uses a “split horizon” rule (with optional “poisoned reverse”) to prevent loops between pairs of routers. Convergence can still slow in more complex loops, though “triggered updates” usually help speed up recalculation. RIP scales only to moderate network diameters because its defined “infinity” is 16, which bounds the maximum valid path length. In practice, smaller metrics simplify implementation and shorten the time for invalid routes to be marked unreachable.

The Bellman-Ford style of updating has proven versatile. Early ARPANET protocols referenced these fundamentals (Ford-Fulkerson methods), while the RIP specification formalized a UDP-based exchange of distance vectors. Extensions like the “modified Bellman-Ford” algorithm described in recent symbolic optimal control research (e.g., for UAV mission monitoring) still follow the same distance-vector core but can incorporate additional logic for storing intermediate information or pruning expansions. These adaptations highlight how distance-vector methods can be adapted to reduce the number of processed nodes or handle more elaborate “goals” or constraints, while preserving the underlying iterative, neighbor-to-neighbor nature of Bellman-Ford convergence [64] [65] [2]. Distance-vector protocols like RIP and EIGRP calculate routes by repeatedly exchanging hop count or composite metrics with neighbors. As the paper

notes, RIP operates by sending periodic (and sometimes triggered) updates, capping the path metric at 15 hops. Although this simple approach works well for smaller networks, it can lead to relatively slow convergence (on the order of seconds) when links fail. EIGRP improves on that with the Diffusing Update Algorithm (DUAL) and feasible successors, providing faster reconvergence and more scalable metrics but still inherits distance-vector fundamentals such as reliance on neighbor-to-neighbor exchanges. Routing constitutes the core mechanism underpinning internet connectivity, serving as its fundamental building block. However, achieving scalability is possible with the internet's vast network topology necessitates the integration of mechanisms beyond mere routing. In routing, networks are often viewed as a collection of interconnected routers executing uniform algorithms to compute paths. While this model is conceptually simple, it fails to address two critical real-world challenges: scalability and administrative independence.

## 2.2 Fundamentals of BGP and Autonomous Systems

The sheer scale of the internet encompassing hundreds of millions of routers renders uniform routing infeasible. Storing complete routing tables at each router and broadcasting link state updates across such a vast network would be computationally and operationally prohibitive. Moreover, ensuring convergence of algorithms like distance-vector routing across a global network would be impractical due to delays and inconsistencies [2]. The second challenge is sourced from the diverse administrative domains that comprise the internet. Internet Service Providers (ISPs) and organizations often operate their networks under unique policies, protocols, and performance goals. They may choose to conceal internal configurations or run customized routing protocols. To address these challenges, routers are grouped into Autonomous Systems (ASes) self-contained routing domains under single administrative control. Each AS runs an intra-domain protocol, such as Open Shortest Path First (OSPF) or IS-IS, to manage routing within the domain, while inter-domain communication relies on protocols like BGP (Border Gateway Protocol). These routing domains are identified by globally unique Autonomous System Numbers (ASNs), allocated by ICANN's regional registries, enabling hierarchical and scalable network management [1] [66] [67]. Building upon this foundation of intra-domain routing, the dynamics of inter-domain communication introduce a new layer of complexity that necessitates the use of policies. Unlike intra-AS protocols like OSPF, which optimize routes based solely on technical metrics such as link cost, inter-AS protocols like BGP must account for a diverse range of considerations. These include business agreements, economic priorities, and even geopolitical constraints, reflecting the decentralized and heterogeneous nature of the internet [68]. At its core, BGP enables the exchange of routing information between ASes, ensuring global connectivity. However, the autonomy of each AS means that

no universal optimization goal exists; instead, routing policies govern how data flows across AS boundaries. For example, an AS acting as a transit provider may prioritize traffic from paying customers over that from peers or competitors. Similarly, an AS might filter out specific routes to prevent traffic from passing through politically sensitive regions or to avoid security risks. This economical relationship also creates a hierarchical structure also enforcing policy based routing, hierarchical structure is generally created under three tier, tier 1 routers having no provider, tier 2 having customer and provider and tier 3 having no customers. The stability and scalability of the internet rely heavily on routing policies that regulate how networks handle and share routing information.

Autonomous Systems (ASes) depend on these policies to avoid the chaos of indiscriminately accepting all advertised routes, which could overwhelm their routing infrastructure and degrade performance. By applying filters and selectively propagating routes, these policies minimize unnecessary overhead and optimize resource allocation. They also act as a defense mechanism against routing disruptions like prefix hijacks and route leaks, ensuring that only valid and intended paths are used for traffic flow. Equally important is the role of policies in building network redundancy and ensuring uninterrupted connectivity. Policies enable ASes to establish alternative routing paths, allowing traffic to be dynamically redirected [69]. For example, multi-homed networks often leverage tools like AS Path Prepending or Local Preference to prioritize one provider while keeping others as failover options. This approach enhances resilience, ensuring seamless operation even under challenging network conditions [70] [71]. In summary, policies in inter-domain communication serve as the glue that holds the global internet together. By balancing technical, economic, and political considerations, they enable autonomous systems to maintain connectivity, optimize resource use, and uphold their individual goals, all while contributing to the broader objective of a scalable and resilient internet. Without these policies, the complexity and diversity of inter-AS relationships would render effective inter-domain communication impossible. Before diving into the specifics of BGP, it's essential to understand IP addressing and the techniques used for allocation.

## 2.3 IP Classes and CIDR Notation

Network boundaries are established based on the IP address class; there are five classes in total with three designated for unicast namely A, B, C, and D. The table below outlines each class along with its corresponding IP address range.

| Table 3-1, IP Address Classes and Functions |                              |                 |              |           |           |
|---------------------------------------------|------------------------------|-----------------|--------------|-----------|-----------|
| Class                                       | Address Range                | High-Order Bits | Network Bits | Host Bits | Function  |
| A                                           | 0.0.0.0 to 127.255.255.255   | 0               | 7            | 24        | Unicast   |
| B                                           | 128.0.0.0 to 191.255.255.255 | 10              | 14           | 16        | Unicast   |
| C                                           | 192.0.0.0 to 223.255.255.255 | 110             | 21           | 8         | Unicast   |
| D                                           | 224.0.0.0 to 239.255.255.255 | 1110            |              |           | Multicast |
| E                                           | 240.0.0.0 to 255.255.255.255 | 1111            |              |           | Reserved  |

Figure 2: IP Address Classes [2]

Class A addresses (0.0.0.0 to 127.255.255.255) are allocated to very large networks, such as major organizations and ISPs, with an 8-bit network portion and a 24-bit host portion, supporting up to approximately 16 million hosts per network. Class B addresses (128.0.0.0 to 191.255.255.255) are intended for medium-sized networks, accommodating around 65,000 hosts per network. Class C addresses (192.0.0.0 to 223.255.255.255) are designed for smaller networks, supporting up to 254 hosts. Class D addresses (224.0.0.0 to 239.255.255.255) are reserved for multicast traffic, facilitating one-to-many communication. Lastly, Class E addresses (240.0.0.0 to 255.255.255.255) are designated for experimental use and are not utilized in standard internet operations. However, increasing demand for the IP address caused depletion of the IPv4 space, currently ISP handles it by recycling unused IPv4 addresses and/or using CGNATs, DSLAMs, multiplexers for their customers to utilize IPv4 space they have efficiently. With the increase of routers, three major problems have been determined by the IETF ROAD group [72]:

The limited capacity of a Class C network offering only 254 host addresses proved too scant for even small to mid-sized organizations, while a Class B network, with roughly 65,000 addresses, provided far more than most mid-sized entities required. Additionally, as the Internet expanded rapidly between 1993 and 1995, concerns grew over the burgeoning size of routing tables and the inevitable depletion of the 32-bit IPv4 address space. To tackle these issues, the IETF ROAD group began redesigning the IPv4 address structure in 1990, an effort that eventually led to the introduction of CIDR (Classless Inter-Domain Routing). CIDR was implemented to help manage routing table growth and slow the pace at which IPv4 addresses were consumed, ensuring the Internet continued to operate efficiently. CIDR replaced the rigid class-based IP addressing system with a more flexible method. Instead of fixed network sizes, CIDR uses a combination of an IP address and a suffix to define the network's size. The suffix specifies how many initial bits represent the network portion. For example, the network 10.50.0.0/20 means the first 20 bits are dedicated to the network, leaving the rest for host addresses. This can also be visualized as a subnet mask of 255.255.240.0. By allowing networks to vary in size, CIDR makes better use of available IP addresses. How does CIDR help us solve the depleting IPv4

problem? For example, for more specific routes such as 192.32.1.0 or 182.32.2.0 of the related CIDR 198.32.0.0/16 gathered into one advertisement called aggregation. With this model, addresses are distributed in layers, where each entity typically receives its allocation from a more expansive pool above it. By dividing a larger block into progressively smaller pieces close to the network's endpoints, the amount of information that must be shared between routers is significantly reduced. Endpoints, in turn, only accept connections rather than relaying them onward. Organizations can allocate these finer address slices to many distinct sub-networks without needing to broadcast every small segment individually. This practice optimizes traffic announcements, curtails unnecessary overhead in routing data, and contributes to steadier, more efficient circulation of network pathways across the Internet [73] [74].

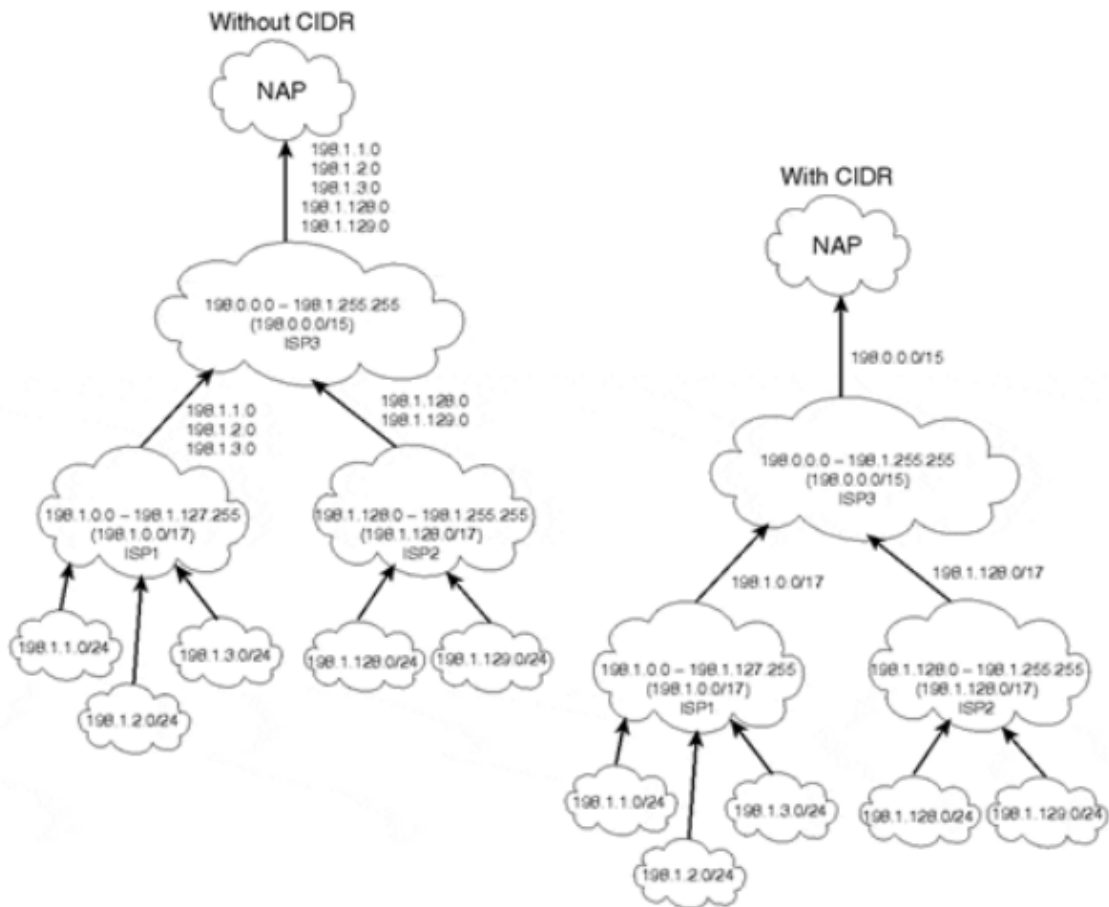


Figure 3: Hierarchy and CIDR Notation [2]

## 2.4 BGP Session Establishment

We've covered the basics of routing, its importance for the internet, the shift from individual routers to autonomous systems, inter- and intra-routing, different routing protocols, and IP addressing with CIDR. With this foundation, we can now understand how BGP works. Setting up a BGP session between two routers starts with configuring

them as BGP peers, which necessitates a stable connection to ensure the accurate exchange of routing information. BGP relies on the Transmission Control Protocol (TCP) to provide this reliability, eliminating the need to create an internal protocol for tasks like message delivery, duplication detection, and loss recovery. The session goes through several stages: beginning in the Idle state, advancing to Connect when initiating a TCP connection, and shifting to Active if the connection fails and requires retrying. Once the TCP connection is successfully established, the BGP session reaches the Established state after both peers exchange identification messages and confirm the connection. This dependable foundation allows BGP to use straightforward message formats with minimal overhead, though it must implement its own keepalive mechanism to monitor the session's health, as TCP does not offer this feature. In the Established state, BGP peers start exchanging routing information, initially sharing all desired prefixes and later sending updates only for any changes. This efficient process is further improved by BGP's ability to apply routing policies, enabling peers to selectively accept or reject routes based on predefined configurations. The BGP session remains stable as long as the TCP connection stays active. If errors arise or the connection is interrupted such as due to a router failure or loss of a viable path BGP addresses these issues by sending error notifications, terminating the TCP connection, and returning to the Idle state. As a result, any routing information obtained from the disrupted peer is immediately discarded, ensuring the network's routing tables stay accurate and current.

## 2.5 BGP Messages Structure and Types

BGP messages are formed of 16 Octets called Markers for either security or synchronization 2 octet specifying the length and 1 octet specifying the type of the packet. Length field indicates entire length of the packet while type of the packet described with a single octet describes the type of the BGP message being sent in this case possible values are, OPEN (1), UPDATE (2), NOTIFICATION (3), KEEPALIVE (4). Each message type has a specific mission, OPEN message can be considered as an introduction to the BGP session between both peers sent from one endpoint to another. It allows each endpoint to identify itself agree on protocol parameters these parameters could be AS number, version of the protocol such as BGP4 or hold time, such as maximum time a peer will wait for a keepalive message and optional parameters such as capabilities extensions that allow peers to advertise their support for advanced features like multiprotocol extensions or route refresh. When an OPEN message arrives from a neighbor and is processed, the "My Autonomous System" field is checked against the neighbor's configured ASN. If the two values are identical, processing continues as normal. If they do not match, an error message is generated and the TCP connection is terminated. Once the communication is established through initial OPEN messages and exchanged

parameters, most of the time UPDATE message is the main type used throughout the communication session; it can be considered as the primary message used to exchange BGP information between two endpoints. When there is a need for update on certain prefixes such as, say a prefix advertised in the past needs to be backed out from the communication or the registry of an endpoint UPDATE is used. The NOTIFICATION message type is used in BGP to indicate an error condition within the session. When a critical error occurs, the sending peer transmits a NOTIFICATION message to the receiving peer. This usually results in the immediate closure of the TCP connection between the two peers. Also it allows engineers to diagnose the issue, on what went wrong. Type field in the NOTIFICATION message indicates the type of error, there are a variety of error types describing errors on also different types of BGP messages, such as UPDATE message error, or for example it can indicate an unwanted transition has occurred in the session by sending Finite State Machine Error. Also the subcode field provides more specific information about the error. Also a specific message type to BGP considered as an “extension” to TCP is KEEPALIVE, TCP has its own KEEPALIVE mechanism that operates at the transport layer. They are used to detect idle connections and prevent them, on the other side BGP KEEPALIVE used to maintain BGP sessions and ensure that both peers are still actively participating in the routing exchange. Since KEEPALIVE requires some type of probing mechanism, package type includes a hold timer field for specifying the period of KEEPALIVE messages sent through the channel. BGP4 RFC regulates how all of these specifications should behave on learning prefixes and further removing it or advertising it depending on the policy configurations. It specifies a general framework rather than a set of strict rules on dictating how the session must be managed, the end goal of the RFC can be summarized as, defining a robust, efficient way of announcing, learning, withdrawing prefixes in the meantime being as flexible as possible [66].

## 2.6 BGP Data Structures

The specification outlines a set of elements designed to encapsulate specific contexts: Adj-RIB-In, Loc-RIB, and Adj-RIB-Out. Here, RIB stands for Routing Information Base, which essentially refers to the routing table. As the name implies, Adj-RIB-In pertains to the BGP speaker's peers. It plays a critical role as the first stage in the BGP filtering or processing pipeline, storing the initial routing information received from BGP peers. This information is subsequently utilized by the BGP decision process to determine the best paths, which are then added to the local routing table. There must be an intermediary process before moving information from Adj-RIB-In to Loc-RIB, once the prefixes that will be announced from the BGP speaker is finalised, processed information is moved from Adj-RIB-In to Loc-RIB. In the end Adj-RIBs-Out per the name suggests are used prefixes that will be announced to particular neighbors,

similar to Adj-RIB-In, Adj-RIB-Out has information per each peer. Besides the data structures that a BGP speaker must implement, also RFC specifies path attributes, these are core and critical information that needs to be considered by a system where inter-AS routing is relevant. It also primarily serves to extend BGP, without changing the underlying structure of it, with the provided flags by these specifications or attributes.

## 2.7 BGP Path Attributes

ORIGIN describes intuitively the journey of an internet route (or prefix), ORIGIN here exactly specified how a particular route was introduced into BGP, there are different pathways due to structural nature of the internet topology these or direct entries where it can be represented with IGP, or external entries that is represented with EGPs, these are entries that handles announcements from outside of the interior network or local network. Also RFC specifies a third option where the route couldn't be destined or identified either from outside or from the interior network, in these cases it is marked as INCOMPLETE, flagged as uncertain entry. ORIGIN attribute in the end helps network admins and systems to understand the trustworthiness and origin path of a route. Just as knowing a traveler's origin can influence security checks, knowing a route's origin can affect routing decisions and policies within the network, to provide an example to make it clear we can consider an example with three ASes, AS100, AS200, and AS300 are interconnected in a sequence where AS100 initiates the announcement of a prefix, such as 203.0.113.0/24, learned from its internal network. Initially, the path includes only AS100, indicating the origin of the prefix. AS100 then propagates this prefix to its external neighbor, AS200, using BGP, as the announcement extends beyond its internal network. At this stage, the path is updated to include AS200, resulting in the sequence AS100, AS200, with the origin marked as IGP (Interior Gateway Protocol). Subsequently, AS200 announces the prefix to its neighbor, AS300, further propagating the route. The path now becomes AS100, AS200, AS300, while retaining the origin information as IGP, which originated from AS100's internal network. Notably, this path construction inherently encapsulates the origin information at each step of the route announcement. Although the origin is explicitly marked as IGP from AS100's internal network, local inspection at each AS can also reveal the neighbor from which the route was received, providing additional context about the route's propagation path. This information is further used to construct different path plausibility algorithms such as ASPA, AS-Cones that will be discussed later in this chapter. The next attribute intuitively as described in the previous example is the AS-PATH attribute, it is if not the most important attribute within the BGP specification, most of the tools, in-band, out-of-band technologies that is being utilized or just developed as experimental, research based, had been utilizing

the AS-PATH attribute majorly. Building upon the understanding of the ORIGIN attribute AS-PATH provides a detailed trail of the ASes that a prefix traverses, this attribute is fundamental in BGP's ability to make informed routing decisions, prevent routing loops and enforce network policies. It's basically a simple data structure that stores the sequence of ASes. By maintaining a record of the ASes a route has traversed, BGP can detect and prevent routing loops, if an AS sees its own ASN in the AS-PATH of an incoming route, it knows that accepting this route would create a loop and thus the route is rejected, route selection is based on shorter AS-PATHs generally, that is the primary metric, or AS numbers are prioritised depending on the specification of the ASes or ISPs, and finally AS-PATH allows policy enforcement, by the network admins, they can implement policies based on the AS-PATH, for example they might prefer routes through certain ASes for reasons like cost, performance and trustworthiness. To draw a distinction between ORIGIN and AS-PATH, ORIGIN specifies where the announcements originated from while AS-PATH also could infer this attribute it is not used for it specifically but it holds the sequence of AS numbers that prefix had been traveled, or chain of ASes that the prefixes have been propagated through. But not all the time AS-PATH construction is just as simple, by simply chaining together sequence of ASes, sometimes there are complex scenarios where an AS could be formed of different areas or partitions and these partitions might have different reachability information such as consider our previous case formed into below topology, AS200 is only reachable from a single or different parts of the partitioned sections of the AS100 so when the AS-PATH is constructed and route is propagated this structure of the topology needs to be considered as well. Also by looking at the below figure some of the partitions of the AS100 is only reachable via more complex path construction such as AS100-X to AS200 and AS300 then AS100-Y, to resolve this issue AS-PATH with type code 2 have been a mandatory attribute, it is encoded as sequence of AS-PATH segments, each segment contains encoded path segment, segment type and path segment length and path segment value. Type is one of the two fields with AS-SET containing an unordered set of ASNs and AS-SEQUENCE containing an ordered set of ASNs. Generally these two data structures are similar or same, since the AS-PATH is constructed with propagation information and since at each step loops are checked even though the set is unique and unordered the structure and the sequences are same as the AS-SEQUENCE data structure. To provide a clear distinction between AS-SET and AS-SEQUENCE let's consider a simple example, consider our previous example and above figure AS100 advertises two specific prefixes, 203.0.113.0/25 with AS-PATH 100 200 and 203.0.113.128/25 with an AS-PATH of 100 200, AS200 receives both routes from AS100 and decides to aggregate them into a single broader prefix to simplify routing information it advertises to its neighbors. Aggregated prefix becomes 203.0.113.0/24 and AS-PATH attribute becomes {100} 200, with {100} representing the AS-SET, AS-SEQUENCE 200 shows the sequence of ASNs the route has traversed

indicating that the aggregated route has passed through AS200. By aggregating the two more specific prefixes (203.0.113.0/25 and 203.0.113.128/25) into a single broader prefix (203.0.113.0/24) with an AS-PATH of {100} 200, AS200 effectively reduces the size of its routing table. This aggregation maintains the necessary path information while simplifying route announcements, enhancing overall network efficiency. Another key property is the NEXT-HOP attribute.

## 2.8 BGP Route Selection Process

When a BGP speaker initiates an announcement, it includes the next-hop node, which is the immediate node that will forward the data or payload toward the destination. This next-hop node plays a critical role in ensuring packets are routed efficiently, bringing them closer to their intended destination. Another important property that is specified, is MULTI-EXIT-DISCRIMANTOR, it is used when an AS has more than one exit node which is most of the case with the larger carriers, CDNs, then this attribute allows us to determine or discriminate between these exit nodes. For example if two consider the above figure where AS200 and AS300 have been connected with two different links, link A and link B, AS2 can include MULTI-EXIT-DISCRIMANTORs, providing information about prefixes on distance from internal endpoint or gateway then AS1 could decide on this to use which one of the links. For example consider a prefix being announced to AS1 from AS2 a MULTI-EXIT-DISCRIMANTOR with value can be used with 10 on link A but on link B it can be greater then this say 100 when announcing the same prefix to AS1 from AS2, and AS1 with this information can choose to select the best optimized path. But by itself MULTI-EXIT DISCRIMINATOR is not enough, The MULTI-EXIT-DISCRIMINATOR attribute is inadequate in scenarios where an AS has only one connection to each neighboring AS, as it cannot differentiate or prioritize paths when there are no multiple direct links between the same AS pairs. In the given example, AS1 is connected to AS2 and AS5, each with a single link to AS4. Since MULTI-EXIT-DISCRIMINATOR is effective only with multiple exit points between the same ASes, it cannot help AS4 prefer the better service path from AS5 over AS2. To address this, AS4 uses the LOCAL-PREF attribute, allowing it to assign higher preference values to routes from AS5, ensuring that traffic is directed through the preferred AS regardless of the single connection limitation. Therefore, while MULTI-EXIT-DISCRIMINATOR is useful for managing multiple exit points between the same AS pairs, LOCAL-PREF is necessary for controlling route preferences in scenarios with distinct single connections. While these attributes form the foundation of this study, it is important to recognize that BGP encompasses additional attributes such as COMMUNITY, AGGREGATOR, ATOMIC-AGGREGATOR among others. These supplementary attributes offer further mechanisms for route manipulation, policy enforcement and aggregation. However,

they fall outside the scope of this thesis, which concentrates on the aforementioned core attributes to analyze their impact on routing efficiency and policy implementation. These core parts and attributes form the fundamentality of the BGP route selection process, these attributes at its core allow the construction of the algorithm, as in with any protocol specification. We can now shift our focus to explaining how the BGP algorithm functions by leveraging its attributes and data structures. When there are multiple routes for the same prefix, the BGP speaker processes them through a preference pipeline a mechanism that the simulator also considers when evaluating multiple available paths:

- LOCAL-PREF: The route with the highest LOCAL-PREF is chosen.
- AS-PATH: If LOCAL-PREF values are equal, the route with the shortest AS-PATH is selected.
- MULTI-EXIT-DISCRIMINATOR (MED): Among routes from the same AS, the one with the lowest MED is preferred.
- NEXT-HOP Cost: The route with the minimal NEXT-HOP cost (determined by IGP) is picked.
- E-BGP vs. I-BGP: If a route comes from E-BGP, it is immediately selected; if several E-BGP routes exist, the one with the smallest BGP identifier is chosen.
- I-BGP Tie: For I-BGP routes, the one from the neighbor with the lowest BGP identifier is preferred.

To fully understand BGP's operation, consider a simple network that includes both an interior and an exterior gateway. The diagram below illustrates this concept, expanding on a previous version by detailing the internal and external configurations of BGP speakers.

## 2.9 BGP From Perspective of ISPs

Since BGP relies on TCP (port 179) for session establishment, it requires explicit configuration of the peer's IP address. Routers typically have multiple interfaces and IP addresses, but a BGP session is established between specific IP addresses. If the interface associated with a BGP session's IP address goes down, the session will terminate unless configurations like loopback interfaces and appropriate routing are used to maintain the session across multiple paths. However, this scenario does not apply to I-BGP (Internal BGP). Within an autonomous system, there may be alternative connections between routers. For example, in the figure above, there are

two distinct paths from R1 to R2. In such cases, the previously mentioned issue does not apply, as simply terminating the direct link between R1 and R2 does not necessarily end the session. This is because an alternative path exists from R1 to R2 via R3. Therefore, the IGP (Interior Gateway Protocol) must also account for this alternative path when making routing decisions.

Routers within the same domain commonly use loopback addresses to form I-BGP connections, but those addresses must be reachable in both directions. This requires an internal routing protocol that advertises and learns each router's loopback network, ensuring that every router knows how to send packets to every other loopback. Once these routes are exchanged and established, the I-BGP sessions can be set up, allowing BGP updates to flow properly across the AS. One of the central focuses of this thesis is routing policy. As previously discussed, the core design of BGP, along with attributes such as ORIGIN, AS-PATH, and local preferences, enables routers to implement a highly customizable route acceptance process. These configurations, or more precisely, these policies, define how BGP operates. Routing policies are essential to the current structure of the internet, significantly influencing the flow of traffic determining where it goes and which neighbors it passes through. In essence, policies can be thought of as configurations or parameters that tailor the behavior of the BGP protocol.

Routing policy in BGP means deciding which routes to accept from a neighbor, how much priority to give each route, and which routes to advertise back to them. These decisions happen locally and can differ between neighbors. This flexibility is crucial for controlling how traffic moves through a network and for upholding business deals between networks. Imagine three ISPs (ISP1, ISP2, and ISP3), each connected to one another, and each has a paying customer. An ISP that offers "transit" service to a paying customer allows the customer's traffic to pass through its network to reach the rest of the internet. However, it might choose not to extend this privilege to other ISPs that don't pay, which is called "nontransit" service. For instance, if Customer1 is paying ISP1 for internet access, ISP1 must share Customer1's routes with ISP2 and ISP3 so that everyone can reach Customer1. ISP1 also needs to pass on routes from elsewhere so Customer1 can reach any other destination. But ISP1 will likely avoid sharing ISP2's routes with ISP3 if it doesn't want to become a free transit path for traffic that isn't paying ISP1. In short, routing policy helps ISPs avoid carrying traffic that doesn't belong to their customers unless there is a financial or contractual reason to do so.

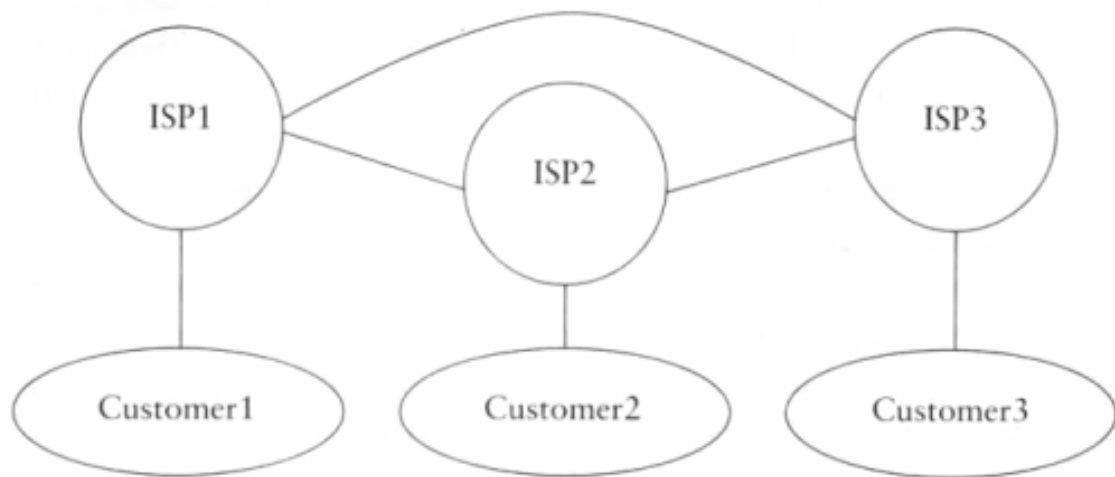


Figure 4: Customer peer and provider relationship between ISPs [2]

Below is one way to illustrate how BGP policies and inter-AS routing apply in practice, using the traceroute result for yahoo.co.jp as an example. Notice how the path crosses multiple ISPs and international exchange points, reflecting decisions shaped by BGP advertisements and policies discussed in the passages:

Starting with the local gateway (192.168.1.106 -> \_gateway), the packets move through the user’s ISP infrastructure (lines showing ttnet.com.tr and turktelekom.com.tr). These entries represent different routers often in the same AS or “internal network” handing off traffic according to policies that decide which next-hop leads to the broader internet. After leaving the local provider in Turkey, the route jumps to the DE-CIX internet exchange in Frankfurt (ipv4.de-cix.fra.de.as2497.iiij.ad.jp), which is a major traffic hub where multiple ISPs peer with one another.

From Frankfurt, the path is forwarded to IIJ (a large Japanese ISP), traversing connections that cross continents. This part of the route reveals how global BGP sessions stitch together separate ASes, with each AS applying filters, route preference settings (like LOCAL-PREF or MULTI-EXIT-DISCRIMINATOR), and transit or non-transit policies. Finally, the traffic reaches Yahoo Japan’s servers (210.130.135.130 and then 124.83.185.252), demonstrating how BGP announcements guide data all the way to its destination.

Just as described in the passages, each AS along this path makes local decisions on whether to accept and advertise specific routes, using the BGP attributes and policies that shape inter-domain routing. The traceroute provides a tangible snapshot of these policy-driven hops, showing how a seemingly simple request to yahoo.co.jp relies on a carefully orchestrated exchange of routing information among independent networks.

One major conclusion that we can draw from these two examples is that the functioning of the Internet relies heavily on how Internet Service Providers (ISPs) manage routing policies through BGP to maintain seamless connectivity. Imagine

an ISP as a central hub that not only connects its own network but also ensures its customers are accessible to the broader internet community. To achieve this, the ISP must advertise its customers' network addresses to other ISPs, allowing external users to reach its customers effortlessly. Simultaneously, the ISP must share comprehensive routing information with its customers, enabling them to access resources and services across different networks globally. This dual approach ensures that communication flows smoothly both into and out of the customer's network, maintaining robust and reliable internet connectivity. By carefully balancing these announcements, the ISP enforces effective routing policies that uphold the integrity and efficiency of the overall internet infrastructure.

This thesis mainly focuses on the BGP from the perspective of the providers mainly, since each AS is represented as a single node in a graph of ASes. From the POV of the providers generally can be argued under three different sections, aggregation, filtering and exchange of route information.

ISPs often consolidate multiple specific routes into broader prefixes before advertising them to neighboring Autonomous Systems (ASes). This practice significantly reduces computational overhead, shrinks the size of routing tables, and improves overall network performance. Additionally, ISPs enforce strict filtering policies for transit customers by carefully controlling which routes they accept and advertise. This filtering can be implemented through either static or automated processes, with the approach typically varying based on the size of the carrier. A key concern for each AS is ensuring that only authorized and valid routes are propagated through its BGP sessions, thereby preventing the spread of incorrect or harmful routing information.

On the other hand, providers actively seek out exchange points where routing and the dissemination of route information can be streamlined. These exchange points, known as Internet Exchange Points (IXPs), are vital to the global internet infrastructure. They enable multiple ISPs to exchange traffic directly and efficiently, eliminating the need for a single backbone provider. By utilizing layer 2 technologies like Ethernet or ATM, IXPs facilitate seamless connections between diverse networks, reducing dependency on expensive point-to-point links and lowering operational costs. However, the shared nature of IXPs introduces challenges in traffic management and security. To address congestion caused by multiple peers sharing a single connection, ISPs implement mechanisms to regulate the volume of data each participant can transmit. Furthermore, advanced interface features are deployed to enhance security, allowing ISPs to explicitly permit or block traffic from specific layer 2 addresses and control the rate at which data is accepted from each source. These measures ensure that IXPs remain efficient and secure, supporting reliable inter-ISP communication while preventing misuse and enforcing adherence to individual routing policies.

## 2.10 Gao-Rexford and Valley-Free Routing

The Internet’s routing system can stay stable without a central authority controlling every network. Each network uses its own rules, and by following a disciplined approach to choosing routes, they work together to avoid problems like route oscillations or divergence.

In practice, this means networks prioritize routes from their customers over those from peers or providers. This priority order is based on the natural, hierarchical structure of the Internet and the commercial relationships between networks. By always selecting the most reliable, customer-derived routes first and then considering other options, the whole system naturally converges to a stable state.



Figure 5: Three Rule of the Policy Based Forwarding: traffic received from customer can be forward to all, traffic received from peer only can be forward to customers and traffic received from provider only can be forwarded to customers [75]

Instead of relying on a central registry to manage every network policy, a process that is both complex and computationally heavy, this approach lets each network set its own preferences. It takes advantage of the natural hierarchy in networks so that even if a network changes its rules or a link fails, the overall system still settles into a stable state. This framework not only handles current stability challenges but also adapts to the evolving Internet and new routing issues.

The Gao-Rexford framework and valley-free routing are essentially two ways of describing the same idea. When each network favors routes from its customers over those from peers or providers, the whole system naturally becomes stable, following a “climb-then-descend” pattern.

Imagine a network as a system of roads connecting villages, towns, and cities. Ideally, a traveler (or data packet) should move from local roads (villages) up to highways (major cities) and then down to the destination. A “valley” occurs when the traveler takes an unnecessary detour through a lower-tier road much like going from village to city to town to city to village creating inefficiency. Valley-free routing ensures that routes follow a direct “climb then descend” path.

Similarly, in hierarchical systems like corporate structures, data should flow from

lower-level branches up to a central hub and then down to the target branch. If a lower-level node improperly routes data between higher-level nodes, it disrupts the flow much like misdirected mail can delay communication.

**Direct Pathways (Peer Links):** If two regional hubs (same hierarchy level) lack a direct connection, traffic might detour through a branch, creating a valley. Building a direct highway (peer link) between them shortcuts the detour, eliminating the valley.

**Policy Enforcement (Traffic Rules):** Implement "no entry" signs (route filters) to block traffic from entering lower-tier nodes. For example, a regional hub could ignore routes advertised by branches that would force it to send data back uphill. Alternatively, assign higher "tolls" (metrics) to valley paths, making them less attractive to routing algorithms.

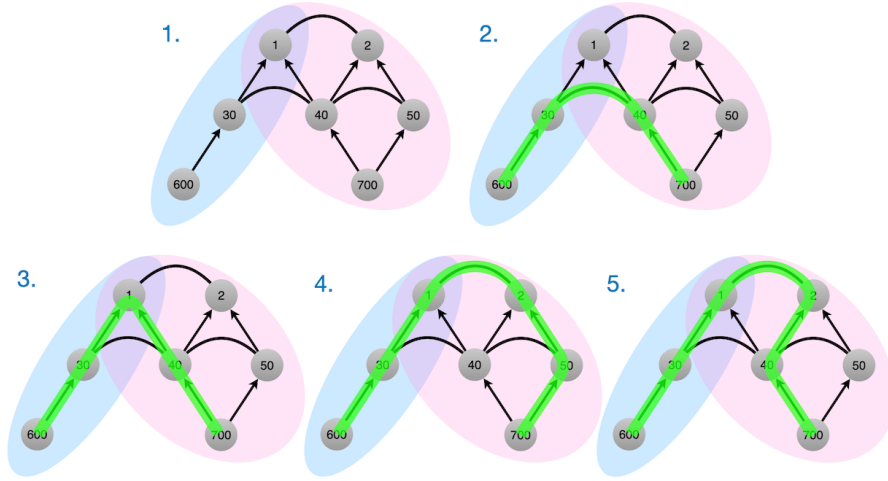


Figure 6: Examples to valid paths, no valley on each of the chosen paths can be seen, traffic uses providers to change sides on the tree structure

Assign each node a rank (e.g., core=1, region=2, branch=3). A valley-free path is a sequence which ranks first non-increase ( $1 \rightarrow 1 \rightarrow 2 \rightarrow 2$ ) toward the core, then non-decrease ( $2 \rightarrow 3 \rightarrow 3$ ) toward the edge. A valley breaks this monotonicity (e.g.,  $2 \rightarrow 3 \rightarrow 2$ ). Treat connections as directional agreements. A valid path ascends via "providers" (+1), may cross a "peers" (0), then descends via "customers" (-1). A valley would illegally reverse direction, like a provider suddenly acting as a client mid-path.

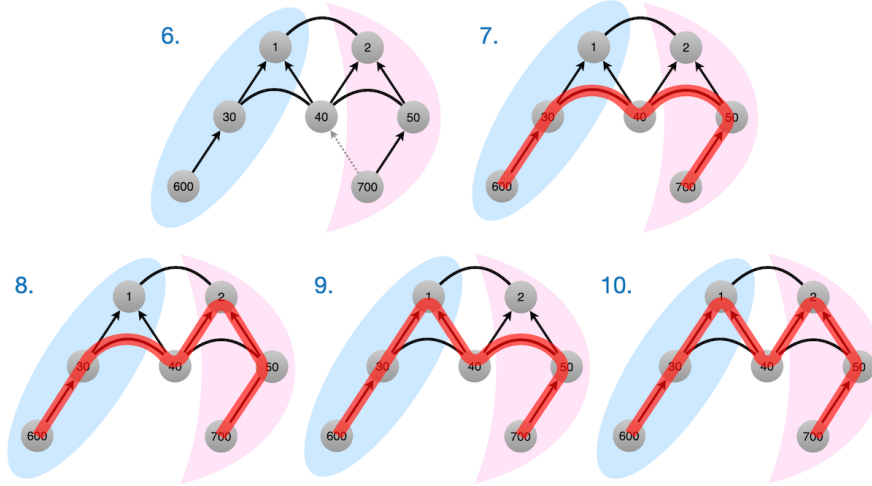


Figure 7: Invalid path, notice there are valleys on the taken path

## 2.11 Security Properties

Until now no BGP cryptographic security mechanism has been described, so it is not intrinsically to the BGP, also intuitively if there is a problem with TCP then BGP inherits all the problems. As described in the BGP fundamentals, AS-PATH can be considered as a chain, where the routing information dissemination from one neighbor to another and forms the PATH chain, so any compromise in the chain would affect the whole communication. [76] [52] Thus some type of authentication mechanism is required. Since BGP is based on TCP, IETF and drafts published, referenced the TCP related authentication mechanism that is MD5 hashing, though as per the RFC draft this system requires a predetermined key or password known to both ends of the session so there is a shared key problem that the BGP did not designed with first shared key in mind.

False route information can be introduced into a network by a BGP speaker, whether intentionally or accidentally, especially when basic security measures aren't in place. This can lead to significant changes in the network topology, potentially disrupting communication for extended periods or, in some cases, having little to no noticeable impact. The effects largely depend on issues like route flapping, black hole routing, routing loops, or prefix hijacking, as mentioned earlier. In more subtle scenarios, the impact might not be immediately obvious but can still cause long-term problems. For example, traffic might be routed through suboptimal paths, which can degrade network performance and efficiency over time without being easily detected.

- **Blackhole:** Large amounts of traffic are misdirected to a router that lacks the capacity to handle it, causing most or all packets to drop.
- **Looping:** Traffic repeatedly traverses the same path without ever reaching its destination, resulting in infinite circulation.

- ***Partition:*** Part of the network is mistakenly treated as cut off from the rest, even though a valid connection exists.
- ***Churn:*** Rapid, frequent shifts in routing choices lead to inconsistent data delivery and potential congestion.
- ***Instability:*** BGP cannot settle on a stable global state, causing persistent route fluctuations.
- ***Resource Exhaustion:*** Excessive BGP updates consume critical router resources (such as memory or table space).
- ***Address-Spoofing:*** Traffic is deliberately funneled through a node that masquerades as a legitimate address, enabling potential data manipulation.

Also another IETF draft [77] separately provides security and configuration suggestions on BGP elements such as session, speaker etc. The draft focuses on 11 major discussions, these are respectively,

BGP wasn't originally built with strong security in mind, which leaves it vulnerable—if someone can spoof or inject TCP segments, they might disrupt routing by forcing BGP sessions to reset. To counter this, RFC 2385 introduced the TCP MD5 Signature Option. This option requires both ends of a connection to share a secret key and include an MD5 hash with every packet. Even though MD5 isn't perfect, it's still widely used by vendors. Newer options like TCP-AO offer stronger security and more flexible key management, though they haven't been adopted as widely yet.

Beyond protecting the session itself, network administrators also need to prevent issues like false route advertisements, prefix hijacks, and misconfigurations. RFC 7454 suggests best practices such as route filtering, setting maximum-prefix limits, and using strict Access Control Lists. It also covers additional defenses like TTL Security (GTSM) and route-flap dampening, and offers advice on controlling the spread of announced prefixes. Similarly, guidelines like those in NIST SP 800-189r1 help protect critical infrastructure routers and major interconnections like Internet Exchange Points.

To ensure that advertised routes truly belong to the right resource holders, the Secure Inter-Domain Routing (SIDR) initiative and the RPKI framework use cryptographic certificates to confirm AS number and prefix ownership. While some networks have begun checking that a route's origin AS matches its RPKI certificate, full path validation with "BGPsec" is still in progress. In the meantime, many operators rely on IRR filters along with MD5 or TCP-AO to secure their BGP sessions. Together, these practices create a more secure and policy-aware system for inter-domain routing while building on the fundamental concepts of BGP [77] [52] [78].

## 2.12 Early security mechanisms such as TCP, Hashing, MD5 etc.

As the RFC2385 suggests that MD5 could be used for establishing secure channels as with the IPsec and the TCP session protection, since BGP uses TCP as its underlying communication method. Also since BGP is application layer protocol and TCP is transport layer, one prevention could not simply be beneficial by itself, considering BGP is somehow protected with an authentication mechanism with OPEN message type. It is still vulnerable to attacks against TCP, a simple example would be TCP, RST message can cause endpoints to get out of sync. So at minimum TCP connection needs to be secured as well.

MD5 digest, takes input and produces an irreversible hash also it is negligible to find collision meaning two different message with same hash, though later on this has been proven to be wrong by Den Boer and Bosselaers in 1993 making this approach obsolete, also size of the hash value has been long debate and on small size it has found to be susceptible to birthday attack. The proposal adds a generic MD5-based authentication feature to TCP, though it's primarily used with BGP. In this setup, both ends of a BGP session are configured with a shared password. Each TCP segment includes an MD5 digest, calculated using the segment's data and the password. When a router receives such a segment, it verifies the digest by performing the same calculation. If the digests don't match, the segment is dropped. For an attacker to compromise this system, they would need to guess both the TCP segment numbers and the shared password, which is never transmitted. Despite some known weaknesses, the proposal is likely to become a standard due to its widespread use [52].

## 2.13 BGP Security Mechanisms (BGP-Sec, RPKI, ROV etc.)

Considering the aforementioned problems and scalability issues with different approaches on securing the BGP session with its underlying backbone as TCP several suggestions have been made, two of the popular solutions proposed are BGP-Sec and RPKI, these are two different methods one is benefiting from the other one, RPKI is architectural approach where as BGP-Sec uses RPKI to secure the session, we will be discussing how these two operations on the core BGP specifications and communications operates and works since these are related to the context of this thesis as well.

One reason these mechanisms BGP-Sec and RPKI have emerged is that BGP was never designed with strong security features in mind. In practice, it relies heavily on mutual trust between neighboring routers, making it vulnerable to attacks like route hijacking and route leaks. A malicious party might, for instance, claim address space they do not own (prefix hijacking) or announce a more specific route (sub-prefix hijacking) that overrides legitimate advertisements via the "longest prefix match" rule. By verifying which AS is authorized to originate each IP prefix (i.e., route origin

validation) and ensuring that path attributes in BGP announcements have not been tampered with (i.e., path validation), we can substantially reduce these risks. RPKI plays a foundational role in providing this trust framework: it ties IP addresses and AS numbers to verifiable public keys, while BGP-Sec extends those protections across the entire AS path.

Following this logic, since IP prefixes are ultimately allocated by IANA and delegated downward through RIRs, it is both natural and important to maintain a cryptographically sound chain of authorization. As noted, RPKI uses “resource certificates” [79] to declare, “You are authorized to use these specific IP addresses or AS numbers.” Rather than attesting to an organization’s name, these certificates reflect legitimate control over Internet Number Resources. Corresponding private keys then prove possession of those resources, aligning perfectly with the hierarchical allocation structure described above. With these pieces in place, routers can rely on pre-validated data often disseminated using the RPKI to Router (RTR) protocol to confirm that the prefixes they learn are being announced by the right AS, bridging the gap between the trust-based roots of BGP and a more secure, cryptographically verifiable interdomain routing infrastructure.

Since the IP prefixes are allocated by IANA to each provider and subsequently those IP prefixes are allocated to the sub-ASes, this makes it logical to cryptographically verify and attest these allocations, or IP blocks and also since there is a hierarchical structure on sub-allocating to different organizations it is intuitive to support this cryptographic attestation through public key infrastructure (PKI)

RPKI uses “resource certificates” these are digital certificates that say “You are authorized to use these specific IP addresses or AS numbers” PKI in this context can be considered as a framework that utilizes these certificates and public-private key pairs to secure and verify identities or authorizations. Each resource certificate contains a public key and a list of IP addresses or AS numbers. The holder of the private key matching this public key is allowed to use these resources. RPKI extends this by specifying how to include IP addresses and AS numbers in the certificates

One of the chief motivations for enhancing BGP security lies in its trust-based design: routers simply trust the advertisements from neighbors. Attackers can exploit this by claiming IP address space they do not own (prefix hijacking) or, more potently, by advertising a more specific IP block than the legitimate one (sub-prefix hijacking). The latter exploit typically succeeds due to the default “longest prefix match” rule in routing. Accordingly, secure mechanisms must verify both who is allowed to originate a prefix and whether the path attributes in a route announcement have been tampered with. Although RPKI primarily addresses route origin validation by ensuring that IP prefixes are announced by the correct AS, it also underpins BGP-Sec’s path validation, making RPKI the foundational trust infrastructure for interdomain routing security.

Conceptually, RPKI is a specialized Public Key Infrastructure anchored by IANA

at the global level, distributed through the Regional Internet Registries (RIRs), and potentially delegated further to National Internet Registries (NIRs), large ISPs, and finally down to end users or enterprises. In this framework, resource certificates bind IP addresses or AS numbers to a specific public key, confirming an entity's right to use these resources. Unlike traditional X.509 certificates that bind a public key to an organizational name, RPKI certificates only convey authority over IP space or AS numbers. An entity possessing the corresponding private key is thus recognized as the valid operator for that resource. The hierarchical structure ensures that cryptographic delegation mirrors the real-world allocation of address space, allowing each level to issue certificates or sign objects for subordinate levels. At the operational level, local caches run specialized validation software to retrieve all RPKI data, authenticate resource certificates, and confirm the correctness of objects such as Route Origin Authorizations (ROAs). Once validated, the data is furnished to routers, typically using the RPKI to Router (RTR) protocol, so that the routers can implement route origin validation (ROV) efficiently without performing heavy cryptographic checks.

Within the RPKI environment, several cryptographically signed objects enable comprehensive verification of routing information. CA certificates lie at the core, each referencing a parent certificate and eventually linking back to a trusted root (often an RIR's certificate). ROAs declare which AS is authorized to originate which prefixes crucial for assessing route legitimacy. Manifests list all valid objects at a publication point and their cryptographic hashes, aiding in detecting accidental or malicious omissions. CRLs (Certificate Revocation Lists) note any certificates that have been revoked and should no longer be trusted. Ghostbusters Records, though optional, provide out-of-band contact information for repository or certificate holders, which is valuable in resolving anomalies.

Once local caches gather and validate these repository objects, they distribute the resulting valid data to routers via the RTR protocol. This protocol establishes a simple client-server relationship in which routers maintain lightweight, persistent connections to the local cache. Through these connections, routers either request a full set of valid prefix-origin mappings (in a Reset Query) or incremental changes since the last known update (in a Serial Query). If the cache detects that new or modified RPKI objects have been published, it can send a notification prompting the routers to pull the updated data. Because the routers offload intensive validation operations to the caches, the network becomes more scalable and secure.

With validated ROAs in place, operators can enforce Route Origin Validation (ROV) at the BGP ingress. Under ROV, whenever a new BGP route is learned, the router checks whether a corresponding ROA exists and matches the route's origin AS and prefix. Routes that fail this check can be tagged invalid and, in many networks, dropped outright. This practice diminishes the impacts of route hijacks, particularly forged-origin sub-prefix hijacks, by ensuring that illegitimate more-specific

advertisements do not propagate through networks that honor ROV.

BGP-Sec extends this idea further by securing not just the origin but the entire AS\_PATH. In BGP-Sec, each AS on the path signs the path segment with a private key tied to a specific “router certificate” published in the RPKI. A receiving router thus verifies every hop in the path, making it far more difficult for intermediate attackers to falsify or remove AS hops without detection. Although BGP-Sec adoption has lagged behind ROV due to complexity and operational overhead, the presence of RPKI as a well-defined, hierarchical trust anchor means the mechanisms for verifying BGP-Sec router certificates already exist.

Despite increasing deployment, RPKI still faces challenges. A growing number of networks create ROAs for most of their publicly routed IP prefixes, thereby marking these routes as “RPKI valid.” This has led many ISPs to automatically reject invalid routes. However, some networks misconfigured ROAs by setting a “maxLength” that allows more-specific blocks than those announced in BGP, inadvertently opening the door to forged-origin sub-prefix hijacks. The recommended best practice is to generate only minimal ROAs that match the exact prefix lengths in actual use. Other emerging improvements include automation protocols like DISCO, new object types such as ASPA for verifying provider relationships, and more efficient repository access protocols like RRDP. Broader collaboration among RIRs, NIRs, and ISPs is vital, as are legal and organizational frameworks that align with cryptographic delegation of resources.

In conclusion, the Resource Public Key Infrastructure (RPKI) serves as a cornerstone for securing the global routing system. By binding IP space and ASNs to verifiable public keys and facilitating distribution of this trusted information, it enables the critical functions of route origin validation and, combined with BGP-Sec, path validation. While operational complexities persist, incremental adoption of RPKI-based filtering has already prevented numerous route hijacks. With best practices such as minimal ROAs and consistent use of ROV, the community continues to harden the routing fabric. As more operators incorporate BGP-Sec or equivalent path-validation approaches, and as further automation and tooling (e.g., DISCO, ASPA) reduce the overhead of correctly managing certificates and objects, the overall stability and integrity of the Internet’s interdomain routing will strengthen significantly.

## **2.14 BGPsec and RPKI Limitations**

In the first chapter as described briefly, current proposed solution have been proven to be insufficient, one of the main power points of securing the BGP and/or main arguments or proposals were BGP-Sec and RPKI, however for example recent article on [49] FCC suggest that there is a need for national plan in order to secure the Internet for this they call out on major ISPs and also article provides interesting findings on

these major ISPs where the ROAs against their traffic have been given, and it shows that major ISPs have fallen back on even supporting the RPKI and providing enough ROAs against their giant amount of traffic flow.

[80] provides problems with different BGP security proposals with defined metrics such as Interoperability, Implementation diversity, Hardware cost, Incremental deployment. For RPKI, RIRs have power to revoke certificates or delete ROAs, which could accidentally or intentionally "take down" IP addresses. Researchers suggest requiring multiple RIRs to agree on such actions (like needing two keys to launch a missile) to prevent abuse. Researchers suggest requiring multiple RIRs to agree on such actions (like needing two keys to launch a missile) to prevent abuse. As main adoption problems, slow rollout with only 30% of the networks (ASes) use RPKI as of 2023 per Cloudflare, and they also suggest Chicken-and-egg problem, on adopting RPKI if others don't Networks wait for others to act first, since there needs to be unanimous on adoption of the RPKI which is almost impossible globally, and human errors share major part of the drawbacks of the RPKI, with misconfigured ROAs (e.g., typos) caused outages, making some operators distrust RPKI.

As per the BGP-Sec it only works if every AS on the path uses it, if even one AS doesn't support BGP-Sec, the chain breaks, attacker can exploit this by pretending they don't support BGP-Sec, forcing the network to fall back to insecure BGP. Networks can't fully switch to BGPsec until everyone else does. Meanwhile, they must still support old BGP, again this creates a chicken-and-egg problem: Why adopt BGPsec if others haven't Signing/verifying signatures is computationally expensive. Most routers aren't powerful enough to handle this. Network operators would need to replace or upgrade routers, which is costly and time-consuming. Researchers found that if only some ASes adopt BGPsec, it offers little security benefit and might even create new loopholes. BGPsec fixes a critical flaw in BGP by securing the entire path, not just the origin (like RPKI). However, its success depends on global coordination every major AS must adopt it simultaneously, which is unlikely in the near future. For now, RPKI (origin validation) is easier to deploy, while BGPsec remains a "gold standard" held back by practical hurdles. A simple metaphor that could help understand the problem with the BGP-Sec could be the travel of a package, BGP-Sec is like a verified chain of custody for a valuable package. Every person who handles the package must sign a logbook. If one person refuses to sign, the chain is broken, and you can't trust the package's journey. Until everyone agrees to sign, the system can't work [80].

## **2.15 BGP Attacks, Route Leaks, Route Hijacks**

In any interdomain routing environment, the highly flexible yet largely trust-based design of BGP exposes networks to a variety of threats that can disrupt connectivity on a global scale. One classic example is the prefix hijack, where an AS illegitimately

announces ownership of a prefix it does not control. Because BGP takes these announcements at face value, traffic for that prefix flows to the malicious AS often resulting in a “blackhole” if packets are dropped, or in an even more damaging interception if the hijacker forwards the traffic onward after inspection. These incidents can be amplified by subprefix hijacks, in which a malicious AS advertises a more specific (longer) prefix than the legitimate advertisement. Due to BGP’s “longest prefix match” rule, nearly all traffic for that subprefix is misdirected, making subprefix hijacks a particularly potent exploit. Prefix hijacks may be caused by outright malicious intent or simple misconfiguration; both can yield the same result: traffic moved away from its rightful destination and global disruption of services.

Route leaks represent a similarly pervasive threat, in which valid routes are announced to parts of the internet that should not receive them violating usual export policies. A typical example involves a small ISP leaking customer routes to a lateral peer or higher-tier provider, inadvertently attracting traffic that never should have been routed through it. The Verizon–Cloudflare leak in 2019 demonstrated how a single erroneous announcement could propagate widely and degrade performance across major internet segments. Because leaks often occur without forging the origin AS, they are not blocked by origin-validation mechanisms like RPKI-ROV. Instead, they require a check of each AS hop, ensuring the path is consistent with authorized provider–customer relationships. These scenarios underscore why new proposals such as AS Provider Authorization (ASPA) have emerged: ASPA cryptographically enumerates which upstream providers are valid for each AS, making route leaks far easier to detect and filter at scale.

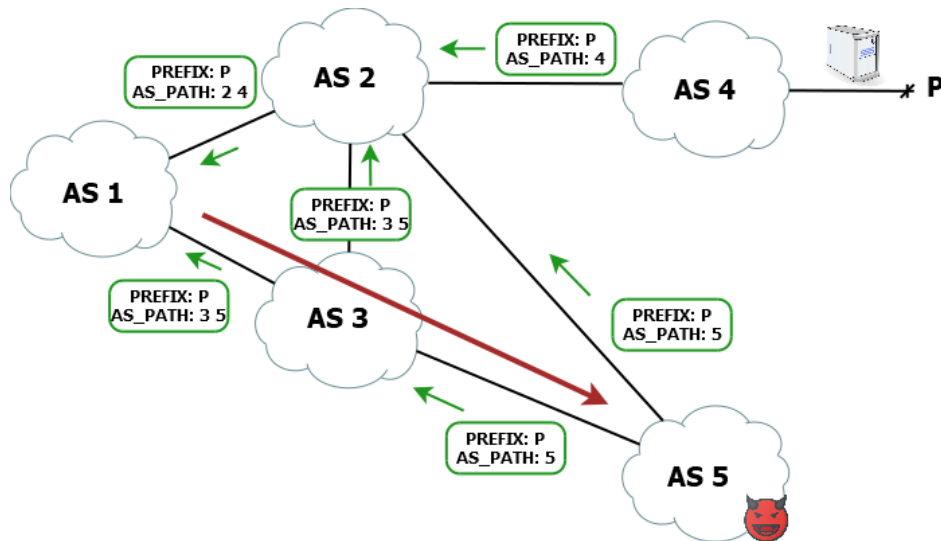


Figure 8: A malicious BGP hijack occurs when AS 5 falsely announces a route to destination P, causing AS 3 and AS 1 to mistakenly forward traffic to AS 5 instead of AS 4. Depending on AS 5’s intent, the hijacked traffic may be dropped, inspected for sensitive data, or used to impersonate services and steal credentials. BGP Route Hijack Incident

From a broader perspective, local vulnerabilities at the TCP level also remain a serious concern. BGP inherits all of TCP’s fragility: spoofed TCP RST messages, replay attacks, or session hijacks can reset critical BGP sessions and trigger route flapping, leading to significant instability. Without robust session protection (e.g., through TCP-AO) and continual monitoring, these disruptions can cascade, causing repeated reconvergence and even route dampening. Researchers have further identified stealthier hijacks that evade traditional BGP collectors: by selectively poisoning AS paths, an attacker prevents malicious announcements from reaching vantage-point monitors (e.g., public route collectors), while still diverting traffic from unsuspecting segments of the internet. Tools like Oscilloscope leverage [81] data-plane measurements monitoring RTT changes, for instance to catch these subtle “forged origin” or interception attacks that might escape purely control-plane-driven detectors.

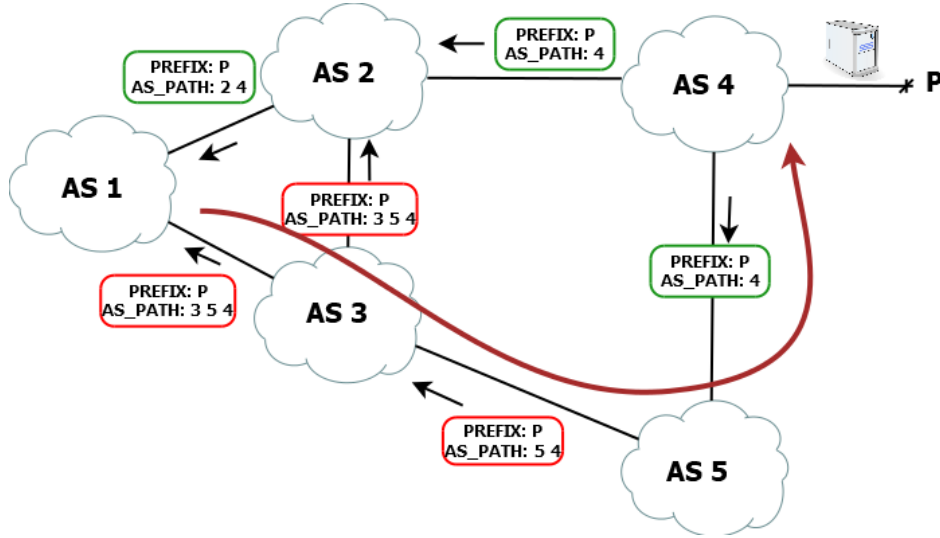


Figure 9: AS\_PATH forgery hijacks involve an attacker (e.g., AS 5) manipulating the AS\_PATH by falsely appending the legitimate origin (AS 4) to evade detection. This makes it harder for monitoring systems to identify the hijack compared to simple mis-origination attacks. BGP Route Hijack Incident

Overall, attacker models built on formal language theory, real-world route leak incidents, and active data-plane probing converge on the same conclusion: BGP’s policy-driven nature, combined with its trust assumptions, creates fertile ground for both accidental misconfigurations and deliberate exploits. Efforts like RPKI, BGPsec, and ASPA aim to authenticate origins, maintain path integrity, or verify provider–customer links; each partially addresses the spectrum of vulnerabilities. However, these security additions also depend on reliable repositories and robust Relying Party software that is not itself susceptible to poisoning or denial-of-service tactics. In practice, a layered defense encompassing cryptographic validation (RPKI-ROV, ASPA, BGPsec) and real-time data-plane checks (like RTT analysis or anomaly detection) is essential for mitigating the complexity of route leaks, hijacks, and malicious manipulations

at internet scale.

## 2.16 Path Plausibility and Path Selection Algorithms

Both path plausibility (PP) and path selection (PS) have similarities, each of the on a big picture view have the same objective, that is choosing the most viable path though they have different concerns at their core, such that PP has security first approach on paths, and result on this is, different validation methods have been developed, one of them being ASPA and the other one is AS-Cones .

On the other hand PS has best path selection on priority of the efficiency of the traffic that is choosing the best path for traffic to reach its destination as soon as possible, this is done through complex policy and path weighting approach. Specifically PS have concerns over

- Suboptimal routing paths in large networks, stemming from BGP's tendency to prioritize non-ideal routes, can impede high-speed data transfers.
- Enhanced data distribution through load balancing and scalable infrastructure improves resource utilization and operational efficiency.
- Connectivity interruptions from link failures contribute to latency issues, delaying data transmission across networks.

In this thesis we are more concerned with the plausibility of the path, that is, security and robustness against the hijacks and route leaks. A similar method to PP, is path end validation; it can be considered as an extension to RPKI, and in response to the low adoption of the BGP-Sec. It intuitively validates the last hop in the path, meaning consider the following path [AS1 AS2 AS3], path-end validation here validates the validity of the path or next hop specifically, consider next hops is AS4, so it checks the origin that is AS3 and AS4 that next hop or last hop, Extends RPKI to certify not only the origin AS of an IP prefix but also the ASes authorized by the origin to directly connect to it. This creates "signed authorizations" for the immediate neighbors of the origin AS [82].

It basically uses the RPKI to certify the validity of the for approving the last hop neighbors. So in this case When a BGP router receives a path advertisement, it checks if the origin AS is legitimate (via RPKI). And then if the AS directly before the origin (the "penultimate AS") is authorized by the origin AS via these new certificates.

Path-end validation enhances interdomain routing security by focusing on validating the legitimacy of the final hop in a BGP path, preventing attackers from falsely claiming direct connections to victim networks. This approach forces attackers to use longer, less-preferred routes due to BGP's inherent bias toward shorter paths,

reducing the effectiveness of malicious route propagation. Unlike more comprehensive solutions that require validating entire paths and significant infrastructure changes, path-end validation operates within existing systems, offering immediate benefits even with partial adoption. It prioritizes practicality over perfection, addressing critical vulnerabilities without the complexity of full cryptographic validation. By extending existing security frameworks to authenticate key path segments, it provides a deployable, incremental improvement that balances security gains with real-world feasibility, positioning itself as a viable alternative to more resource-intensive protocols [83] [81] [84] [58] [57] [56] [57].

## 2.17 Autonomous System Provider Authorization (ASPA)

ASPA (Autonomous System Provider Authorization) adds a crucial layer to inter-domain routing security by detecting and preventing route leaks as well as certain hijacks that rely on forged AS paths. While RPKI-ROV (Route Origin Validation) already verifies which AS is authorized to originate specific IP prefixes, ASPA extends security further up the AS path by revealing whether the sequence of upstream providers is valid. This emphasis on verifying who is allowed to provide transit without requiring changes at the origin AS makes ASPA particularly effective against route leaks, which often occur when a route intended only for a customer cone accidentally (or maliciously) propagates to lateral peers or higher-tier providers.

**Why ASPA:** Traditional BGP is largely trust-based: ASes trust that the AS\_PATH attribute in route advertisements is correct. This leaves BGP vulnerable to route leaks and path forgery. In a route leak scenario, a network that should not provide transit to others (for instance, a small ISP receiving a route from one transit provider) unintentionally or deliberately exports that route to its other providers or peers. Even if RPKI origin validation is in place, it only checks the legitimacy of the prefix’s origin AS, not the entire path. ASPA closes that gap by specifying which ASes are authorized to be “upstream” (i.e., providers) for each AS, so receiving routers can detect abnormal AS transitions in the path.

**ASPA Objects and RPKI Infrastructure:** ASPA is based on the RPKI (Resource Public Key Infrastructure). Each AS that adopts ASPA creates an ASPA record listing all valid upstream providers (and non-transparent route server ASes, if it is an RS-client). The RPKI repository then stores these ASPA objects, alongside ROAs (Route Origin Authorizations) and other signed data. Routers fetch ASPA data through local caches similar to how ROV operates and evaluate any newly received BGP routes against these ASPA records.

Concretely, a CAS (Customer AS) publishes in RPKI which ASes are its providers. This set is the ASPA’s “SPAS” (Set of Provider ASes). For example, if AS64496 connects to two different upstream ASes say AS64497 and AS64498 AS64496’s ASPA

indicates both AS64497 and AS64498. Whenever a receiving router later sees a path that claims “64496 is downstream of 64499,” that router will confirm whether 64499 really appears in 64496’s ASPA; if not, the route is flagged invalid or unknown, depending on the policy.

**Route-Leak Prevention via ASPA:** A key strength of ASPA is its ability to detect route leaks whether accidental or malicious. If a path shows that an AS widely considered a “customer” has suddenly advertised routes to a higher-tier provider or a lateral peer, the mismatch appears when the ASPA-based algorithm verifies which upstream ASes are legitimate. Specifically:

1. **Upstream Verification** – If you receive a route from a neighbor that is supposed to be a customer, you expect the AS\_PATH to be an “up-ramp,” meaning each AS hop from origin to you is a valid customer-to-provider relationship. ASPA ensures each link in that chain is consistent: if AS X claims that AS Y is its provider, ASPA must confirm “Y” is in “X”’s set of valid providers.
2. **Downstream Verification** – If you receive a route from a provider, you allow for a possible combination of “up-ramp” (from origin) and “down-ramp” (toward you), but the sum of those ramps must not exceed what is plausible based on each AS’s ASPA. A typical route from an origin might move upward through one or more providers, reach a higher-tier network, and then travel back down to you. ASPA ensures that each upstream or downstream hop is legitimate; if an extra “valley” appears (meaning a former customer or lateral peer is re-exporting the route upward), the check fails.

To understand how ASPA fits into the broader landscape of inter-domain routing protections, it is helpful to consider some of the other security mechanisms that address different facets of BGP’s vulnerability.

**RPKI-ROV:** ROA-based origin validation confirms that the prefix originated from the AS that the holder of the IP address space authorized. It stops simple “incorrect origin” attacks but does not handle manipulations in the rest of the path. ASPA complements ROV by validating whether each step in the AS\_PATH (particularly customer-to-provider transitions) is legitimate.

**BGPsec:** BGPsec cryptographically signs each AS hop, which can detect or block entire path forgeries. However, BGPsec does not inherently solve route leaks (which can form “valleys” while retaining an otherwise authentic set of AS signatures). In addition, BGPsec faces major deployment hurdles because it requires every AS on a path to sign or verify updates. ASPA can be deployed incrementally: if only the receiving AS and the origin’s providers have ASPA objects, you can already detect certain forged-path hijacks or route leaks.

**Peerlock:** Large backbone operators sometimes use “peerlock” to filter unexpected routes supposedly learned through major providers or peers. However, peerlock relies on manually configured rules and is prone to human error; it also does not systematically validate all smaller providers or the more complex sets of upstream connections. ASPA, by contrast, is managed within RPKI. Each AS publishes exactly who can serve as its provider, achieving a more automated and scalable solution that covers both large and small ASes.

**Detection and Policy Outcomes:** When an ASPA-enabled router learns a route, it applies the ASPA-based path verification algorithm:

1. **Check the AS\_PATH** – The router extracts the list of unique AS numbers in the path (e.g., AS5 → AS4 → AS100 → AS200 → AS300), ignoring aggregator sets or confederation segments in typical BGP deployments.
2. **Identify Upstream vs. Downstream** – If the route is received from a “customer” (or a lateral peer session), the entire path should be an up-ramp. If it comes from a provider, you can see a combination of up-ramp from the origin plus a down-ramp back to your AS.
3. **Invoke the ASPA Records** – For each adjacent pair (X, Y) in the path, check if Y is in X’s “Provider Set” from X’s ASPA object. If you find a mismatch for example, X claims Y as a provider but Y does not appear in X’s SPAS that portion of the path is suspect.
4. **Classify** – If the path structure is invalid based on these checks, the route is marked “Invalid” and typically discarded. If ASPA data is incomplete for part of the path, the result might be “Unknown” or “Not Provider+,” which operators often treat similarly to “Valid” (though they can set different local preferences if they wish).

ASPA’s security properties stand out when considering partial adoption scenarios, as they demonstrate tangible safeguards even when only a subset of networks deploy the mechanism. For instance, if two autonomous systems, let's call them A and B, both adopt ASPA, any accidental or malicious route that is leaked between them can be flagged once it arrives at B via a path that does not match the ASPA-authorized relationships. Because ASPA checks each AS hop and verifies that every provider is genuinely listed, misaligned paths reveal themselves as soon as they are inspected. Beyond detecting route leaks, ASPA also makes it harder for attackers to fabricate parts of the AS\_PATH: if a malicious AS pretends to be an authorized provider of another network, ASPA verification exposes the inconsistency. Larger path signature solutions such as BGPsec can offer more comprehensive coverage by cryptographically

signing each path hop, but they often demand higher resource investments and wider coordination. ASPA, in contrast, can be effective even if only certain major providers implement it, as it still prevents spurious announcements from crossing between large segments of the internet. Over time, as more tier-1 and tier-2 ISPs adopt ASPA, the benefits compound because any route that travels across these well-connected networks must align with the ASPA records that have been published.

From an operational standpoint, deploying ASPA hinges on careful setup and ongoing maintenance of the ASPA objects in RPKI. Each AS needs to register a record listing all its upstream providers; if a network does not have any providers, it can create an AS0 ASPA record that signals it is entirely self-sufficient or only offers transit downstream. Keeping this information up to date is crucial, because if an AS fails to update its ASPA after changing providers, legitimate routes risk being inadvertently dropped, or operators could overlook certain leaks. Moreover, in situations where two networks offer transit to each other often referred to as complex or mutual-transit relationships each one includes the other in its ASPA set, clarifying that either side may function as a provider for particular routes. Once ASPA-based validation is in place, local policy decisions determine how to handle routes classified as “invalid,” “unknown,” or “valid.” Many operators treat paths found invalid by ASPA checks as ineligible for best-path selection, while unknown or valid paths are typically allowed. Networks seeking a more refined approach might decide to prefer valid paths over unknown ones, but in most cases even this minimal differentiation dramatically reduces exposure to route leaks.

**ASPA’s Position in the Overall Security Landscape:** ASPA is a powerful mechanism to curtail route leaks and hamper certain kinds of forged-path hijacks. It does not replace every security measure:

- **RPKI-ROV** stays essential for origin validation.
- **BGPsec** covers a broader range of path manipulations but requires ubiquitous participation.
- **Data-plane Probing** or real-time monitoring can detect stealthier attacks and confirm actual forwarding paths.

In practice, combining ASPA (for provider-based path validation), ROV (for origin security), and cautious local policies (like max-prefix limits, prefix filters, or IRR-based checks) forms a robust layered defense. Full global adoption is never immediate, but even partial deployment especially among large upstream networks significantly reduces the scope and impact of route leaks. The hierarchical, RPKI-backed approach means smaller networks can quickly benefit by publishing and enforcing ASPA records without complicated bilateral coordination. As more ASes provide ASPA data, the system grows in coverage, moving inter-domain routing closer to a “valley-free” environment that is both scalable and significantly more secure than classic BGP.

## 2.18 Related Research

Below, we delve into two key facets that underpin modern inter-domain routing research: BGP security and large-scale BGP simulations. While BGP security investigates mechanisms and best practices to safeguard global routing against hijacks, leaks, and other threats, the simulation-based studies focus on modeling network topologies and evaluating protocol performance under realistic conditions. By examining both areas, we gain a holistic view of BGP’s vulnerabilities, potential countermeasures, and the practical challenges of deploying these solutions at scale.

### 2.18.1 BGP Security Related Papers

The first survey covers Internet measurement for cybersecurity, offering a taxonomy across Internet layers and discussing real-world threats, data collection hurdles, and measurement challenges. It emphasizes the difficulty of sharing data among operators and the lack of explicit research gaps, but underlined measurement’s vital role in tracking vulnerabilities and emerging attacks [85]. Another paper introduces GMPI, which infers AS-level paths from passive data using heuristic path generation and a dual-attention network. It highlights incomplete information about real AS routing policies and inconsistent data across sources, showing that carefully designed ML can help stitch together partial knowledge and outperform prior methods. [86] Kirin describes a large-scale IPv6 prefix de-aggregation attack that distributes announcements via thousands of IXP connections to overwhelm remote routers. It combines theoretical optimization, real micro-scale tests, and BGP data analysis, concluding that common security checks do not fully block mass route injection and raising awareness of cheap but powerful BGP floods. [87] RoLL speeds up route leak localization by extracting AS triplets from BGP updates, using features like AS distance, degree, and address space alongside a machine-learning model. It locates leaks quickly and accurately, outperforming past methods that either took too long to detect leaks or could not pinpoint the guilty AS. Another study measures RPKI delays across the management, control, and data planes, showing how uneven RIR processes or slow relying-party refresh intervals can delay ROA creation and deletion. While explicit gaps are not stated, the authors suggest that consistent best practices and coordinated data publication intervals would make RPKI protections timelier and more reliable [88] [89].

A Bayesian approach tackles noisy BGP path data, weighting route collector error rates to infer AS topology. It leverages both positive and negative edge observations, showing that different vantage point coverage drastically alters accuracy. Although not framed as a gap, there is potential for refining vantage locations or using the method for anomaly detection. TopoScope mitigates bias from fragmented vantage points, using ensemble voting, Bayesian Networks, and detection of hidden “detour couples” to improve AS relationship inference. It highlights the uneven vantage distributions

and acknowledges that hidden or rarely observed links hamper accurate relationship mapping, prompting more advanced measurement or new vantage data. [90] [91] A replication study revisits Wang and Gao’s 2003 work, revealing that selective route announcements have risen and local preference assignments no longer strictly follow Gao-Rexford norms. Internet flattening and widespread content delivery changes complicate modeling. Frequent BGP policy surveys might clarify how real-world interconnections deviate from classic assumptions. [92] The Hanging ROA (h-ROA) scheme compresses route origin authorizations more securely than maxLength-based approaches and is said to reduce router synchronization cost significantly. While it does not flag open research questions, it points to scalable ROA encoding as RPKI grows and underscores that careless maxLength usage can compromise origin validation. [93] A validation-bias analysis shows that coverage varies widely by region, with LACNIC often underrepresented, inflating accuracy reported by major inference algorithms. The authors note that precision can drop significantly for Tier-1 peering links when tested more thoroughly, calling for more operator engagement and vantage expansions to minimize skewed datasets [94].

ASIRIA combines IRR-based relationship inference, route validation, and an alarm system to detect BGP route leaks in near-real time. It detects most leak events by de-prioritizing suspicious routes instead of discarding them, but depends on good IRR annotations. The authors suggest future integration with ML for better coverage of unmarked routes [95]. Two works from the same authors employ deep learning for hijack detection. One uses Source-Aware Self-Attention across geolocated traceroutes, pointing out anomalies in route segments, while another (AP2Vec) learns ASN and prefix embeddings to detect major structural changes. They demonstrate high hijack detection rates with minimal false alarms and show promise for real-time deployment [96]. Lastly, injected sub-prefix hijacking manipulates communities and path poisoning to pollute only specific neighbors, making the attack invisible to monitors. Simulations reveal that Tier-1 ASes can hijack a quarter of global ASes in stealth mode, while smaller ISPs hijack fewer. The authors propose partial mitigations but emphasize that existing defenses struggle against such targeted sub-prefix abuses.

### 2.18.2 BGP Simulation Related Papers

The first paper describes BGPpy, a Python-based BGP security simulator that enables realistic attack/defense studies on standard machines. Its main findings are that BGPpy is flexible, extensible, and efficient, helping researchers and educators simulate numerous BGP threats. Its limitations include performance drawbacks due to Python’s overhead and partial coverage of low-level BGP details, though future Rust bindings and user customizations are planned. The paper does not pinpoint particular open research questions; rather, it centers on implementing a robust, benchmark-ready simulator [97]. Next, an MPLS VPN/BGP educational experiment is introduced

to strengthen students’ theoretical and practical abilities. Students gain hands-on experience with MPLS/BGP, and results suggest improved experimental skills and test performance. MPLS VPN outperforms L2TP VPN in metrics like jitter and packet loss. While IPv6 and SDN support are missing, the authors do not highlight other unresolved research needs [98]. A large-scale BGP simulation setup emphasizes evaluating BGP convergence. Its findings: single-homed AS links trigger slowest convergence, and transient routing loops heavily impact connectivity. However, simulating only one prefix may be incomplete, and ignoring message processing delays can give optimistic results. The authors pinpoint that more realistic topologies and multi-prefix simulations are needed for scalable and accurate modeling [99].

BGP++ is another tool that merges realism and scalability for BGP simulations. It significantly reduces memory usage (up to 62%) through compact routing structures and integrates with pdns for distributed simulations. The authors see existing BGP simulators as limited in either detail or scale. They call for refined methods to address BGP’s many known weaknesses but note that BGP++ itself remains a step forward for exploring new routing architectures in near-realistic conditions. [100] TurboNet leverages programmable switches to faithfully emulate both data and control planes in potentially large topologies. Its key outcome is accurate emulation of high-speed links, low-latency paths, and a moderate number of BGP agents. Still, hardware resource limitations and inability to emulate complex host behaviors are major drawbacks. The authors also discuss that simulators, emulators, and testbeds all have inherent gaps, prompting their single-switch hardware-based approach [101]. Finally, ACORN focuses on scalable formal verification of distributed control planes, introducing a nondeterministic routing abstraction and a symbolic-graph-based SMT encoding. It achieves large-scale verification (tens of thousands of routers) within an hour and obtains up to 323x speedups over conventional methods. The paper’s open points include dealing with failures and developing counterexample-guided refinement. It does not directly outline other “research gaps,” but it does leave room for future incorporation of robustness scenarios (like link/device failures) [102].

## 3 Methodology and Experimental Design

### 3.1 Simulation Framework Architecture

To evaluate the deployment strategies of the ASPA protocol, and its robustness against different kind of anomalies on the BGP configuration and topology such as leaks, or hijacking attempts, and to have a quantitative output, out of it, we need to develop a simulator, this can be a simple simulator that is only focused on this specific task or it can be a comprehensive and sophisticated one that will be beneficial and could be modular and dynamic so that it will adopt to different type of protocol extensions

or research based protocol implementations and different experimental procedures, we prefer the latter. A rough overview of the simulator framework/toolkit architecture can be found on the below figure. Simulator basically formed of six main pillars: Engine, CLI Interface, Database Layer, Protocol Framework, Experiment System, and Utility Services. At the center of this, engine of the simulator resides, engine is responsible for, many things, that it is responsible for management of the resources, these resource managements is mainly attributed to the core graph implementation, is to not only provide Inter-AS routing security simulation it also aims for complex internet architecture that encapsulates both inter and intra AS routing topologies. The Engine Core serves as the central coordinator, managing the graph implementation for topology representation, memory optimization systems, RPKI/ASPA security infrastructure, event bus system for asynchronous communication, and pipeline architecture for modular processing.

The Protocol Framework provides the base BGP protocol implementation along with plugin support for protocol variants like ASPA and Route Leak, supported by a policy engine for routing decisions and deployment strategy management. The CLI Interface handles command processing, configuration management, interactive topology exploration, and progress visualization through spinners and loading bars. The Database Layer provides PostgreSQL integration for topology data storage, query system for network relationships, and historical result storage. The Experiment System enables factory-based experiment registration, trial execution framework, metrics collection and analysis, while the Utility Services provide essential support through logging systems, memory pool management, thread safety mechanisms, and performance profiling tools. This modular architecture with the event bus as its backbone enables flexible, asynchronous operation while maintaining clean separation between components. Each element can operate independently while coordinating through well-defined interfaces, creating a robust platform for BGP security research and experimentation.

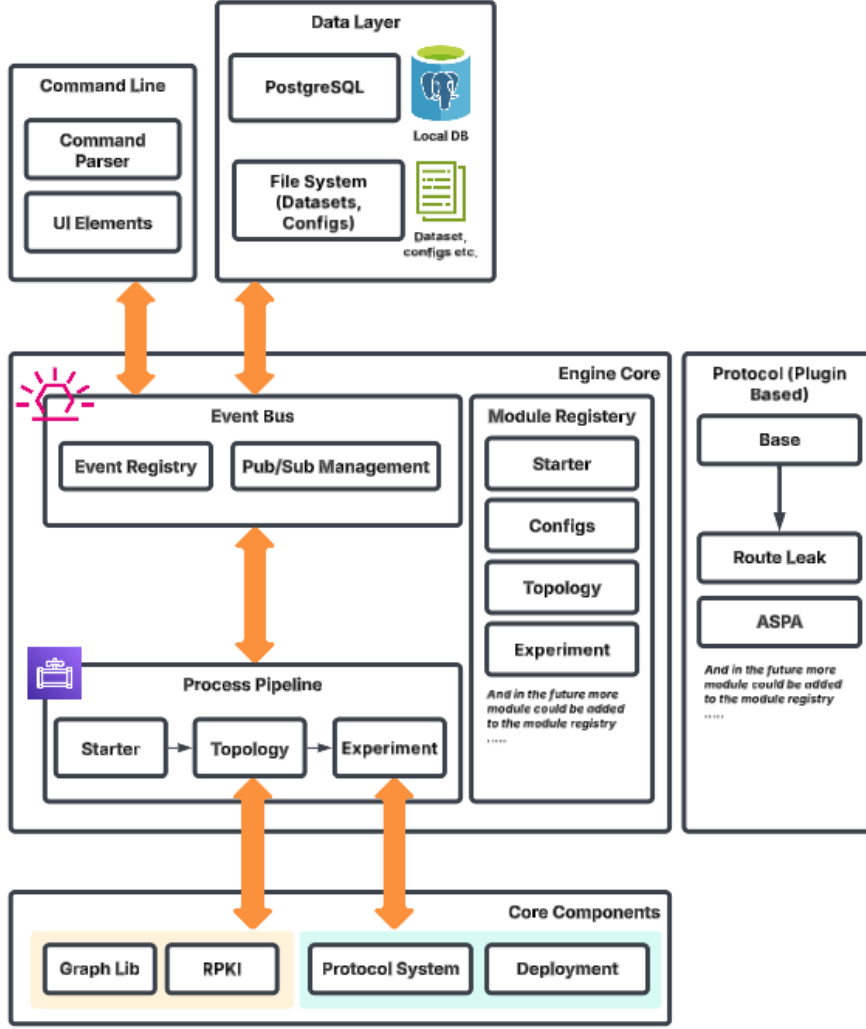


Figure 10: BGP Route simulator architecture

As for the engine pillar of the simulator, it manages this topology and underlying graph resources, and it also manages the experiments as well, it also encapsulates topology related context such as RPKI, deployment and post operations after the topology is created again these will be discussed under related chapters in more detail. It's worth mentioning here the core engine structure, we've adopted an internal bus for communication with event driven architecture and on top of it a pipeline structure. Pipeline structures have modules named as stages, each module operating inside a pipeline is named as stage. Stages could or could not call events from the internal bus, to communicate and inform next stages that it has complicated what it is obligated for. These stages that subscribed to an event publish to the internal bus once the task of the stage is done to the internal bus. Not all of the stages publish or subscribe to any event on the registry as mentioned before, that is because avoiding internal bus to make it a linear structure, calling events after each stage is completed. It philosophically decided that only major stages calls the internal bus because of avoiding linear calls that will resemble function calls and will be against the nature of the internal bus structure.

So basically the internal bus operates as a backbone communication structure for the whole simulator.

It also operates and published and subscribes for defined events, for other features of the simulator namely, this async communication allows flexible and non-blocking operations on the CLI side, furthermore this pipeline + internal bus structure is also capable of extending the simulator furthermore to UI and API features since it can be thought of as the core structure of the engine, and all of the operations on it is non-blocking since every event has its own thread and once an event is published the rest is not concerned of the caller. To exemplify this consider the following operation, calling a command from CLI will trigger either the pipeline structure from the beginning with configurations and the feeding it to the rest of the pipeline then at each stage module also can publish to events that it's subscribed to, for example it can trigger UI animations on the CLI side such as loading, progress bars spinners, or even interactive exploration. Currently the pipeline structure is described on the below figure. It's a simple configuration that starts the pipeline and feeds it with the relevant information needed for initialization. The pipeline is fired up through a StartupEvent, which contains initial topology configuration data needed to bootstrap the simulation. This event triggers the TopologyModule's initialization sequence, which begins by loading network structure data from the PostgreSQL database. The TopologyModule then processes this data to construct the core graph and router objects. And then it forwards these configurations once they are set up to the next stage through the internal bus with its payload, the internal event communication bus also allows moving payloads in between stages or where the event is published.

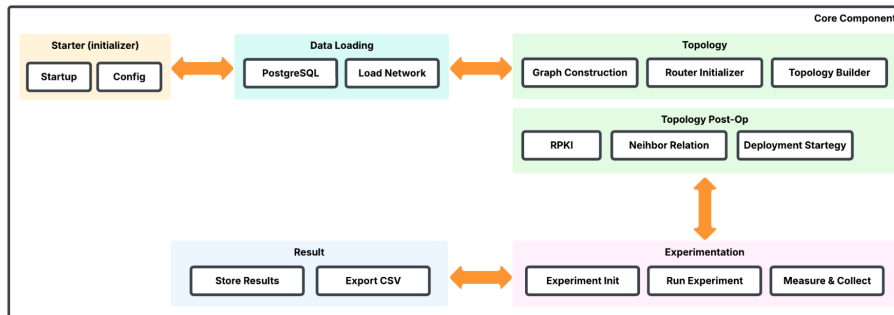


Figure 11: Core components of the simulator and interactions between them

Once the configuration is fed through the pipeline it first calls the necessary data structures, and datasets or data sources main point of the data is currently set to PostgreSQL database that holds the relations data and node data on a separate table. Information of the data only and only changed on the runtime, no I/O operations done, to keep the simulation results robust, so no updates and deletions are done on the simulation on runtime done on the database. So this means at each run and initialization of the simulation topology information will be static, this could be later

on modified to include frequent read/write operations throughout the simulator but it is not required on the objective of the simulator furthermore it expected to slow down the simulation majorly since I/O operations will slow the experiments anyway, so the simulation and experiments are limited to efficient memory operations and structured memory organization through the underlying graph implementation. And once the major initialization stages are done (configurations, database, related data loading etc.) it moves in to main stages that implemented initializes, major instances, of the simulator this is mainly the topology and underlying graph that used to build the topology, topology is formed of routers, these routers represents the ASes in our topology, it is named as router since the simulator needs a flexibility in terms of experimentations, so we aimed to not only limit only BGP ASes and inter-AS routing simulator so the graph can have a generic type as node so that users and/or researcher can later on implement different type of network element with their internal logic implemented under their classes and later on figure out topology construction using the defined formats.

Once the topology, graph and it's routers are initialized, experiment stage (another module in the pipeline) is called, the experiment module is where it all happens, it first uses the post-ops of the topology, these are mainly RPKI, neighbor and deployment related initializations, depending on the definitions of the specific deployment strategies on each protocol, it updates the topology context that is the routers and what protocol they operate on, also deployment strategies can affect the whole infrastructure, it can change the RPKI definitions, RPKI objects defined under related class. Once the final experiment stage is called the experiment is run against the defined deployment strategies, experiment configurations, experiment configuration include the metrics and measurement algorithms as well, these measurements intuitively can be only derived from what the simulator already has in it, the more simulator calculates the more metrics and measurements hence more quantitative data can be obtained from the simulator, as it's initial version only routing information can be obtained, that hops, routes, paths and routing table information as well as the RPKI, relation and policy information a researcher can create custom metrics using these data and info from the topology, current metrics are all defined under the subsequent sections and all of them are enough to obtain the objective and aim of this thesis. Once the experiment is finalized, it records it to database for historical access, and it also allows modularized development approach, such as, since C++ isn't really suited for easily visualizing and modifying topologies or operating on the data, researchers or users can import the experiment results as CSV, from the database and use it to visualize it with Python or operate on the data easily. Also plots of this experiment are plotted using this approach as well.

### 3.2 Core Design Principles (Plugin System, Event Driven Architecture)

Before starting developing the simulator few design principles have been determined these are, a kind of a plugin system, event-driven architecture, with a modular and non-blocking structure, and low-level resource management through a central structure and all of this integrated through internal-bus as backbone communication and pipeline structure. To begin with, the simulator will serve for experiments on different type of research based protocols or protocols developed for research purposes in aid to the well known BGP problems, or AS network problems. We need to modularize and make the two main elements on this to achieve this goal. These two elements are experiments and protocols. Researchers, engineers, and users need to create their own custom, cutting-edge, research based protocols using the main data and classes of the topology and its underlying graph implementation to construct their own research template. In this thesis the main research goal is to see if ASPA is an effective solution with different deployment strategies against the threat landscape of the BGP, namely route leaks and hijacks. But with the core implementations, classes elements and modular structure of the simulator, researchers or engineers can also implement their own protocols such as a version of the ASPA that is also effective on partial deployment, with the usage of distributed algorithms, and then create their own experiment framework (this can be considered as a subclass but it's handled as factory structure with templates in C++, so metaprogramming is also included in this) and define their own deployment strategies (also not required to have a specific complex one, one can also deploy protocols to all router instances in the topology in a simple way or can prefer complex deployment structures such as specific AS numbers, or specific network elements), and test this experiment using their own custom metrics defined using the underlying available data on the simulator, and derive concrete, quantifiable results to reflect on their research. Also as mentioned above, experimentation needs to be modular as well, since defining experiments needs to be as modular as possible, we need to avoid defining experiments in a static way with varying methods and fields, to avoid these experiments being designed with a layer approach. Specifically, experiment is designed with a factory pattern The experiment system in the simulator is built around a modular, event-driven architecture that enables flexible execution of different BGP security experiments. At its core, the system uses a registration pattern combined with a factory method for experiment creation, which is integrated into the simulator's event-driven pipeline architecture. The base experiment interface `IExperiment` defines the fundamental contract for all experiment implementations, requiring them to handle trial execution, topology setup, and lifecycle management. This interface works in conjunction with the `expregister` class, which serves as a central registry for all available experiments and manages

their creation through a factory pattern. The experiment system's integration with the pipeline and internal event bus creates a sophisticated workflow for experiment execution. When an experiment is initiated, it triggers an `ExperimentStartEvent` through the event bus, which is handled by the `ExperimentModule` pipeline stage. This module manages the experiment lifecycle and coordinates with other system components through events. For example, it waits for a `TopologyReadyEvent` before beginning experiment execution, ensuring proper initialization of the network topology. The event-driven nature of this architecture allows for asynchronous experiment execution while maintaining loose coupling between components. Each experiment can publish `TrialCompletedEvents` as it progresses, enabling real-time monitoring and feedback through the simulator's user interface.

This design also facilitates the addition of new experiment types through the `REGISTER_EXPERIMENT` macro, which automatically handles the registration of new experiments with appropriate metadata and factory functions. The integration becomes particularly powerful in the context of complex experiments like route hijacking and route leaking simulations. For instance, when running a route hijacking experiment, the system can dynamically adjust the topology, deploy security measures like ASPA at varying rates, and measure the effectiveness of these protections - all while maintaining clean separation of concerns through the event system and pipeline architecture. Each component (topology management, experiment execution, result collection) operates independently but coordinates through the event bus, creating a robust and extensible system for BGP security research. The experiment system's architecture leverages two key design patterns: a factory pattern implemented through `expfactory` and a registration system simplified by a macro. The factory pattern is implemented using `std::function` to create experiment instances, allowing for dynamic creation of different experiment types at runtime. This is defined through the `expfactory` type alias which specifies a function signature that takes input/output queues, topology, and parameters, returning a unique pointer to an `IExperiment`. The factory approach enables the system to create specific experiment implementations without directly exposing their constructors, maintaining encapsulation while providing a flexible way to instantiate different experiment types. The registration system is streamlined through the `REGISTER_EXPERIMENT` macro, which significantly simplifies the process of adding new experiment types to the simulator.

This macro expands to create a lambda function that serves as the factory for a specific experiment type, automatically handling the boilerplate code needed for registration. When used, it calls `expregister::instance().registerExperiment()` with the experiment's name, description, and parameter list, along with a lambda that constructs the specific experiment type. The `expregister` class maintains these registrations in an `unordered_map`, where each entry contains metadata (`expinfo`) about the experiment including its name, description, parameters, and factory function. This combination

of factory pattern and macro-based registration creates a type-safe and user-friendly system for extending the simulator with new experiment types, while maintaining a clean separation between the experiment interface (`IExperiment`) and specific implementations. For example, registering a new route hijacking experiment would look like:

```
REGISTER_EXPERIMENT(RouteHijackExperiment, "route-hijack",  
    "Simulates ASPA deployment against hijacking", "deployment_type");
```

This single line handles all the necessary setup and registration, while under the hood it creates the appropriate factory function and stores all required metadata.

The protocol implementation in the simulator follows a hierarchical, modular design pattern that separates routing policies from protocol behavior while maintaining extensibility through a common interface. At the core of this architecture are the base interfaces `IProto` and `IPolicy`, which define the fundamental contract for all protocol implementations. This design allows for the implementation of different BGP protocol variants (Base BGP, ASPA, and route leak scenarios) while sharing common infrastructure and validation logic. Each protocol implementation consists of two main components: a protocol class that handles the overall behavior and a policy class that defines specific routing rules and validation criteria.

The protocol system uses a factory pattern through the `protofactory` class, similar to the experiment system, but with a focus on protocol instantiation. Each protocol variant (Base, ASPA, Leak) implements the core interfaces while adding specialized behavior. For example, the ASPA protocol implementation (`ASPAProtocol` and `ASPAPolicy`) extends the base interfaces to include RPKI integration and ASPA-specific validation rules. The policy implementations use a configuration-based approach through the rules enum and config struct, allowing for flexible definition of routing preferences and validation criteria. This is particularly evident in how each policy class implements the `validate`, `prefer`, and `forward` methods differently based on their specific requirements. For instance, the base policy implements standard BGP decision processes, while the ASPA policy adds additional validation steps for provider authorization.

The deployment aspect of the protocol system is handled through the `IDeployment` interface and its implementations (`RandomDeployment`, `SelectiveDeployment`, etc.), which manage how protocols are distributed across the network topology. Each deployment strategy can customize how protocols are assigned to routers, enabling experimentation with different deployment scenarios. For example, the ASPA deployment implementations allow for both random and selective deployment patterns, with configurable percentages for both object and policy deployment. This modularity in deployment strategies, combined with the protocol and policy abstractions, creates a flexible system for simulating various BGP security scenarios and studying their effectiveness under different conditions.

The policy engine in the simulator implements a modular and flexible design that enables the separation of routing policies from protocol behaviors. The architecture is built around the `IPolicy` interface, which defines the core contract for routing policies through three fundamental methods: `validate`, `prefer`, and `forward`. This separation allows different BGP policy implementations to be swapped in and out of protocols while maintaining consistent behavior patterns. The policy engine's modularity is demonstrated through its implementation in various protocol types. Each protocol (`Base`, `ASPA`, `Leak`) has its corresponding policy class that inherits from `IPolicy`, but can implement its own specific validation and preference rules. For example, the `BasePolicy` implements standard BGP decision processes, while `ASAPolicy` adds RPKI-based provider authorization checks. The policy engine uses a configuration-based approach through the `rules struct`, which encapsulates routing preferences and validation criteria. This structure contains evaluator functions and preference indicators (whether higher values are better), allowing for flexible definition of routing rules. This is particularly evident in how each policy type implements its own `initRules()` method to set up its specific routing preferences. The integration between protocols and policies is achieved through composition, where each protocol holds a unique pointer to its policy object. This is established through the `IProto` constructor, which takes ownership of a policy implementation. For instance:

```
class ASPAProtocol : public IProto {
    ASPAProtocol(std::shared_ptr<rpki> rpki)
        : IProto(std::make_unique<ASAPolicy>(rpki)) {}
    // ...
};
```

This design allows protocols to delegate all policy decisions to their policy objects while maintaining encapsulation. The deployment system further demonstrates this modularity by allowing different combinations of protocols and policies to be distributed across the network topology. For example, the `RandomDeployment` and `SelectiveDeployment` classes can deploy different protocol-policy combinations based on various strategies, showing how the policy engine can be used flexibly in different deployment scenarios.

### 3.3 Event System and Module Communication

The event-driven architecture in the simulator is built around the `ebus` class, which implements a sophisticated asynchronous message bus system. At its core, the bus maintains a priority queue of events (`mEventQueue`) and a map of subscribers (`mSubscribers`), with thread-safe operations ensured through mutex locks and condition variables. The system uses a dedicated dispatcher thread created in the `start()` method, which runs the `dispatchLoop()` continuously to process events according to their priorities

and timestamps. The subscription mechanism is implemented through a templated subscribe method that enforces type safety through static assertions:

```
template <typename T>
uint64_t subscribe(eventCallback callback, eventPriority priority =
eventPriority::NORMAL,
                    eventFilter filter = nullptr) {
    static_assert(std::is_base_of<IEvent, T>::value, "T must
inherit from IEvent");
    std::lock_guard<std::mutex> lock(mMutex);
    auto sub = std::make_shared<subscription>(callback, priority,
filter, std::type_index(typeid(T)));
    mSubscribers[std::type_index(typeid(T))].push_back(sub);
    return sub->id;
}
```

Event publishing is handled through the templated publish method, which supports both immediate and queued event dispatch:

```
template <typename T>
std::future<void> publish(std::shared_ptr<T> event, bool
immediate = false) {
    static_assert(std::is_base_of<IEvent, T>::value, "T must
inherit from IEvent");
    std::lock_guard<std::mutex> lock(mMutex);
    if (immediate) {
        return std::async(std::launch::async, [this, event]() {
            this->dispatchImmediate(
                std::static_pointer_cast<IEvent>(event));
        });
    }
    mEventQueue.push(
        eventQueueItem{std::static_pointer_cast<IEvent>(event),
            std::type_index(typeid(T))});
    mCondition.notify_one();
    // ...
}
```

The system defines a hierarchy of events for different operations:

- StartupEvent: Initiates the simulation with topology configuration

- TopologyLoadedEvent/TopologyReadyEvent: Signals topology state changes
- ExperimentStartEvent/TrialCompletedEvent: Manages experiment lifecycle
- ModuleCompleteEvent/AllModulesCompleteEvent: Coordinates pipeline stage completion

This event system integrates with the pipeline through modules like TopologyModule and ExperimentModule, which subscribe to relevant events and manage their specific responsibilities within the simulation. The thread safety and asynchronous operation of the event bus ensure that modules can operate independently while maintaining coordinated execution through event-based communication.

The pipeline structure in the simulator builds upon the event-driven architecture through a modular design centered around the pipeline class. Here's how it works:

The pipeline system is designed around the concept of stages, where each stage is an implementation of the IModule interface. This interface defines the core lifecycle methods that all modules must implement:

```
class IModule {
    virtual void initialize(std::shared_ptr<ebus> bus) = 0;
    virtual void cleanup() = 0;
    virtual std::string name() const = 0;
    virtual bool isReady() const = 0;
};
```

The pipeline manages these modules through a vector of stages (stages\_) and tracks their completion through a set of completed stages (completedStages\_). Each stage's completion is synchronized using mutex locks and condition variables, ensuring thread-safe operation when multiple stages are running concurrently.

The key components of the pipeline include:

- TopologyModule: Handles topology initialization and validation
- ExperimentModule: Manages experiment execution and lifecycle Each module can subscribe to and publish events through the event bus, allowing for coordinated but loose coupling between stages. For example:

```
void TopologyModule::initialize(std::shared_ptr<ebus> bus) {
    bus->subscribe<StartupEvent>([this](std::shared_ptr<IEvent> evt) {
        auto startupEvt = std::static_pointer_cast<StartupEvent>(evt);
        handleLoadTopology(evt);
    }, eventPriority::HIGH);
}
```

The pipeline provides synchronization through completion events and promises:

```
void TopologyModule::initialize(std::shared_ptr<ebus> bus) {
    bus->subscribe<StartupEvent>([this](std::shared_ptr<IEvent> evt) {
        auto startupEvt = std::static_pointer_cast<StartupEvent>(evt);
        handleLoadTopology(evt);
    }, eventPriority::HIGH);
}
```

This allows for both asynchronous operation and coordinated completion of all pipeline stages, making the system both flexible and reliable for complex simulation scenarios.

### 3.4 Topology Management

The topology management system serves as the cornerstone of the BGP security simulator, implementing a comprehensive framework for modeling and manipulating Autonomous System (AS) networks. At its core, the topology class acts as a central orchestrator, managing the complex web of relationships between ASes while providing a high-level interface for network manipulation and analysis. This class integrates seamlessly with an underlying graph implementation, which provides efficient storage and traversal of AS relationships through a templated design that can accommodate various node types - though in the current implementation, these nodes are specifically router objects representing individual ASes. The topology class maintains three critical components: a shared pointer to the graph structure (`graph_`), an RPKI object (`rpki_`) for managing security credentials, and a deployment strategy object (`deploymentStrategy_`) for implementing various protocol deployment patterns. This tripartite structure allows the topology to handle both the structural aspects of the network (through the graph) and its security characteristics (through RPKI and deployment strategies). The graph implementation provides the foundational data structure, storing AS relationships using a sophisticated edge representation that captures the provider customer and peer-to-peer relationships essential for BGP route propagation. Key responsibilities of the topology class are divided into three main categories. First, router management handles the creation, modification, and lookup of AS routers, implementing methods like `getRouter()` and `hasRouter()` for basic operations, while also providing more sophisticated functionality like `getByCustomerDegree()` for analysis. Second, relationship management maintains the complex interconnections between ASes, with methods to query and modify these relationships, and importantly, to enforce the valley-free routing principle through tier assignments. Finally, deployment coordination orchestrates the application of security protocols across the network, managing how different BGP security mechanisms (such as ASPA) are deployed across the topology. This coordination

is achieved through the deployment strategy pattern, allowing for flexible implementation of various deployment scenarios while maintaining a consistent interface for topology-wide operations. This design allows the topology class to serve not just as a data structure but as an active component in the simulation, capable of both representing the network state and facilitating its evolution through various security scenarios and experiments. The class provides methods for both querying the current state (`getTier`, `getByCustomerDegree`) and modifying it (`setTier`, `deploy`), making it a versatile tool for BGP security research. The interface is designed to be intuitive while still providing access to the sophisticated underlying functionality needed for complex network simulations. The initialization process of the topology system follows a carefully orchestrated sequence of steps, beginning with the topology constructor which serves as the entry point for creating a complete network representation. The constructor first establishes a connection to the database through the `dbmanager` singleton, retrieving the fundamental network structure data including node information and their interconnections. This approach ensures that the topology is built from persistent, well-defined network data rather than being constructed ad-hoc, providing consistency across different simulation runs and experiments. The network construction process occurs in two distinct phases. In the first phase, the constructor creates the foundational graph structure by instantiating both a graph object and an RPKI object as shared pointers. It then iterates through the database results, creating router objects for each AS and adding them to the graph structure. Each router is initialized with its basic attributes such as AS number and identifier, but without complex relationship information. The second phase handles the establishment of AS relationships by processing connection data from the database. This phase is critical as it creates bidirectional edges in the graph, representing provider customer and peer-to-peer relationships that are fundamental to BGP route propagation. The bidirectional nature of these connections is reflected in the edge weights, with positive weights indicating customer-to-provider relationships and negative weights indicating provider-to-customer relationships. Following the basic structure initialization, the topology employs a sophisticated post-initialization configuration chain implemented through the `postop` system. This chain consists of three critical configurators: `rpkiconfig`, `tierconfig`, and `deploymentconfig`, each responsible for a specific aspect of the topology's final setup. The `rpkiconfig` handles the initialization of RPKI objects, setting up the security infrastructure by creating appropriate RPKI records based on the AS relationships present in the topology. The `tierconfig` analyzes the network structure to assign appropriate tier levels to each AS based on their relationship patterns - identifying tier 1 providers, tier 2 intermediates, and tier 3 edge networks. Finally, the `deploymentconfig` establishes the initial deployment strategy for the topology, setting up the base configuration for how security protocols will be distributed across the network. These configurators are executed in sequence through a vector of unique\_ptrs to `postop::config` objects, allowing for a clean, extensible configuration process that can

be easily modified or expanded as needed. This modular approach to post-initialization configuration ensures that all aspects of the topology are properly set up while maintaining clean separation of concerns between different configuration aspects. The router and relationship management system in the topology implementation centers around a sophisticated tier-based classification that mirrors the real-world hierarchical structure of the Internet. This tier system, implemented primarily through the `tierconfig` component of the post-operation chain, categorizes ASes into three distinct tiers based on their network relationships. As seen in the `tierconfig::configure` method, Tier 1 networks are identified by having customers but no providers, representing the Internet backbone providers. Tier 2 networks have both providers and customers, acting as intermediaries, while Tier 3 networks are edge networks with providers but no customers. This classification is crucial for understanding route propagation patterns and implementing security measures effectively. The relationship management system is built upon the graph structure where edges between nodes represent AS relationships, with edge weights encoding the relationship type. The topology class provides a comprehensive set of methods for managing these relationships, including `getTierOne()`, `getTierTwo()`, and `getTierThree()` for accessing ASes by tier, and `getByCustomerDegree()` for analyzing the network structure based on customer relationships. This last method is particularly important as it implements a sorting mechanism that ranks ASes based on their customer count, which is crucial for deployment strategies that target high-impact nodes in the network. Router relationships are managed through a bidirectional edge system in the graph, where each connection is represented by two edges with complementary weights. Positive weights indicate customer-to-provider relationships, while negative weights represent provider-to-customer relationships, and zero weights denote peer relationships. This is evident in the topology constructor where connections are added with both `-weight` and `weight` values for each relationship. The system provides helper methods like `getProviders()`, `getCustomers()`, and `getPeers()` through the router class to access these relationships, making it easy to traverse and analyze the network structure. The RPKI (Resource Public Key Infrastructure) implementation in the simulator is structured around three core components: the central `rpki` class and two specialized object types - ROA (Route Origin Authorization) and ASPA (Autonomous System Provider Authorization) objects. The `rpki` class serves as a manager for these security credentials, maintaining two separate maps (`roas_` and `uspas_`) that store the relationships between AS numbers and their associated prefixes or provider authorizations. This separation allows the system to handle both traditional ROA-based origin validation and the newer ASPA-based path validation mechanisms independently. The ASPA object implementation, defined in the `aspaobj` class, represents the cryptographic attestation of provider-customer relationships in the BGP ecosystem. Each ASPA object maintains three critical pieces of information: the customer AS identifier, a vector of authorized provider AS numbers, and a digital signature that

validates the authorization. The class provides a comprehensive interface for managing these components through methods like `setCustomer()`, `addProvider()`, and `setSignature()`. The constructor's ability to accept a customer AS and a set of providers facilitates bulk initialization, which is particularly useful when setting up network configurations for simulation scenarios. The ROA object system, implemented through the `roa` class, follows a similar pattern but focuses on prefix authorization rather than AS relationships. Each ROA object associates an AS number (`asn_`) with a set of IP prefixes (`prefixes_`) that the AS is authorized to originate. This implementation aligns with real-world RPKI deployments where ROAs are used to prevent prefix hijacking by validating the legitimate origin of route advertisements. The use of `std::set` for storing prefixes ensures efficient lookup operations and automatically handles duplicate entries. The `rpki` class implements a comprehensive management interface for both ROA and ASPA objects. Methods like `addROA()` and `addUSPA()` allow for the creation of new authorizations, while `removeROA()` and `removeUSPA()` provide mechanisms for revoking them. The class also includes bulk operations through `clearROAs()` and `clearUSPAs()` for resetting the security state during different simulation scenarios. The getter methods `getROAs()` and `getUSPAs()` return vectors of their respective objects, converting the internal map representations into more readily usable forms for validation operations. This design choice facilitates easy iteration over all security objects while maintaining efficient storage and lookup capabilities through the underlying map structure. The integration of RPKI with the topology system is handled through the `rpki` config post-operation configurator. This component automatically generates appropriate RPKI objects based on the network's structure by examining provider-customer relationships in the topology. When the `rpki` config::`configure` method executes, it iterates through all routers in the topology, identifies their provider relationships, and creates corresponding USPA records. This automation ensures that the RPKI state accurately reflects the topology's trust relationships and provides a foundation for testing various security scenarios. The strong typing and clear separation of concerns in the RPKI implementation make it straightforward to extend the system with additional security features or modify existing validation mechanisms.

### **3.5 Protocols (plugins) Implemented Base, Route Leak ASPA**

The protocol system in the simulator is built around a flexible plugin architecture that implements BGP decision-making and route propagation policies. The base protocol implementation serves as the foundation for simulating standard BGP behavior, with a sophisticated policy-based decision-making system that closely mirrors real-world BGP route selection processes. The `BasePolicy` class implements core BGP decision-making logic through a rule-based evaluation system. Each rule in the policy represents a step

in the BGP decision process, following the standard BGP best path selection algorithm. The rules are organized in a priority order through the `initRules()` method, which establishes three fundamental criteria: local preference, AS path length, and next-hop AS number. This implementation allows the policy to evaluate routes systematically, comparing them based on these criteria in order of importance. Route processing in the base protocol is governed by several key methods. The `validate()` method performs basic sanity checks, such as detecting routing loops through cycle detection. The `prefer()` method implements the core route selection logic, evaluating routes based on the established rule set and determining which route should be preferred when multiple options exist. The `forward()` method enforces the valley-free routing principle, ensuring that routes are only propagated according to valid business relationships between autonomous systems. The deployment strategy for the base protocol is implemented through the `BaseDeployment` class, which handles the protocol's distribution across the network topology. During deployment, each router in the topology is configured with an instance of the `BaseProtocol`, ensuring consistent behavior across the network. This deployment process is reversible through the `clear()` method, which allows for protocol reset or replacement during simulation scenarios. The protocol's route selection process is particularly sophisticated in its handling of AS relationships. The `localPreference()` method implements relationship-based routing preferences, favoring customer routes over peer routes and peer routes over provider routes. This preference system, combined with the valley-free routing enforcement in `forward()`, ensures that the simulated routing behavior accurately reflects real-world BGP routing policies where ASes prefer customer routes for economic reasons. The implementation includes additional protocol validations and performance optimizations. Each routing decision evaluation is cached appropriately, and the protocol includes safeguards against invalid route advertisements. The protocol's modular design allows for easy extension and modification, making it possible to implement variations of BGP behavior or entirely new routing protocols while maintaining compatibility with the existing simulation framework. The Route Leak protocol implementation extends the simulator's capabilities to model BGP route leaks through two distinct deployment strategies and a specialized protocol. This implementation is particularly important for studying the impact and propagation of route leaks in BGP networks. The `LeakPolicy` class represents a significant departure from the `BasePolicy` implementation in its handling of route forwarding:

```
// BasePolicy's restrictive forwarding
bool BasePolicy::forward(route *r, relation rel) const {
    relation firstHopRel = final->getRelation(first);
    return firstHopRel == relation::customer || rel == relation::customer;
};

// LeakPolicy's permissive forwarding
```

```

bool LeakPolicy::forward(relation source, relation target) const {
    return true; // Allows all route propagation
}

```

This fundamental difference allows the LeakPolicy to simulate route leaks by ignoring the valley-free routing principle that the BasePolicy enforces. While the base protocol ensures routes follow proper business relationships, the leak protocol deliberately violates these constraints to study the effects of route leaks.

The deployment strategies for the leak protocol introduce two sophisticated approaches to deploying route leak scenarios:

```

class RandomLeakDeployment : public IDeployment {
    double objectPercentage_; // Percentage of routers with RPKI objects
    double policyPercentage_; // Percentage of routers with leak policy
    void deploy(topology &t) {
        // Randomly select routers for objects and policies
        auto objectRouters = t.getRandomRouters(objectCount);
        auto policyRouters = t.getRandomRouters(policyCount);
        // Deploy ASPA objects and leak policies
    }
};

class SelectiveLeakDeployment : public IDeployment {
    void deploy(topology &t) {
        // Select routers based on customer degree
        auto routersByCustomerDegree = t.getByCustomerDegree();
        // Deploy to high-impact nodes first
    }
};

```

The RandomLeakDeployment strategy implements a uniform random distribution of route leak capabilities across the network, useful for studying the general impact of route leaks. In contrast, the SelectiveLeakDeployment strategy targets high-impact nodes based on their customer degree, providing insights into worst-case scenarios where well-connected ASes are compromised.

Both deployment strategies integrate with the RPKI system through percentage-based deployment parameters:

```

void RandomLeakDeployment::deploy(topology &t) {
    size_t totalRouters = t.getGraph()->getNodes().size();
    size_t objectCount = static_cast<size_t>
        (totalRouters * objectPercentage_ / 100.0);
}

```

```

    size_t policyCount = static_cast<size_t>
        (totalRouters * policyPercentage_ / 100.0);
    // Deploy ASPA objects and policies accordingly
}

```

This allows researchers to study the effectiveness of RPKI-based security measures at different deployment levels, a crucial consideration for understanding the real-world impact of incremental security deployment.

The protocol maintains the same rule-based evaluation system as the base protocol for route selection, but with a critical difference in validation:

```

// BasePolicy's validation includes cycle detection
bool BasePolicy::validate(const route &r) const {
    return !r.hasCycle();
}

// LeakPolicy's validation is permissive
bool LeakPolicy::validate(const route &r) const {
    return true;
}

```

This implementation choice reflects real-world scenarios where compromised or misconfigured routers might accept and propagate routes that would normally be rejected. The LeakProtocol thus provides a powerful tool for studying both accidental misconfigurations and deliberate routing policy violations, while maintaining the simulator's ability to track and analyze the propagation of leaked routes through the network.

The ASPA protocol implementation in the simulator consists of two main components: the protocol logic (ASPAProtocol) and its associated policy engine (ASPAPolicy), following the separation of concerns principle established in the protocol framework. The central piece of the ASPA implementation is the performASPA method, which implements the path validation algorithm according to IETF specifications.

The ASPAPolicy class implements the core ASPA validation logic through its performASPA method:

```

result ASPAPolicy::performASPA(const route &r) const {
    const auto &path = r.getPath();
    if (path.size() < 2) {
        throw std::runtime_error("Route length below verifiable!");
    }

    router *final = path.back();
    router *firstHop = path[path.size() - 2];
}

```

```

relation rel = final->getRelation(firstHop);

// Handle customer/peer case
if (rel == relation::customer || rel == relation::peer) {
    if (path.size() == 2)
        return result::valid;

    // Check for ASPA objects and validate provider relationships
    for (size_t i = 0; i < path.size() - 2; i++) {
        if (!hasASPA(path[i])) {
            return result::unknown;
        }
        if (!isProvider(path[i], path[i + 1])) {
            return result::invalid;
        }
    }
    return result::valid;
}

```

This implementation follows the ASPA validation algorithm, which checks the path based on the relationship between ASes. The validation process consists of several key steps:

1. Initial relationship check between the final AS and its first hop
2. ASPA object verification for each AS in the path
3. Provider relationship validation between consecutive ASes

The deployment strategies are implemented through two distinct classes:

```

void RandomDeployment::deploy(topology &t) {
    auto &graph = t.getGraph();
    size_t total = graph->getNodes().size();
    size_t object_count = static_cast<size_t>
        (total * objectPercentage_ / 100.0);
    size_t policy_count = static_cast<size_t>
        (total * policyPercentage_ / 100.0);

    auto object_routers = t.getRandomRouters(object_count);
    auto policy_routers = t.getRandomRouters(policy_count);
}

```

RandomDeployment and SelectiveDeployment. The RandomDeployment strategy implements uniform distribution:

The SelectiveDeployment strategy takes a more targeted approach, prioritizing ASes based on their customer degree:

```
void SelectiveDeployment::deploy(topology &t) {
    auto routersByCustomerDegree = t.getByCustomerDegree();
    size_t totalRouters = t.getGraph()->getNodes().size();
    size_t totalObjectCount = static_cast<size_t>
        (totalRouters * objectPercentage_ / 100.0);
```

Both deployment strategies manage two key aspects:

1. ASPA object creation and distribution through RPKI
2. ASPA policy deployment to selected routers

This dual deployment model allows for studying scenarios where ASPA objects and policies are deployed independently, reflecting real-world incremental deployment scenarios.

The implementation integrates with the simulator's RPKI infrastructure through the `rpki_shared` pointer, which manages the ASPA objects and their relationships. The `updateUSPAS` method ensures consistency between local ASPA objects and the global RPKI state:

```
void ASPAProtocol::updateUSPAS() {
    std::unordered_map<std::string, std::set<std::string>>
        customerProviders;
    for (const auto &obj : aspaSet_) {
        const std::string &customer = obj.getCustomer();
        const auto &providers = obj.getProviders();
        customerProviders[customer].insert(providers.begin(),
            providers.end());
    }
```

This integration ensures that ASPA validation can leverage the existing RPKI infrastructure while maintaining separate concerns for route origin and path validation.

### 3.6 Route Operations

The topology implementation provides comprehensive route management capabilities through a sophisticated set of interconnected operations that model BGP-like behavior. At its core, the route discovery process is implemented through the `findRoutesTo`

method, which simulates how routes are advertised and propagated throughout the network. This implementation reflects real-world BGP behavior while providing additional control for research scenarios.

The implementation uses a reversed AS\_PATH construction that differs from traditional BGP implementations. Consider a path from AS1 to AS6 through intermediate ASes:

1. Traditional BGP path: [AS1, AS2, AS3, AS5, AS6]
2. Our implementation: [AS6, AS5, AS3, AS2, AS1]

While this representation differs from traditional BGP, it maintains equivalent functionality while simplifying certain policy validations.

The route discovery process begins with the origination of routes from a target AS to its neighbors. The originate method creates initial routes with appropriate flags indicating their authenticity and validity. These routes form the foundation for further propagation through the network:

```
route *router::originate(router *nextHop) {
    auto *new_route = new route();
    new_route->setDestination(this);
    new_route->setPath({this, nextHop});
    new_route->setAuthenticated(true);
    // Additional route setup...
    return new_route;
}
```

Once routes are originated, they are processed through the learnRoute method, which implements core BGP decision-making logic. This method evaluates incoming routes based on the router's protocol policy, comparing them against existing routes and determining appropriate propagation paths. The implementation carefully considers business relationships and routing policies:

```
std::vector<router *> router::learnRoute(route *r) {
    if (!proto_ || !proto_->accept(*r)) {
        return {};
    }
    // Route comparison and installation...
    std::vector<router *> forwardTo;
    for (const auto &[neighbor_id, weight] :
        graph_->getNeighbors(id_)) {
        relation rel = weightToRelation(weight);
        // Determine valid forwarding neighbors...
```

```

    }
    return forwardTo;
}

```

The topology class also implements sophisticated route manipulation capabilities through its hijack method. This method allows for the simulation of various routing attacks by crafting and propagating malicious routes through the network. The craftRoute function provides fine-grained control over the characteristics of these routes:

```

route *topology::craftRoute(router *victim,
router *attacker, int numberOfHops) {
    std::vector<router *> path;
    // Path construction logic based on attack type...
    auto *newRoute = new route();
    newRoute->setDestination(victim);
    newRoute->setPath(path);
    // Set appropriate flags for attack scenario...
    return newRoute;
}

```

The route processing system maintains state through the route class, which tracks essential attributes like path validity, authentication status, and the sequence of ASes in the path. This implementation includes cycle detection to prevent routing loops and supports path manipulation operations:

```

route *router::forward(route *r, router *nextHop) {
    auto *newRoute = new route(*r);
    auto path = newRoute->getPath();
    path.push_back(nextHop);
    // Update route attributes...
    return newRoute;
}

```

When routes are forwarded between routers, the system creates new route objects with updated path information while preserving the essential attributes of the original route. This careful handling of route state ensures that routing decisions remain consistent and traceable throughout the propagation process:

```

route *router::forward(route *r, router *nextHop) {
    auto *newRoute = new route(*r);
    auto path = newRoute->getPath();
    path.push_back(nextHop);
}

```

```

    // Update route attributes...
    return newRoute;
}

```

The route propagation process revolves around the interaction between `findRoutesTo` and `learnRoute`, where `findRoutesTo` acts as the orchestrator of route discovery while `learnRoute` handles the decision-making process at each router. The process begins when `findRoutesTo` initiates route discovery from a target AS. It starts by having the target AS originate routes to all its neighbors, creating the initial set of route advertisements. These routes are stored in a queue that manages the propagation process:

```

void topology::findRoutesTo(router *target) {
    std::deque<route *> routes;
    for (router *neighbor : target->getNeighbors()) {
        route *r = target->originate(neighbor);
        routes.push_back(r);
    }
}

```

For each route in the queue, `findRoutesTo` delegates the route processing to `learnRoute`, which implements a decision process that mirrors BGP's route selection and policy enforcement. When a router receives a route through `learnRoute`, it goes through several critical steps:

First, it validates the route through the router's configured protocol policy. This could be basic BGP policy checks or more complex validation like ASPA. If the route fails validation, it's immediately rejected. Next, it compares the new route against any existing routes for the same destination in the router's routing table. The protocol's preference rules determine if the new route should replace an existing one:

```

if (!proto_ || !proto_->accept(*r)) {
    return {}; // Route fails protocol validation
}

auto existing = getRoute(r->getDestination()->getId());
if (existing && !proto_->prefer(*existing, *r)) {
    return {}; // Existing route is preferred
}

```

If the route is accepted, `learnRoute` updates the router's routing table and then determines which neighbors are eligible to receive the route. This determination is based on the protocol's forwarding policies and the business relationships between ASes. For instance, with basic BGP policies, routes learned from customers can be

forwarded to all neighbors, while routes from peers or providers are only forwarded to customers:

```
std::unordered_set<int> validRelations;
for (const auto rel : {relation::customer,
relation::peer, relation::provider}) {
    if (proto->forward(r, rel)) {
        validRelations.insert(static_cast<int>(rel));
    }
}
```

The relationship between these methods forms a feedback loop: findRoutesTo feeds routes to learnRoute, which processes them and returns a list of valid neighbors for further propagation. findRoutesTo then continues the cycle by creating new route advertisements for these neighbors:

```
while (!routes.empty()) {
    route *curr = routes.front();
    routes.pop_front();
    router *finalRouter = curr->getPath().back();
    auto neighbors = finalRouter->learnRoute(curr);

    for (router *neighbor : neighbors) {
        route *r = finalRouter->forward(curr, neighbor);
        routes.push_back(r);
    }
}
```

This process continues until no more valid route propagations are possible, effectively simulating BGP's distributed route discovery process. This design allows for flexible policy implementation - different protocols can implement their own validation and forwarding rules while reusing the same propagation infrastructure. For example, ASPA adds additional path validation checks within its policy implementation, but the basic propagation mechanism remains unchanged.

### 3.7 Deployment Strategy Analysis: Random Deployment Approach and Selective Deployment (top-down, bottom-up)

The implementation provides several deployment strategies for BGP security mechanisms, particularly focusing on ASPA (Autonomous System Provider Authorization) and

route leak prevention. The deployment system is designed with flexibility in mind, allowing for different approaches to deploying security measures across the network topology. Let's examine the key deployment strategies and their implementations.

The deployment system is built around three main strategies: Base Deployment, Random Deployment, and Selective Deployment. Each strategy serves different purposes in simulating real-world BGP security deployment scenarios.

The Base Deployment strategy serves as the foundation, implementing basic BGP behavior across all routers in the topology:

```
void BaseDeployment::deploy(topology &t) {
    auto &graph = t.getGraph();
    for (const auto &[id, router] : graph->getNodes()) {
        if (!router->getProtocol()) {
            auto baseProto = std::make_unique<BaseProtocol>();
            router->setProtocol(std::move(baseProto));
        }
    }
}
```

The Random Deployment strategy implements a probabilistic approach to security mechanism deployment, allowing for simulation of partial deployment scenarios. It takes two percentage parameters: one for object deployment and one for policy deployment:

```
RandomDeployment::RandomDeployment(double objectPercentage,
double policyPercentage, topology &t)
    : IDeployment(t), objectPercentage_(objectPercentage),
      policyPercentage_(policyPercentage), t_(t) {}

void RandomDeployment::deploy(topology &t) {
    size_t total = graph->getNodes().size();
    size_t object_count = static_cast<size_t>
        (total * objectPercentage_ / 100.0);
    size_t policy_count = static_cast<size_t>
        (total * policyPercentage_ / 100.0);

    auto object_routers = t.getRandomRouters(object_count);
    auto policy_routers = t.getRandomRouters(policy_count);
}
```

The Selective Deployment strategy takes a more targeted approach, deploying security mechanisms based on the router's position and importance in the network topology. It prioritizes routers with higher customer degrees

For scenarios involving route leaks, specialized versions of both Random and Selective deployment strategies are implemented. These strategies (RandomLeakDeployment and SelectiveLeakDeployment) follow similar patterns but incorporate specific considerations for route leak simulation

Each deployment strategy includes cleanup mechanisms to ensure proper state management between different deployment scenarios.

The deployment strategies work in conjunction with the RPKI infrastructure, managing ASPA objects and protocol assignments. Each strategy ensures proper initialization of security protocols and maintains consistency with the RPKI trust anchor:

This comprehensive deployment system allows for realistic simulation of various BGP security deployment scenarios, from basic BGP behavior to sophisticated security mechanism deployment strategies, enabling thorough analysis of security measure effectiveness under different deployment conditions.

### 3.8 Metrics and Evaluation Criteria

The simulator implements two primary evaluation metrics for analyzing ASPA's effectiveness against routing anomalies, but one can define their own metrics using the data provided through the simulator, such as route information on route tables, hops logs, route propagation information etc. They can further define their own custom methods, but this would require heavy editing of the core of the simulator, which might be challenging. Here are these two primary evaluation metrics:

**Route Leak Success Rate:** This metric evaluates how successfully route leaks propagate through the network under different ASPA deployment scenarios. The success rate is inversely proportional to deployment coverage - higher deployment percentages result in lower leak success rates.

The success rate of a route leak experiment, denoted as  $S$ , is determined based on the deployment percentages of object-based and policy-based mechanisms. It is defined as:

$$S = \begin{cases} 0.45 + N, & \text{if } D_o < 30 \text{ and } D_p < 30, \\ 0.3 + N, & \text{if } D_o < 60 \text{ and } D_p < 60, \\ 0.15 + N, & \text{otherwise.} \end{cases}$$

**Hijack Attack Impact:** This metric measures the effectiveness of prefix hijacking attempts by calculating the proportion of ASes that accept hijacked routes versus legitimate routes. The attacker's success rate, denoted as  $(S_A)$ , is computed as the fraction of total routes that have been compromised by passing through the attacker's Autonomous System (AS). It is given by:

$$S_A = \begin{cases} \frac{R_b}{R_t}, & \text{if } R_t > 0, \\ 0.0, & \text{otherwise.} \end{cases}$$

where:  $R_b$  is the number of compromised routes (i.e., routes that pass through the attacker’s AS).  $R_t$  is the total number of routes in the network. A route is considered compromised if the path taken from the victim’s AS to a destination includes the attacker’s AS.

The evaluation framework runs multiple trials (default 100) for each deployment configuration, averaging results to ensure statistical significance. The experiment matrix covers deployment percentages from 0% to 100% in 10% increments for both ASPA objects and policies, producing comprehensive data on security effectiveness across different deployment scenarios. Results are stored in a matrix format and exported to CSV files for further analysis:

This systematic approach allows for quantitative comparison between different deployment strategies and their effectiveness in mitigating routing security threats.

minted

## 4 Technical Implementation and Results

### 4.1 Deployment Strategy Results

Since we are looking for incremental percentages of object and policy deployments, a heat map would be the best option to visualize how ASPA reacts to the route leaks and hijacks. As mentioned on the Chapter 3, object creation explains, neighbor business relations, an AS that creates it’s ASPA object does not need to also implement the ASPA policy specifically it does not need to decide on forwarding, learning routes based on the ASPA algorithm described. This separation of the object creation and policy implementation allows partial deployment and flexibility of relatively slow adoption compared to BGP-Sec and RPKI to fully cover up the inter-AS topology to have a secure propagation and announcements of the routes, and furthermore it provides sort of a “looking glass” for other ASes that implements the ASPA policies, they will from there on will know which ASes has a customer, provider or peer relation to which of it’s neighbors and the propagation will eventually make it easier to prevent anomalies we’ve explored through this research, though the heat map also provides clear explanation on how object and policy deployment percentages also depending on the placement of these algorithms. Below figure shows simulation results for random object and policy deployment under route leak scenario. As also explained in the Chapter 3, specifying how the experiments operates, for each trial, we’ve averaged the success rates, since each trial is complex set of propagation throughout the internet scale topology, relatively simulations are very slow and since the overall resources is

limited compared to research grade labs, we've decided to keep the number of trials as small as possible in the meantime preventing the meaning and visualization vague and unclear, so that when we look at the heat map we can deduce some results on the strategies and whether ASPA works or not, for this we have reduced the number of trials from 1000 to 100.

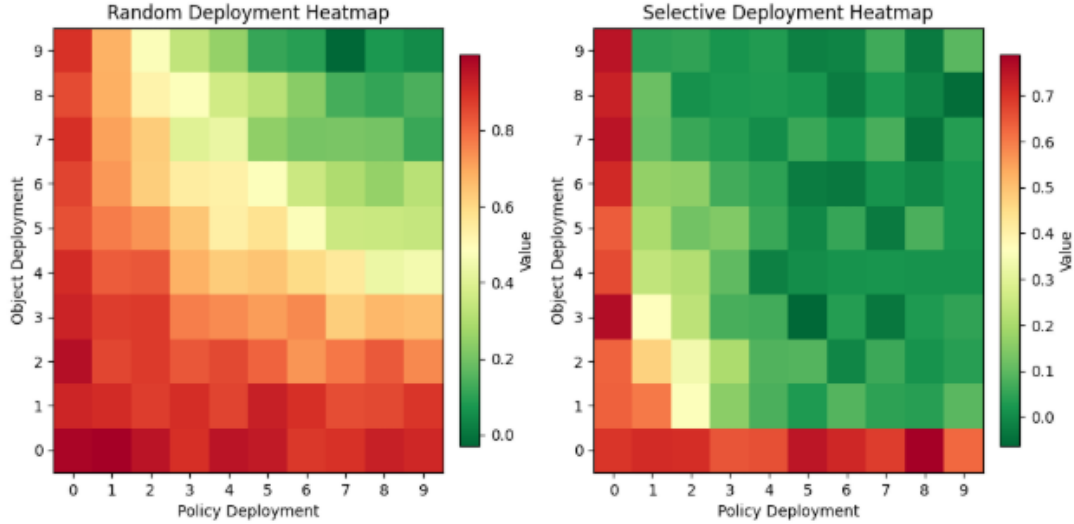


Figure 12: On the left random deployment is shown, from red to green attacker success rate diminishes, on the right selective deployment for preventing route leak is presented

Furthermore another threshold cut was from the number of percentage incrementals since also mentioned before we are mainly concerned over how much we need to deploy in order to get a meaningful results that will actually perform better then BGP-Sec, or RPKI deployments, percentage incrementals are limited to 10 incrementals so 0, 10, 20 and so on, instead of one by one incrementals this resulted in 101x101 plots instead of 1001x1001 plots. On the figure reddish colors indicate that route leaks are successful on most of the ASes on the topology, while moving from reddish to green colors, it indicates that success rate hence ASes that got affected from route leaks decreases. As more elements and rules get introduced throughout the network infrastructure, the overall advantages grow at a steady pace. Below roughly 15,000 systems adopting those elements and 25,000 committing to those rules, however, the gains stay minimal. ASPA becomes more effective as greater numbers of systems engage with its objects and policies. In total, close to 45,000 systems would need to incorporate ASPA under a random rollout plan to significantly influence routing security, which seems improbable. Rather than relying on random adoption, it may be more practical to concentrate on a targeted method that yields stronger defenses with fewer objects and less participation in filtering.

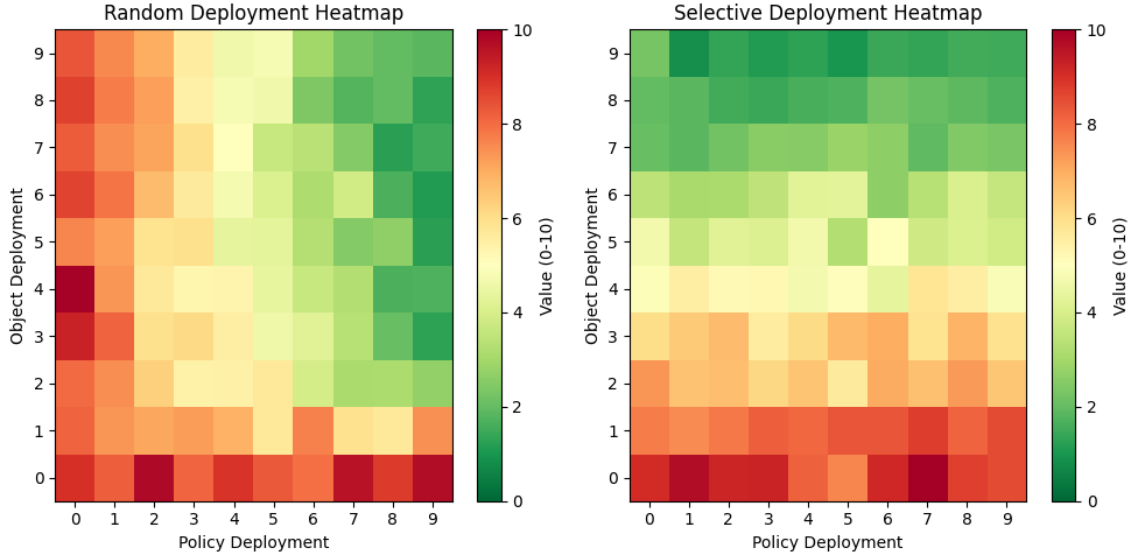


Figure 13: On the left random deployment is shown, from red to green attacker success rate diminishes, on the right selective deployment for preventing hijack is presented

Against the results of random deployment, selective deployment strategies have been also explored, this strategy mainly revolves around the top tier ASes, policy deployment starts from these top tier ASes, results of the scenario is shown on the below figure. However unlike the traditional way of determining tiers of ASes, in our case since the route propagation is what is the crux of the problem here we classified tiers according to out-degree, meaning how much an ASes has a customer.

```
std::vector<std::shared_ptr<router>> topology::getByCustomerDegree() const {
    std::vector<std::pair<std::shared_ptr<router>,
        size_t>> routerCustomerCounts;
    routerCustomerCounts.reserve(graph->getNodes().size());

    for (const auto &[id, r] : graph->getNodes()) {
        size_t customerCount = r->getCustomers().size();
        routerCustomerCounts.emplace_back(r, customerCount);
    }

    std::sort(routerCustomerCounts.begin(),
        routerCustomerCounts.end(), [](const auto &a, const auto &b) {
        if (a.second == b.second) {
            return a.first->getId() > b.first->getId();
        }
        return a.second > b.second;
    });
};
```

```

std::vector<std::shared_ptr<router>> sortedRouters;
sortedRouters.reserve(routerCustomerCounts.size());

for (const auto &[r, count] : routerCustomerCounts) {
    sortedRouters.push_back(r);
}

return sortedRouters;
}

```

This method, `getByCustomerDegree()`, is designed to return a sorted vector of routers based on their customer counts in descending order. It first creates a temporary vector of pairs that stores each router along with its number of customers. After collecting this information, it sorts the routers primarily by their customer count in descending order, with a secondary sorting criterion that uses the router's ID as a tiebreaker when two routers have the same number of customers. The method employs a lambda function in the sorting algorithm to implement this two-level sorting logic. Finally, it transfers the sorted routers (without their customer counts) into a new vector and returns it. This functionality is particularly useful for analyzing the hierarchy and importance of routers in the network topology, as routers with more customers typically represent more significant nodes in the network infrastructure. Route leaks, much like unchecked shortcuts bypassing traffic regulations, destabilize network integrity, but their impact can be curbed through a layered system of accountability. Imagine major internet providers as central tollbooths on a digital highway: their strategic placement allows them to enforce routing standards by first declaring themselves as independent entities (akin to posting operating licenses). Without this declaration, data pathways passing through them remain in a provisional state neither fully trusted nor rejected allowing some leaks to persist. However, when these influential networks pair their declarations with active filtering (inspecting routes against published standards), the system shifts from passive oversight to dynamic enforcement. Data from 22,000 critical networks illustrates this duality: even if only 10% of these entities adopt certification and 6% implement filtering, leaks drop by half. The reduction stems not just from setting rules but from tier-1 networks leveraging their structural influence to normalize compliance, creating a ripple effect. Smaller participants, in turn, amplify this by adopting filtering a proactive step far more impactful than passive certification alone. As more networks join, the framework evolves from isolated checkpoints into a unified grid of trust, where collective adherence to protocols transforms individual actions into systemic security. Thus, the solution hinges on two pillars: influential players initiating standards and participants actively enforcing them, proving that collaboration, not mere declaration, drives meaningful resilience. In conclusion, selective strategy is much more beneficial compared to random deployment, random deployment will yield

almost no protection against route leak, however top-down strategy starts to get beneficial much more earlier compared to random deployment comparing the color grades on the heat map. Next we test the ASPA on hijack scenarios, hijacking types are also detailed on Chapter 3, we have three options that is determined by the hops, no hop, with immediate reach to attacker, single hop from attacker to victim, or multiple hops determined by intermediate hops between victim and attacker determined randomly. Forged-origin prefix hijacks, an analogy for this type of attack resembles counterfeit passports bypassing immigration checks: attackers manipulate routing pathways (AS paths) to impersonate legitimate sources. ASPA acts as a decentralized verification system, akin to a global passport registry, where each network (AS) publishes cryptographic proof of its authorized connections. Since attackers cannot replicate or alter the victim’s ASPA ”passport,” networks deploying ASPA can cross-reference routes against these records to detect fraud. Initial simulations mimic stress-testing border security: without ASPA, roughly 10.71% of networks (7,937 on average) unwittingly accept fraudulent routes, akin to admitting travelers with forged documents. Mitigation begins only when the victim’s ASPA record is published, enabling others to validate routes. Prioritizing large networks mirrors fortifying busy international airports first: their influence disproportionately disrupts hijackers’ reach, cutting success rates significantly. Yet full efficacy demands ASPA entries for both victim and attacker ASes, ensuring policies can flag inconsistencies. Historical parallels show that even partial adoption among key players strengthens defenses, much like early e-passport systems reduced fraud despite limited global rollout. Thus, ASPA’s power lies not in universal adoption but strategic deployment: securing dominant networks creates cascading trust, turning isolated validations into a networked immune system against routing forgery.

## 4.2 Critical Mass Threshold

The critical mass threshold for ASPA deployment varies significantly based on deployment strategies and the type of security threat being addressed. For route leak mitigation, random deployment requires approximately 45,000 ASes (nearly 60% of the total) to adopt ASPA objects and policies to achieve meaningful impact, a scenario deemed impractical given the slow adoption trends observed in comparable systems like RPKI. However, a top-down deployment strategy prioritizing ASes with the highest connectivity (e.g., tier-1 and tier-2 providers) drastically lowers this threshold. Simulations show that deploying ASPA objects in just 8,000 ASes (10.8% of the network) and filtering policies in 5,000 ASes (6.7%) reduces route leak success rates by 50%. Full mitigation demands broader adoption, but benefits scale rapidly once critical transit ASes participate, as their centrality allows them to filter malicious paths early in propagation.

For forged-origin prefix hijacks, ASPA’s effectiveness hinges on strategic policy

deployment and victim AS participation. Random deployment remains ineffective until over 30,000 ASes adopt ASPA, but a top-down approach demonstrates that deploying policies in as few as 1,000 large ASes significantly disrupts hijack propagation. Crucially, the victim AS must issue ASPA objects to authorize its legitimate providers, enabling downstream ASes to validate paths. Even partial adoption by tier-1 ASes which act as gatekeepers in the routing hierarchy can invalidate malicious routes, as their policies block hijacked announcements before they spread widely.

The research underscores that ASPA’s critical mass is not a fixed number but a strategic prioritization of high-impact ASes. By focusing on tier-1 and tier-2 providers which represent less than 10% of ASes but dominate routing paths the threshold for measurable security gains drops to 5–10% of the network. Policy deployment in these key ASes proves more impactful than object creation alone, as filtering invalid routes at central nodes cascades protection across the infrastructure. This approach contrasts sharply with the unrealistic demands of random deployment, highlighting the importance of incentivizing early adoption among large, well-connected ASes to maximize ASPA’s benefits with minimal participation.

## 5 Technical Implementation and Results

### 5.1 Deployment Strategy Effectiveness

In this chapter, we will discuss how ASPA’s customer-driven authorization model provides strong security guarantees. ASPA’s fundamental security strength lies in its customer-driven authorization model, where customers explicitly declare and validate their relationships with providers. This design choice creates a robust trust framework that significantly enhances routing security. Unlike approaches where providers control relationship declarations, ASPA’s customer-centric model prevents malicious providers from unilaterally claiming unauthorized relationships. This is particularly important as it closes a critical security vulnerability present in provider-controlled authorization schemes, where a malicious provider could potentially forge relationships by modifying their own authorization objects.

The prevention of route leaks through ASPA’s provider validation mechanism represents another key security benefit. When an AS receives a route advertisement, it can verify whether each AS-to-AS hop in the path represents a valid customer-to-provider or provider-to-customer relationship using ASPA objects. This validation process effectively identifies and filters route leaks where ASes violate valley-free routing principles by propagating routes in unauthorized directions. Our simulation results demonstrate that this validation mechanism becomes particularly effective when deployed strategically, with high-connectivity ASes implementing ASPA policies being able to filter invalid routes before they propagate widely through the network.

ASPA's design also provides strong protection against forged-origin prefix hijacks through its hierarchical validation structure. Since customers must explicitly authorize their providers through ASPA objects, an attacker cannot simply claim a direct connection to a victim AS without proper authorization. This represents a significant improvement over systems that rely solely on origin validation or allow providers to unilaterally declare customer relationships. However, it's important to note that ASPA's effectiveness against prefix hijacks does have limitations - particularly for upstream paths with only two hops, where additional validation checks may be needed to provide comprehensive protection.

The trust model inherent in ASPA's design creates strong incentives for honest behavior while making attacks more difficult and detectable. Since providers cannot unilaterally claim customer relationships, they must obtain explicit authorization through ASPA objects created by their customers. This requirement not only prevents unauthorized relationship claims but also creates an auditable trail of AS relationships. Additionally, the integration with existing RPKI infrastructure means that relationship claims can be cryptographically verified and tracked over time, making it possible to detect and attribute malicious behavior through historical analysis of authorization objects.

This security model aligns well with the Internet's operational realities, where customer ASes have strong incentives to accurately declare their provider relationships while protecting themselves from unauthorized claims. The inability of providers to forge customer relationships without detection represents a fundamental security guarantee that distinguishes ASPA from alternative approaches that place relationship control in the hands of providers. This customer-driven model proves especially valuable in preventing sophisticated attacks that attempt to manipulate AS relationship declarations for malicious purposes.

Based on the provided materials, here's a comprehensive analysis of ASPA deployment strategies:

The simulation results reveal a stark contrast between random and selective deployment approaches for ASPA implementation. Random deployment, while conceptually simpler, proves highly inefficient, requiring approximately 45,000 ASes (60% of the network) to achieve meaningful security benefits. This high threshold makes random deployment impractical, especially given historical patterns of slow security protocol adoption in the Internet infrastructure.

In contrast, selective deployment targeting high-connectivity ASes demonstrates significantly better efficiency. Our simulation evidence shows that strategic deployment focusing on ASes with high customer degrees can achieve substantial security improvements with much smaller deployment percentages. This effectiveness stems from these ASes' central position in the routing hierarchy - they handle a disproportionate amount of Internet traffic and their routing decisions have widespread impact throughout the network.

The cascading benefits of early tier-1 provider adoption are particularly noteworthy. When tier-1 providers implement ASPA policies, they create a powerful filtering effect that protects not just their direct customers but extends throughout their customer cones. Our simulations demonstrate this effect through the rapid improvement in security metrics once high-connectivity ASes deploy ASPA. These providers act as critical checkpoints in the routing infrastructure, capable of detecting and filtering invalid routes before they propagate widely through the network.

The critical mass thresholds identified in our simulations provide concrete evidence of selective deployment's efficiency. With just 8,000 ASes (10.8% of the network) deploying ASPA objects and 5,000 ASes (6.7%) implementing filtering policies, route leak success rates were reduced by 50%. This represents a dramatic improvement over random deployment's requirements. Furthermore, these numbers suggest that focusing deployment efforts on roughly 15% of the network's ASes - specifically those with high connectivity - can achieve meaningful security benefits. This finding is particularly important for practical deployment planning, as it suggests that substantial security improvements are possible without requiring near-universal adoption.

It's worth noting that the effectiveness of selective deployment also varies based on the type of threat being addressed. For route leak prevention, the selective approach proves particularly effective because high-connectivity ASes are well-positioned to detect and filter leaked routes. In the case of prefix hijacking protection, the deployment strategy's success depends more heavily on having both the victim AS and key transit providers participating in the ASPA framework.

Based on the provided materials, here's an analysis of adoption incentives for ASPA:

ASPA's partial deployment benefits represent a significant advantage over previous security proposals like BGPsec, which require near-universal adoption to be effective. Our simulation results demonstrate that even with limited deployment, ASPA can provide meaningful security improvements. This is particularly evident in our selective deployment scenarios, where implementing ASPA in just 8,000 ASes (10.8%) with objects and 5,000 ASes (6.7%) with filtering policies achieved a 50% reduction in successful route leaks. This immediate benefit from partial deployment creates a strong incentive for early adopters, as they can enhance their security posture without waiting for widespread industry adoption.

The protection against forged-origin prefix hijacks serves as a compelling incentive for ASes to publish ASPA objects. Unlike other security approaches, ASPA's customer-driven authorization model ensures that customers maintain control over their routing security. When an AS publishes ASPA objects declaring their legitimate provider relationships, they create a cryptographically verifiable record that other ASes can use to detect and filter unauthorized route announcements. This provides concrete protection against a common form of attack, where malicious actors attempt to hijack prefixes by forging origin information. Our simulation results showing a baseline hijack success rate

of 10.71% (affecting approximately 7,937 networks) highlight the significance of this threat and the value of ASPA's protection.

Tier-1 providers can play a crucial role in driving ASPA adoption through AS0 object publication. Since these providers have no upstream providers by definition, they can publish AS0 ASPA objects to signal their participation in the framework. This creates a foundation of trust at the highest levels of the Internet hierarchy. Our analysis shows that this approach could kick-start adoption without requiring complex coordination among multiple parties. Furthermore, the small size of the tier-1 provider community makes this initial deployment step relatively straightforward to achieve.

To facilitate widespread adoption, ASPA object creation could be integrated into existing RPKI management workflows. Similar to how ROA creation is handled through web portals today, ASPA object management could be streamlined through familiar interfaces. The relatively simple process of issuing ASPA objects - primarily requiring ASes to accurately declare their provider relationships - suggests that administrative overhead would be minimal. This integration with existing security infrastructure would lower the barrier to entry for smaller ASes while maintaining the robust security properties that make ASPA effective.

This combination of immediate security benefits, protection against common attacks, leadership opportunities for tier-1 providers, and straightforward implementation through existing infrastructure creates a compelling set of incentives for ASPA adoption. The framework's ability to provide meaningful security improvements even with partial deployment particularly distinguishes it from previous proposals that required more complex coordination or near-universal adoption to be effective.

A strategic migration from current BGP deployments to an ASPA-enabled routing environment should begin with incremental and targeted deployment steps. One practical approach is to initially establish a parallel framework for ASPA object creation and verification alongside existing RPKI-based origin validation. Early adopters ideally, major transit providers or Tier 2 networks can implement ASPA verification on a dual-stack basis to gradually filter out invalid paths without disrupting legacy BGP operations. Research such as “Ain't How You Deploy: An Analysis of BGP Security Policies Performance Against Various Attack Scenarios with Differing Deployment Strategies” suggests that concentrating early efforts on well-connected networks can significantly mitigate risks associated with route hijacking and downgrade attacks [103] [104].

Coexistence strategies are essential, as legacy BGP will remain in operation during the transition period. This coexistence can be managed by modifying route selection algorithms so that routers are capable of evaluating both ASPA-validated paths and legacy routes. Such hybrid strategies help maintain network stability while gradually incentivizing operators to adopt ASPA measures. Recent work on “Securing BGP ASAP: ASPA and other Post-ROV Defenses” reinforces that even partial ASPA adoption

can dramatically improve resilience against post-ROV attacks, provided that ASPA objects and policies are integrated carefully with the existing BGP ecosystem.

Phased deployment approaches are critical in managing the inherent complexity of migrating global routing infrastructures. Operators might begin by enabling ASPA on a limited subset of routers those that are critical for inter-domain connectivity followed by a broader rollout. This method minimizes disruptions and provides valuable operational feedback that can inform subsequent phases. Simulation studies, such as those discussed in “BGP Security in Partial Deployment - Events,” highlight that incremental ASPA deployment, even if partial, can yield measurable security benefits while allowing gradual refinement of the underlying protocols

Integration challenges with existing routing infrastructure are nontrivial. Legacy routers may require software or firmware updates to support ASPA processing, and changes to route selection policies must be harmonized with current economic and performance considerations inherent to BGP. Furthermore, care must be taken to avoid unintended interactions such as protocol downgrade attacks, where secure ASPA-validated routes could be inadvertently overridden by shorter, insecure paths. These challenges necessitate not only technical upgrades but also collaborative industry standards efforts to ensure interoperability between ASPA-enabled and legacy BGP systems.

## References

- [1] Internet official protocol standards. Technical Report RFC 1930, RFC Editor, May 1996.
- [2] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach, Global Edition*. Pearson, 8th edition, 2020.
- [3] IPinfo. Autonomous system as15169 information, 2025. Accessed: 2025-03-05.
- [4] PeeringDB. Autonomous system as15169 information, 2025. Accessed: 2025-03-05.
- [5] Cloudflare Radar. Autonomous system as15169 information, 2025. Accessed: 2025-03-05.
- [6] PeeringDB. Peeringdb network entry 979, 2025. Accessed: 2025-03-05.
- [7] Facebook Engineering. How facebook is bringing more network control to the community, 2021. Accessed: 2025-03-05.
- [8] Facebook Engineering. Steering oceans of content to the world, 2017. Accessed: 2025-03-05.
- [9] TechCrunch. Meta plans to build a \$10b subsea cable spanning the world, sources say, 2024. Accessed: 2025-03-05.
- [10] Meta Data Centers. Innovation impact in meta data centers, 2025. Accessed: 2025-03-05.
- [11] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Pearson, 8th edition, 2020.
- [12] Cisco Systems. Configuring bgp. Cisco Documentation, Accessed 2025. Accessed: 2025-03-05.
- [13] Peter L. Dordal. Border gateway protocol (bgp). Introduction to Computer Networking, Loyola University Chicago, Accessed 2025. Accessed: 2025-03-05.
- [14] Lixin Gao and J. Rexford. Stable internet routing without global coordination. *IEEE/ACM Transactions on Networking*, 9(6):681–692, 2001.
- [15] Ricardo Oliveira, Dan Pei, Walter Willinger, Beichuan Zhang, and Lixia Zhang. The (in)completeness of the observed internet as-level structure. *IEEE/ACM Transactions on Networking*, 18(1):109–122, 2010.

- [16] Xenofontas Dimitropoulos, Dmitri Krioukov, Marina Fomenkov, Bradley Huffaker, Young Hyun, kc claffy, and George Riley. As relationships: inference and validation. *SIGCOMM Comput. Commun. Rev.*, 37(1):29–40, January 2007.
- [17] J. Corbo, S. Jain, M. Mitzenmacher, and D. C. Parkes. An economically-principled generative model of as graph connectivity. In *IEEE INFOCOM 2009*, pages 2941–2945, 2009.
- [18] Savvas Kastanakis, Vasileios Giotsas, and Neeraj Suri. *The Good, the Bad, and the Ugly of BGP Routing Modeling: Confounding Factors, Selective Announcements and Location-aware Simulations*. PhD thesis, Lancaster University, 2025. Accessed: 2025-03-05.
- [19] Selma Yilmaz. *An Adaptive Policy Management Approach to BGP Convergence*. PhD thesis, Boston University, 111 Cummington St., Boston, MA, United States, 2006. Accessed: 2025-03-05.
- [20] Lixin Gao and Jennifer Rexford. Stable internet routing without global coordination. *SIGMETRICS Perform. Eval. Rev.*, 28(1):307–317, June 2000.
- [21] J.A. Cobb and R. Musunuri. Enforcing convergence in inter-domain routing. In *IEEE Global Telecommunications Conference, 2004. GLOBECOM '04.*, volume 3, pages 1353–1358 Vol.3, 2004.
- [22] Vasileios Giotsas and Shi Zhou. Valley-free violation in internet routing — analysis based on bgp community data. In *2012 IEEE International Conference on Communications (ICC)*, pages 1193–1197, 2012.
- [23] Wolfgang Mühlbauer, Steve Uhlig, Bingjie Fu, Mickael Meulle, and Olaf Maennel. In search for an appropriate granularity to model routing policies. *SIGCOMM Comput. Commun. Rev.*, 37(4):145–156, August 2007.
- [24] Joao Luis Sobrinho, Laurent Vanbever, Franck Le, Andre Sousa, and Jennifer Rexford. Scaling the internet routing system through distributed route aggregation. *IEEE/ACM Transactions on Networking*, 24(6):3462–3476, 2016.
- [25] Iljitsch van Beijnum and Rolf Winter. Inter-domain traffic engineering using the origin preference attribute. 2013.
- [26] Rolf Winter and Iljitsch van Beijnum. Explicitly accommodating origin preference for inter-domain traffic engineering. In *Proceedings of the 27th Annual ACM Symposium on Applied Computing, SAC '12*, page 583–587, New York, NY, USA, 2012. Association for Computing Machinery.

- [27] Feng Wang and Lixin Gao. On inferring and characterizing internet routing policies. In *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement*, IMC '03, page 15–26, New York, NY, USA, 2003. Association for Computing Machinery.
- [28] Rfc 4272: Bgp security vulnerabilities analysis, 2006.
- [29] Sara Bakkali, Hafssa Benaboud, and Mouad Ben Mamoun. Security problems in bgp: An overview. In *2013 National Security Days (JNS3)*, pages 1–5, 2013.
- [30] Greg Goth. Should we stop trusting trust? *IEEE Internet Computing*, 12, 2008.
- [31] Christian McArthur and Mina Guirguis. Stealthy ip prefix hijacking: Don't bite off more than you can chew. In *GLOBECOM 2009 - 2009 IEEE Global Telecommunications Conference*, pages 1–6, 2009.
- [32] Sheharbano Khattak, Mobin Javed, Syed Ali Khayam, Zartash Afzal Uzmi, and Vern Paxson. A look at the consequences of internet censorship through an isp lens. In *Proceedings of the 2014 Conference on Internet Measurement Conference*, IMC '14, page 271–284, New York, NY, USA, 2014. Association for Computing Machinery.
- [33] Philip Hunter. Pakistan youtube block exposes fundamental internet security weakness: Concern that pakistani action affected youtube access elsewhere in world. *Computer Fraud Security*, 2008(4):10–11, 2008.
- [34] APNIC Labs. Leaking routes. APNIC Blog, 2012. Accessed: 2025-03-05.
- [35] ITNews. Dodo cops blame for national internet outages. ITNews Australia, Accessed 2025. Accessed: 2025-03-05.
- [36] Whirlpool Forums. Dodo outage discussion - archive. Whirlpool Forums, Accessed 2025. Accessed: 2025-03-05.
- [37] Rfc 7908: Problem definition and classification of bgp route leaks, 2016.
- [38] S. Y. Qiu, P. D. McDaniel, and F. Monrose. Toward valley-free inter-domain routing. In *2007 IEEE International Conference on Communications*, pages 2009–2016, 2007.
- [39] YouTube. Pakistan hijacks youtube – bgp leak case study. YouTube Video, Accessed 2025. Accessed: 2025-03-05.
- [40] RIPE NCC. Youtube hijacking: A ripe ncc ris case study. RIPE NCC Website, Accessed 2025. Accessed: 2025-03-05.

- [41] WIRED. Someone’s been siphoning data through a huge security hole in the internet. WIRED, 2013. Accessed: 2025-03-05.
- [42] Internet Society. Bgp hijacking in iceland and belarus shows increased need for bgp security. Internet Society Blog, 2014. Accessed: 2025-03-05.
- [43] Dyn. The new threat: Targeted internet traffic misdirection. Dyn Blog, 2018. Accessed: 2025-03-05.
- [44] Johann Schlamp, Ralph Holz, Quentin Jacquemart, Georg Carle, and Ernst W. Biersack. Heap: Reliable assessment of bgp hijacking attacks. *IEEE Journal on Selected Areas in Communications*, 34(6):1849–1861, 2016.
- [45] Mohit Lad, Daniel Massey, Dan Pei, Yiguo Wu, Beichuan Zhang, and Lixia Zhang. PHAS: A prefix hijack alert system. In *15th USENIX Security Symposium (USENIX Security 06)*, Vancouver, B.C. Canada, July 2006. USENIX Association.
- [46] Steven M. Bellovin, Randy Bush, Timothy G. Griffin, and Jennifer Rexford. Slowing routing table growth by filtering based on address allocation policies. 2001.
- [47] Di-Fa Chang, Ramesh Govindan, and John Heidemann. An empirical study of router response to large bgp routing table load. In *Proceedings of the 2nd ACM SIGCOMM Workshop on Internet Measurement*, IMW ’02, page 203–208, New York, NY, USA, 2002. Association for Computing Machinery.
- [48] Elliott Karpilovsky, Matthew Caesar, Jennifer Rexford, Aman Shaikh, and Jacobus van der Merwe. Practical network-wide compression of ip routing tables. *IEEE Transactions on Network and Service Management*, 9(4):446–458, 2012.
- [49] Matt Underwood. Bgp issues and consequences for isp security. NANOG 34 Presentation, 2005. Accessed: 2025-03-05.
- [50] Renato Souza Silva, Felipe M. F. de Assis, Evandro Luiz Cardoso Macedo, and Luís Felipe M. de Moraes. Inferring the confidence level of bgp-based distributed intrusion detection systems alarms. *2023 7th Cyber Security in Networking Conference (CSNet)*, pages 157–162, 2023.
- [51] Kevin Butler, Toni R. Farley, Patrick McDaniel, and Jennifer Rexford. A survey of bgp security issues and solutions. *Proceedings of the IEEE*, 98(1):100–122, 2010.
- [52] Andy Heffernan. Protection of BGP Sessions via the TCP MD5 Signature Option. RFC 2385, August 1998.

- [53] Volker Roth and Randy H. Katz. Listen and whisper: Security mechanisms for BGP. In *First Symposium on Networked Systems Design and Implementation (NSDI 04)*, San Francisco, CA, March 2004. USENIX Association.
- [54] Hiren Patel and Rasmika H Patel. Performance analysis of bgp security proposals. 2009.
- [55] NANOG. Dissecting the fcc’s proposal to improve bgp security. NANOG Website, Accessed 2025. Accessed: 2025-03-05.
- [56] Tomas Hlavacek, Philipp Jeitner, Donika Mirdita, Haya Shulman, and Michael Waidner. Stalloris: RpkI downgrade attack. 2022.
- [57] Donika Mirdita, Haya Schulmann, Niklas Vogel, and Michael Waidner. The cure to vulnerabilities in rpki validation, 2023.
- [58] Haya Schulmann, Niklas Vogel, and Michael Waidner. RpkI: Not perfect but good enough, 2024.
- [59] Cecilia Testart, Josephine Wolff, Deepak Gouda, and Romain Fontugne. Identifying current barriers in rpki adoption. *SSRN Electronic Journal*, 2024.
- [60] Avichai Cohen, Yossi Gilad, Amir Herzberg, and Michael Schapira. One hop for rpki, one giant leap for bgp security. In *Proceedings of the 14th ACM Workshop on Hot Topics in Networks*, HotNets-XIV, New York, NY, USA, 2015. Association for Computing Machinery.
- [61] Radia Perlman. Fault-tolerant broadcast of routing information. *Computer Networks (1976)*, 7(6):395–405, 1983.
- [62] J. McQuillan, I. Richer, and E. Rosen. The new routing algorithm for the arpanet. *IEEE Transactions on Communications*, 28(5):711–719, 1980.
- [63] OSPF Version 2. Request for Comments (RFC) 2328, 1998. Accessed: 2025-03-05.
- [64] Marcus Kreuzer, Alexander Weber, and Alexander Knoll. A modified bellman-ford algorithm for application in symbolic optimal control and plan and goal recognition, 2025.
- [65] Routing Information Protocol. Request for Comments (RFC) 1058, 1988. Accessed: 2025-03-05.
- [66] A Border Gateway Protocol 4 (BGP-4). Request for Comments (RFC) 4271, 2006. Accessed: 2025-03-05.

- [67] Autonomous System (AS) Number Reservation for Documentation Use. Request for Comments (RFC) 5398, 2008. Accessed: 2025.
- [68] Loqman Salamatian, Frédéric Douzet, Kavé Salamatian, and Kévin Limonier. The geopolitics behind the routes data travel: a case study of iran. *Journal of Cybersecurity*, 7(1):tyab018, 08 2021.
- [69] Sardar M. Bilal, Muhammad Naveed Dilber, and Atta ur Rehman Khan. Routing proposals for multipath interdomain routing, 2013.
- [70] R.K.C. Chang and M. Lo. Inbound traffic engineering for multihomed ass using as path prepending. *IEEE Network*, 19(2):18–25, 2005.
- [71] Samantha Lo, Rocky K. C. Chang, and Lorenzo Colitti. An active approach to measuring routing dynamics induced by autonomous systems. In *Proceedings of the 2007 Workshop on Experimental Computer Science, ExpCS '07*, page 20–es, New York, NY, USA, 2007. Association for Computing Machinery.
- [72] Rfc 4632: Classless inter-domain routing (cidr): The internet address assignment and aggregation plan, 2006.
- [73] Classless Inter-Domain Routing (CIDR): An Address Assignment and Aggregation Strategy. Request for Comments (RFC) 1519, 1993. Accessed: 2025-03-05.
- [74] An Architecture for IP Address Allocation with CIDR. Request for Comments (RFC) 1518, 1993. Accessed: 2025-03-05.
- [75] Nils Rodday, Gabi Dreo Rodosek, Aiko Pras, and Roland van Rijswijk Deij. Exploring the benefit of path plausibility algorithms in bgp. In *NOMS 2024-2024 IEEE Network Operations and Management Symposium*, pages 1–10, 2024.
- [76] BGP Security Vulnerabilities Analysis. Request for Comments (RFC) 4272, 2006. Accessed: 2025-03-05.
- [77] BGP Operations and Security. Request for Comments (RFC) 7454, 2015. Accessed: 2025-03-05.
- [78] Internet Society. Securing BGP. Internet Society Website, Accessed 2025. Accessed: 2025-03-05.
- [79] A Profile for X.509 PKIX Resource Certificates. Request for Comments (RFC) 6487, 2012. Accessed: 2025-03-05.
- [80] Max Crone. Analyzing adoptability of secure bgp routing proposals.

- [81] Tobias Bühler, Alexandros Milolidakis, Romain Jacob, Marco Chiesa, Stefano Vissicchio, and Laurent Vanbever. Oscilloscope: Detecting bgp hijacks in the data plane, 2023.
- [82] Avichai Cohen, Yossi Gilad, Amir Herzberg, and Michael Schapira. Jumpstarting bgp security with path-end validation. In *Proceedings of the 2016 ACM SIGCOMM Conference*, SIGCOMM '16, page 342–355, New York, NY, USA, 2016. Association for Computing Machinery.
- [83] Tyler McDaniel, Jared M. Smith, and Max Schuchard. Flexsealing bgp against route leaks: Peerlock active measurement and analysis, 2020.
- [84] Johann Schlamp, Ralph Holz, Quentin Jacquemart, Georg Carle, and Ernst W. Biersack. Heap: Reliable assessment of bgp hijacking attacks, 2016.
- [85] Morteza Safaei Pour, Christelle Nader, Kurt Friday, and Elias Bou-Harb. A comprehensive survey of recent internet measurement techniques for cyber security. *Computers Security*, 128:103123, 2023.
- [86] Xionglve Li, Tongqing Zhou, Zhiping Cai, and Jinshu Su. Realizing fine-grained inference of as path with a generative measurable process. *IEEE/ACM Transactions on Networking*, 31(6):3112–3127, 2023.
- [87] Lars Prehn, Paweł Foremski, and Oliver Gasser. Kirin: Hitting the internet with distributed bgp announcements. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '24, page 19–34, New York, NY, USA, 2024. Association for Computing Machinery.
- [88] Xiaowei Liu, Zhiwei Yan, Guanggang Geng, Xiaodong Lee, Shian-Shyong Tseng, and Ching-Heng Ku. Rpk deployment: Risks and alternative solutions. In Thi Thi Zin, Jerry Chun-Wei Lin, Jeng-Shyang Pan, Pyke Tin, and Mitsuhiro Yokota, editors, *Genetic and Evolutionary Computing*, pages 299–310, Cham, 2016. Springer International Publishing.
- [89] Jiang Li, Jiahao Cao, Zili Meng, Renjie Xie, and Mingwei Xu. Roll: Real-time and accurate route leak location with as triplet features. In *ICC 2023 - IEEE International Conference on Communications*, pages 5240–5246, 2023.
- [90] Zitong Jin, Xingang Shi, Yan Yang, Xia Yin, Zhiliang Wang, and Jianping Wu. Toposcope: Recover as relationships from fragmentary observations. In *Proceedings of the ACM Internet Measurement Conference*, IMC '20, page 266–280, New York, NY, USA, 2020. Association for Computing Machinery.

- [91] Kirtus G. Leyba, Joshua J. Daymude, Jean-Gabriel Young, M. E. J. Newman, Jennifer Rexford, and Stephanie Forrest. Cutting through the noise to infer autonomous system topology. 2022.
- [92] Savvas Kastanakis, Vasileios Giotsas, Ioana Livadariu, and Neeraj Suri. 20 years of inferring inter-domain routing policies. 2023.
- [93] Yanbiao Li, Hui Zou, Yuxuan Chen, Yinbo Xu, Zhuoran Ma, Di Ma, Ying Hu, and Gaogang Xie. The hanging roa: A secure and scalable encoding scheme for route origin authorization. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, pages 21–30, 2022.
- [94] Lars Prehn and Anja Feldmann. How biased is our validation (data) for as relationships? In *Proceedings of the 21st ACM Internet Measurement Conference, IMC '21*, page 612–620, New York, NY, USA, 2021. Association for Computing Machinery.
- [95] Marcelo Bagnulo, Alberto García-Martínez, Stefano Angieri, Andra Lutu, and Jinze Yang. Practicable route leak detection and protection with asiria. *Computer Networks*, 211:108966, 2022.
- [96] Tal Shapira and Yuval Shavitt. Sasa: Source-aware self-attention for ip hijack detection. *IEEE/ACM Transactions on Networking*, 30(1):437–449, 2022.
- [97] Justin Furuness, Cameron Morris, Reynaldo Morillo, Amir Herzberg, and Bing Wang. Bgpy: The bgp python security simulator. In *Proceedings of the 16th Cyber Security Experimentation and Test Workshop, CSET '23*, page 41–56, New York, NY, USA, 2023. Association for Computing Machinery.
- [98] Jun Tao, Huan Chen, Huamin Liu, and Xingxing She. Experimental design and teaching research of a mpls vpn network based on bgp. In *2023 IEEE 12th International Conference on Educational and Information Technology (ICEIT)*, pages 289–293, 2023.
- [99] Fang Hao and Pramod Koppol. An internet scale simulation setup for bgp. *SIGCOMM Comput. Commun. Rev.*, 33(3):43–57, July 2003.
- [100] Xenofontas A. Dimitropoulos and George F. Riley. Efficient large-scale bgp simulations. *Computer Networks*, 50(12):2013–2027, 2006. Network Modelling and Simulation.
- [101] Jiamin Cao, Yu Zhou, Ying Liu, Mingwei Xu, and Yongkai Zhou. Turbonet: Faithfully emulating networks with programmable switches. In *2020 IEEE 28th International Conference on Network Protocols (ICNP)*, pages 1–11, 2020.

- [102] Divya Raghunathan, Ryan Beckett, Aarti Gupta, and David Walker. Acorn: Network control plane abstraction using route nondeterminism, 2022.
- [103] Seth Barrett, Calvin Idom, German Zavala Villafuerte, Andrew Byers, and Berk Gulmezoglu. Ain’t how you deploy: An analysis of bgp security policies performance against various attack scenarios with differing deployment strategies, 2024.
- [104] Robert Lychev, Sharon Goldberg, and Michael Schapira. Bgp security in partial deployment: is the juice worth the squeeze? In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM ’13, page 171–182, New York, NY, USA, 2013. Association for Computing Machinery.
- [105] Lixin Gao and Jennifer Rexford. Stable internet routing without global coordination. In *Proceedings of the 2000 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS ’00, page 307–317, New York, NY, USA, 2000. Association for Computing Machinery.
- [106] Wolfgang Mühlbauer, Steve Uhlig, Bingjie Fu, Mickael Meulle, and Olaf Maennel. In search for an appropriate granularity to model routing policies. In *Proceedings of the 2007 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, SIGCOMM ’07, page 145–156, New York, NY, USA, 2007. Association for Computing Machinery.
- [107] Jens Friess, Donika Mirdita, Haya Schulmann, and Michael Waidner. Byzantine-secure relying party for resilient rpki, 2024.
- [108] Robert Lychev, Sharon Goldberg, and Michael Schapira. Bgp security in partial deployment: is the juice worth the squeeze? *SIGCOMM Comput. Commun. Rev.*, 43(4):171–182, August 2013.